



UNIVERSIDAD DE BUENOS AIRES
FACULTAD DE CIENCIAS EXACTAS Y NATURALES

Clasificación arancelaria con
procesamiento de lenguaje natural
(borrador)

Tesis presentada para optar al título de Magister en
Explotación de Datos y Descubrimiento de Conocimiento

Tesista: Ing. Santiago S. Tedoldi
Director: Dr. Bruno Bianchi

Buenos Aires, 10 de noviembre de 2025

Contexto

En los últimos años, el comercio internacional se ha acelerado alcanzando volúmenes récord y una creciente fragmentación de las cadenas globales de valor. En 2024, el volumen de comercio alcanzó un récord cercano a los 33 billones de dólares, un 3,3 % superior al año previo [1]. Este dinamismo convive con una creciente diversidad de bienes (alimentos, químicos, fármacos, dispositivos tecnológicos, entre otros), lo que eleva la complejidad de identificar correctamente cada producto en los regímenes de comercio exterior.

Para ordenar ese universo de bienes, existe el Sistema Armonizado de Designación y Codificación de Mercancías (HS, por sus siglas en inglés), desarrollado por la Organización Mundial de Aduanas (OMA/WCO), y adoptado por más de 190 países y bloques económicos como base para sus aranceles y estadísticas de comercio [2]. El HS (versión 2022) comprende:

- 97 categorías de bienes definidas a dos dígitos (capítulos o HS02)
- 1.244 grupos de bienes definidos a cuatro dígitos (partidas o HS04)
- 5.612 subgrupos definidos a seis dígitos (subpartidas o HS06)

En Argentina, además, la declaración a consumo utiliza el código SIM de 11 dígitos (subpartida HS06 + NCM + dígitos nacionales), con un universo de ~30 mil códigos locales. Por ejemplo, el código SIM de teléfonos inteligentes es 8517.13.00.000C [3]:

Código Argentino	SIM				
Código MERCOSUR	NCM				
Subpartida HS	HS06				
Partida HS	HS04				
Capítulo HS	HS02				
Teléfonos inteligentes	85	17	13	00	000 C

Cabe destacar que, en cualquier jurisdicción o país, la correcta clasificación es crítica: una inexactitud puede acarrear multas relevantes y desvíos en tratamientos arancelarios o intervenciones de otros organismos del estado. Para Argentina, una clasificación incorrecta se enmarca como una infracción por declaración inexacta del Artículo 954 del Código Aduanero.

Por otro lado, de nuevo en el plano global, las empresas y los operadores del comercio internacional, responsables de clasificar los productos, están experimentando una transformación de la forma de operar de la mano del e-commerce. Según la UNCTAD, el número de compradores en línea superó los 2,3 mil millones en 2021 y las ventas de comercio electrónico de 43 economías alcanzaron casi 27 billones de dólares en 2022, alrededor de un 10 % más que en 2021 [4].

Este crecimiento se refleja en un aumento significativo de envíos de bajo valor, pedidos fraccionados y descripciones comerciales breves, heterogéneas y, a veces, ambiguas. Para las administraciones de aduanas y para las empresas que participan en el comercio exterior, esto implica un volumen de decisiones de clasificación sin precedentes, muchas veces a partir de textos cortos y ruidosos.

La OMA, a través del proyecto BACUDA, subraya que la clasificación HS sigue siendo una de las tareas más difíciles del entorno aduanero, altamente dependiente de la pericia

humana y fuente frecuente de errores y controversias [5] [6]. Los sistemas tradicionales —manuales, basados en reglas o en búsquedas keyword-driven— requieren incorporar herramientas inteligentes que permitan priorizar casos, sugerir códigos probables y documentar mejor el razonamiento, asistiendo la labor de los clasificadores humanos en lugar de sustituirla.

En este escenario, la explotación de datos, el aprendizaje automático y el procesamiento de lenguaje natural (NLP) abren la posibilidad de modelar descripciones de mercaderías mediante técnicas de embeddings (Word2Vec, Doc2Vec y FastText) [7] [8] [9] y modelos de lenguaje basados en Transformers como BERT [10] y sus variantes (por ejemplo, DistilBERT) [11] para construir clasificadores automáticos o asistidos que “lean” descripciones comerciales y sugieran códigos HS o NCM con buena precisión.

Pese a estos avances, la aplicación sistemática de estas técnicas sobre universos amplios de mercaderías —que cubran la mayor parte de la nomenclatura y escenarios operativos realistas— sigue siendo un desafío abierto, especialmente en contextos como el comercio electrónico transfronterizo y las operaciones minoristas.

Problema abordado y motivación

La tesis busca asistir la clasificación arancelaria de mercaderías a nivel HS04 a partir de descripciones comerciales en texto libre. El caso de estudio utiliza un dataset de ~500 mil pares (descripción, código) en inglés, proveniente del proyecto BACUDA [12]. El problema es multiclase, con miles de etiquetas y alto desbalanceo. La pregunta central es si, con estos datos, es posible entrenarse un modelo recomendador de códigos con precisión útil para la práctica aduanera.

Objetivos

Objetivo general: Entrenar un modelo de clasificación que logre recomendar códigos HS04 partiendo de descripciones comerciales de mercaderías.

Objetivos específicos:

- Analizar el efecto de distintas estrategias de preprocesamiento textual, incluyendo normalización, remoción de stopwords e incorporación de n-gramas, para determinar en qué condiciones mejoran o perjudican la clasificación.
- Entrenar y evaluar modelos baseline basados en Doc2Vec y FastText, con el fin de cuantificar el valor incremental del uso de modelos basados en Transformers.
- Analizar los errores de clasificación para identificar patrones de confusión por capítulo y partida, así como casos especialmente difíciles.

Antecedentes

Los primeros antecedentes relevantes provienen del desarrollo de métodos de representación distribuida de palabras y documentos. Word2Vec introdujo el uso de redes neuronales simples para aprender vectores de palabras a gran escala [7], mientras que Doc2Vec extendió esta idea a secuencias más largas, como oraciones y documentos [8]. Luego, para abordar la problemática de ruido y errores de tipeo, FastText presentó una forma de procesar subpalabras (particiones) o subtokens [9].

Estos enfoques permitieron construir clasificadores basados en embeddings a partir de texto libre, y han sido aplicados como baseline en tareas de categorización de productos tanto en comercio electrónico como en análisis de catálogos. Con el desarrollo de la arquitectura Transformers [13], BERT [10] y sus variantes (por ejemplo, DistilBERT) [11] incorporan representaciones contextuales bidireccionales, se han convertido en métodos muy difundidos para la clasificación de texto en dominios generales, partiendo de modelos pre-entrenados con distintas técnicas de transfer learning [14].

En el terreno específico de la clasificación arancelaria, la OMA y varias administraciones aduaneras han comenzado a experimentar con modelos de Machine Learning (ML) y Deep Learning (DL). El proyecto BACUDA de la OMA desarrolló un modelo neuronal para recomendación de códigos HS a partir de descripciones comerciales, que se integra en una plataforma demostrativa de recomendación de códigos y visualización de resultados [5] [12]. Sin embargo, la información pública disponible no expone detalles reproducibles sobre su desempeño o sobre conjuntos de datos abiertos.

Un desarrollo en Corea del Sur propone un modelo basado en KoELECTRA (una variante de BERT entrenado en coreano) para asistir a la clasificación HS, alcanzando un 95,5 % de precisión top-3 (aciertos en los primeros tres códigos sugeridos) sobre 129.084 casos históricos en 265 subpartidas [15]. Este trabajo demuestra la viabilidad de usar modelos de lenguaje combinados con sistemas de recuperación para referenciar la clasificación a textos del HS o a casos previos, pero su universo está acotado a un subconjunto de subpartidas (265 subpartidas es menos del 5 % de las comprendidas en el HS).

De forma contemporánea, el estudio del BID de Marra de Artiñano et al. evalúa distintos algoritmos (modelos clásicos de ML y GPT-3.5) sobre descripciones de productos agrícolas y alimentarios provenientes de organismos públicos (incluida información aduanera y del USDA) [16]. Los autores encuentran que los modelos tradicionales funcionan bien dentro del conjunto de datos en que se entrenan, incluso superando al LLM, pero su rendimiento cae abruptamente al transferirlos a casos de otros países/jurisdicciones. En cambio, GPT-3.5 exhibe una performance relativamente estable entre conjuntos y niveles de agregación HS, con accuracies del orden del 60–90 % según la cantidad de dígitos (HS04, HS02, etc.) definidos como objetivo.

En una línea complementaria, Gholamian et al. abordan el problema de clasificación de productos para comercio y compliance en un entorno industrial real, evaluando el uso de LLMs con in-context learning frente a enfoques supervisados tradicionales [17]. Sus resultados muestran que los LLMs son robustos ante descripciones incompletas o perturbadas, pero (de nuevo) el estudio se centra en un conjunto de productos acotados, sin cubrir toda la amplitud de la nomenclatura HS.

Más recientemente, en el ámbito del comercio electrónico y el retail, se trabajó con un conjunto de datos jerárquico de más de un millón de productos del marketplace chino JD.com y los autores proponen HFT-BERT, un modelo BERT con fine-tuning jerárquico

que mejora la clasificación de productos en una estructura de tres niveles de categorías [18]. Aunque trabaja con categorías de e-commerce y no con códigos HS, ilustra cómo los modelos basados en Transformers pueden aprovechar estructuras jerárquicas para mejorar la clasificación de catálogos grandes.

Por último, a fines del 2024 se propuso un benchmark comparativo de herramientas comerciales y modelos basados en ML/AI para clasificación HTS (HS de 10 dígitos) en el contexto de los Estados Unidos [19]. Aunque el documento aún no fue publicado por una revista, se trata de una iniciativa en la dirección acertada. Cabe también destacar que la autoría es unipersonal y de alguien vinculada a Tarifflo (una de las compañías dueña de una de las herramientas evaluadas), razones que motivan a una revisión/validación de la metodología empleada.

En síntesis, la literatura reciente muestra avances significativos en: (i) representaciones de texto y modelos de lenguaje aplicados a clasificación de productos, (ii) prototipos de clasificación HS en administraciones aduaneras, y (iii) aplicaciones de LLMs para escenarios de comercio y compliance. Sin embargo, estos estudios tienden a concentrarse en subconjuntos específicos de mercaderías, contextos lingüísticos particulares o herramientas aún en fase de consolidación. Esto deja abierta una oportunidad para desarrollar y validar clasificadores basados en NLP que cubran un espectro más amplio de mercaderías, que puedan integrarse a arquitecturas híbridas con LLMs (por ejemplo, esquemas RAG o de re-ranking experto) y que sean evaluados con métricas transparentes sobre datos representativos del comercio real, incluyendo e-commerce transfronterizo.

Descripción del Conjunto de Datos

Contenido y origen

El conjunto de datos utilizado en este trabajo está formado por tuplas con información de mercaderías sometidas al comercio internacional (descripción_comercial, código_tarifario) en idioma inglés, en un total de 500 mil ítems. Estos datos provienen de material de un programa de entrenamiento profesional en el marco de una capacitación internacional en Corea del Sur, del proyecto BACUDA de la Organización Mundial de Aduanas [5] [12].

Características

Muestra Variada: El conjunto de datos presenta una diversidad considerable en cuanto a tipos de productos y mercaderías, clasificadas en 1244 códigos a 4 dígitos y 5612 códigos a 6 dígitos.

Desbalanceo: Predominan las descripciones de productos relacionados con máquinas y material eléctrico/electrónico, como también de vehículos automotores.

Análisis exploratorio

Como se ha anticipado, la muestra a trabajar resulta particularmente sencilla, con solo dos columnas (descripción_comercial, código_tarifario). Por esta razón, para la etapa exploratoria, además de evaluar cuestiones típicas de calidad (nulos, duplicados, outliers) y distribución/frecuencias, corresponde trabajar con técnicas de ingeniería de variables.

En términos generales, el análisis exploratorio busca entender cómo se compone el dataset, que mercaderías abarca, que información contiene y que limitaciones se deben que considerar de cara al objetivo.

Ingeniería de variables

Para mejorar el análisis exploratorio del dataset, y contemplar toda la información “escondida” en el mismo, se procede a:

- Desagregar la variable HS06 en otros códigos subyacentes.
- Buscar fuentes externas que complementen a los datos disponibles.
- Aplicar técnicas de encoding de texto, buscando representaciones latentes.
- Reducir dimensiones (PCA, UMAP y/o t-SNE).

Definición del target/ variable objetivo

Tal como se describe en el Contexto, la desagregación del código tarifario HS06, en códigos regionales o nacionales, tiene fundamento de negocio y permite considerar futuros cambios en la variable objetivo. Por ejemplo, contar con 6 dígitos (HS06) permite ejercitar la clasificación también en HS04 y HS02 (4 y 2 dígitos) sin mayores dificultades. A menor cantidad de dígitos, menos cantidad de clases, lo que también simplifica la tarea del algoritmo.

Sin embargo, cabe destacar que, en el comercio internacional, una clasificación debe tener tantos dígitos como la normativa lo requiera. Al contrario de quitar dígitos, sumarlos en el código implica conocer detalles de la mercadería que pueden requerir investigar en catálogos y detalles técnicos, inspeccionar muestras, realizar análisis de laboratorio y demás tareas complejas, según el caso.

Perfilado y análisis de los datos

En una revisión rápida del dataset crudo, se analizan nulos, duplicados y frecuencias respecto a códigos de capítulo (HS02), partida (HS04) y subpartida (HS06):

Valores nulos por columna:

Columna	Valores nulos
HS06	0
HS04	0
HS02	0
GOODS_DESCRIPTION	0

Frecuencias por HS02, TOP10:

HS02	Cantidad	Frecuencia relativa	Frecuencia acumulada
84	54.901	20,5%	20,5%
85	33.571	12,54%	33,04%
87	28.476	10,63%	43,67%
73	16.173	6,04%	49,71%
39	12.218	4,56%	54,28%
90	11.611	4,34%	58,61%
82	7.972	2,98%	61,59%
94	7.921	2,96%	64,55%
40	7.526	2,81%	67,36%
83	4.285	1,60%	68,96%

Frecuencias por HS02, BOTTOM10:

HS02	Cantidad	Frecuencia relativa	Frecuencia acumulada
41	22	0,01%	99,96%
81	19	0,01%	99,97%
45	19	0,01%	99,97%
05	15	0,01%	99,98%
80	15	0,01%	99,98%
50	11	0,00%	99,99%
14	11	0,00%	99,99%
78	10	0,00%	100,0%
51	6	0,00%	100,0%
43	5	0,00%	100,0%

Frecuencias por HS04, TOP10:

HS04	Cantidad	Frecuencia relativa	Frecuencia acumulada
8708	8.524	3,18%	3,18%
8703	7.341	2,74%	5,92%
7318	5.700	2,13%	8,05%
8536	5.487	2,05%	10,10%
8482	4.895	1,83%	11,93%
8421	4.593	1,72%	13,65%
8431	3.910	1,46%	15,11%
8483	3.819	1,43%	16,53%
8481	3.562	1,33%	17,86%
8714	3.485	1,30%	19,16%

Frecuencias por HS06, TOP10:

HS06	Cantidad	Frecuencia	Frecuencia
------	----------	------------	------------

		relativa	acumulada
870323	3.869	1,44%	1,44%
848280	2.924	1,09%	2,54%
871120	2.779	1,04%	3,57%
731815	2.632	0,98%	4,56%
870322	2.247	0,84%	5,40%
870899	2.074	0,77%	6,17%
950300	1.988	0,74%	6,91%
940540	1.874	0,70%	7,61%
848180	1.857	0,69%	8,31%
901890	1.836	0,69%	8,99%

Respecto a las frecuencias por HS04 y HS06, BOTTOM10, los mínimos están en 1 única muestra por código.

Agregaciones de las descripciones

Para cada descripción, se evalúan el largo en cantidad de palabras y de caracteres y un indicador de tokenización, que mide cómo se tokeniza la descripción en relación con la cantidad de palabras.

- GOODS_DESCRIPTION_len_words
- GOODS_DESCRIPTION_len_chars
- subtokenization_indicator

El tokenizer utilizado es un algoritmo de Hugging Face Transformers, llamado "distilbert-base-uncased". Este algoritmo corresponde al modelo pre-entrenado que se utiliza luego para procesar los textos en estudio:

<https://huggingface.co/distilbert/distilbert-base-uncased>

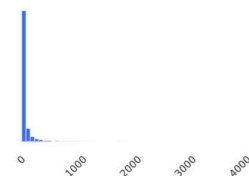
Luego, se procesaron las variables en agregaciones estadísticas (conteo, suma, mínimos, máximos, medios, medianos y desviación estándar) para agrupaciones en HS06, HS04 y HS02. A continuación, se muestran algunas distribuciones, con datos básicos de las variables:

HS06_count

Real number (R)

High correlation

Distinct	430	Minimum	1
Distinct (%)	10.3%	Maximum	3869
Missing	0	Zeros	0
Missing (%)	0.0%	Zeros (%)	0.0%
Infinite	0	Negative	0
Infinite (%)	0.0%	Negative (%)	0.0%
Mean	63.909308	Memory size	65.5 KiB



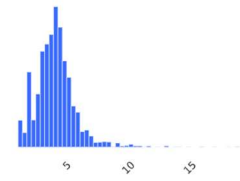
El desbalance en las clases/códigos se correlaciona con variables como: HS06_count, GOODS_DESCRIPTION_len_words_sum y GOODS_DESCRIPTION_len_chars_sum, con valores medios lejos de la mediana y distribuciones de colas largas.

GOODS_DESCRIPTION_len_words_mean

Real number (R)

High correlation

Distinct	1692	Minimum	1
Distinct (%)	40.4%	Maximum	19
Missing	0	Zeros	0
Missing (%)	0.0%	Zeros (%)	0.0%
Infinite	0	Negative	0
Infinite (%)	0.0%	Negative (%)	0.0%
Mean	4.0534472	Memory size	65.5 KiB



A su vez, surgen distribuciones más acampanadas para variables como el largo promedio (tanto para palabras como para caracteres), cuando se agrupa en los distintos códigos HS06, HS04 y HS02.

GOODS_DESCRIPTION_len_words_std

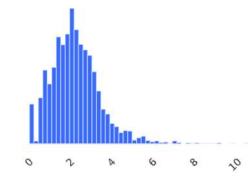
Real number (R)

High correlation

Missing

Zeros

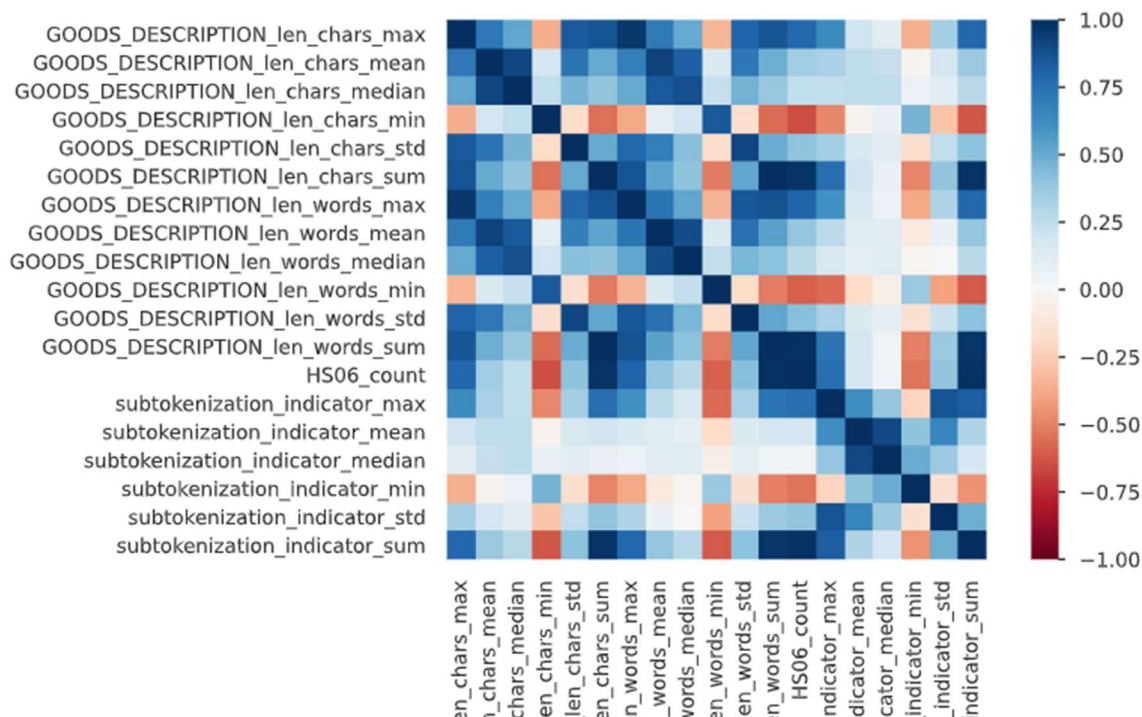
Distinct	2713	Minimum	0
Distinct (%)	74.3%	Maximum	10.843585
Missing	540	Zeros	100
Missing (%)	12.9%	Zeros (%)	2.4%
Infinite	0	Negative	0
Infinite (%)	0.0%	Negative (%)	0.0%
Mean	2.2447975	Memory size	65.5 KiB



Finalmente, la desviación estándar del largo de las descripciones tiene una distribución similar a un Xi cuadrado, tanto para el largo en palabras como en caracteres. Se destaca que hay códigos con una única muestra, lo que genera faltantes al cálculo la desviación estándar.

En términos de correlaciones, no hay grandes sorpresas, ya que se correlacionan variables que agregan cantidades como:

- HS06_count
- GOODS_DESCRIPTION_len_words_sum
- GOODS_DESCRIPTION_len_chars_sum
- GOODS_DESCRIPTION_len_words_max
- GOODS_DESCRIPTION_len_chars_max



Nomenclatura HS06 en inglés

Como desarrollo del dataset original, se trabajó con la nomenclatura de los códigos de HS06 (HS versión 2022) que consiste en descripciones genéricas para cada código y que, además puede separarse en capítulos, partidas y subpartidas.

Primeros 5 códigos HS06:

HS06	English code sub-heading	HS04	HS02
010120	Live horses, asses, mules and hinnies. && - Horses :	0101	01
010121	Live horses, asses, mules and hinnies. && - Horses : && -- Pure-bred breeding animals	0101	01
010129	Live horses, asses, mules and hinnies. && - Horses : && -- Other	0101	01
010130	Live horses, asses, mules and hinnies. && - Asses	0101	01
010190	Live horses, asses, mules and hinnies. && - Other	0101	01

Últimos 5 códigos HS06:

HS06	English code sub-heading	HS04	HS02
961590	Combs, hair-slides and the like; hairpins, curling pins, curling grips, hair-curlers and the like, other than those of heading 85.16, and parts thereof. && - Other	9615	96
961610	Scent sprays and similar toilet sprays, and mounts and heads therefor; powder-puffs and pads for the application of cosmetics or toilet preparations. && - Scent sprays and similar toilet sprays, and mounts and heads therefor	9616	96
961620	Scent sprays and similar toilet sprays, and mounts and heads therefor; powder-puffs and pads for the application	9616	96

	of cosmetics or toilet preparations. && - Powder-puffs and pads for the application of cosmetics or toilet preparations		
970110	Paintings, drawings and pastels, executed entirely by hand, other than drawings of heading 49.06 and other than hand- painted or hand-decorated manufactured articles; collages and similar decorative plaques. && - Paintings, drawings and pastels	9701	97
970190	Paintings, drawings and pastels, executed entirely by hand, other than drawings of heading 49.06 and other than hand- painted or hand-decorated manufactured articles; collages and similar decorative plaques. && - Other	9701	97

IMPORTANTE: Encontramos que 4.5 % de los datos no encuentran su descripción de nomenclatura, lo que puede deberse a que se están utilizando una nomenclatura diferente a la usada en el registro de los datos.

Embeddings con DistilBERT y PCA

Para evaluar las descripciones se procede a utilizar un modelo pre-entrenado, disponible en Hugging Face, llamado "distilbert-base-uncased":

<https://huggingface.co/distilbert/distilbert-base-uncased>

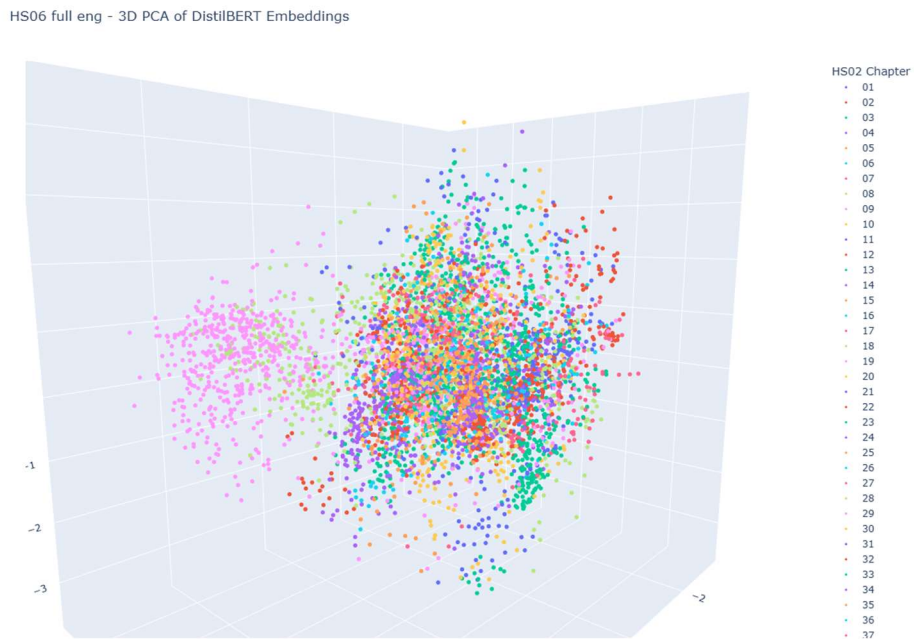
Los tokens generados se procesan y se obtienen embeddings de 768 dimensiones, uno para cada caso. A su vez, a modo de referencia, se procesan los textos de la nomenclatura de los códigos HS06.

Para la visualización de estos embeddings se utiliza PCA, en 2 y 3 dimensiones. A continuación, se observan los resultados, según HS06 descripciones de mercaderías muestra de un 5 % (13,389 casos):

Goods Description sampled - 3D PCA of DistilBERT Embeddings



En amarillo, se observa un marcado clúster que consiste en las descripciones del capítulo HS02 87 (vehículos y material automotor); y HS06 full eng (nomenclatura):



En este caso, en rosa y verde claro se observan un clúster que corresponde a la nomenclatura de los capítulos HS02 28 y 29 (químicos inorgánicos y orgánicos).

Medición de similitud del coseno

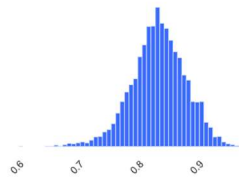
Como último ejercicio exploratorio, se procede a medir la similitud, para cada caso, entre GOODS_DESCRIPTION y full_eng (descripción de nomenclatura), para evaluar cuánto distan los embeddings de estos textos. En una agregación por HS06 se observan valores medios y medianos altos y coincidentes, del orden de 0.84:

[cosine_sim_gd_vs_hs_text_mean](#)

Real number (R)

High correlation Missing

Distinct	3960	Minimum	0.60158304
Distinct (%)	100.0%	Maximum	0.96980786
Missing	230	Zeros	0
Missing (%)	5.5%	Zeros (%)	0.0%
Infinite	0	Negative	0
Infinite (%)	0.0%	Negative (%)	0.0%
Mean	0.83501187	Memory size	65.5 KiB



Quantile statistics

Minimum	0.60158304
5-th percentile	0.75920722
Q1	0.80768058
median	0.83575511
Q3	0.86522526
95-th percentile	0.9077215
Maximum	0.96980786
Range	0.36822482
Interquartile range (IQR)	0.057544687

Descriptive statistics

Standard deviation	0.045425905
Coefficient of variation (CV)	0.054401508
Kurtosis	0.57477478
Mean	0.83501187
Median Absolute Deviation (MAD)	0.028919566
Skewness	-0.31314854
Sum	3306.647
Variance	0.0020635128
Monotonicity	Not monotonic

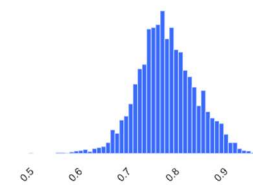
En términos de similitud mínima por código HS06, la similitud también es relativamente alta, con una media de 0.79.

[cosine_sim_gd_vs_hs_text_min](#)

Real number (R)

High correlation Missing

Distinct	3957	Minimum	0.50259566
Distinct (%)	99.9%	Maximum	0.96980786
Missing	230	Zeros	0
Missing (%)	5.5%	Zeros (%)	0.0%
Infinite	0	Negative	0
Infinite (%)	0.0%	Negative (%)	0.0%
Mean	0.78660514	Memory size	65.5 KiB



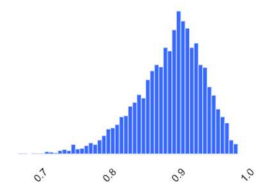
En términos de similitud máxima por código HS06, el valor medio ronda el 0.89, aunque se muestran valores muy próximos a la unidad.

[cosine_sim_gd_vs_hs_text_max](#)

Real number (R)

High correlation Missing

Distinct	3951	Minimum	0.66693276
Distinct (%)	99.8%	Maximum	0.98533964
Missing	230	Zeros	0
Missing (%)	5.5%	Zeros (%)	0.0%
Infinite	0	Negative	0
Infinite (%)	0.0%	Negative (%)	0.0%
Mean	0.88693692	Memory size	65.5 KiB



Lo antes descripto, motiva a observar los casos de similitud media, mínima y máxima.

Casos de similitud media o regular:

HS06	Descripción de la mercadería	Descripción completa del HS06	Similitud de coseno
853950	ASSY LED Base Strobe Upgrade	Electric filament or discharge lamps, including sealed beam lamp units and ultra-violet or infra-red lamps – Light-emitting diode (LED) lamps	0.827174
220870	VODKA FRAISE JELZIN	Undenatured ethyl alcohol of an alcoholic strength by volume of less than 80 % vol;	0.831804

	STRAWBERRY	spirits, liqueurs and other spirituous beverages – Liqueurs and cordials	
620590	SHORT SLEEVE REPAIR SHIRT	Men's or boys' shirts – Of other textile materials	0.830535
732620	HANGER ROD	Other articles of iron or steel – Articles of iron or steel wire	0.830890
843143	8-3/8SH Extension Overshot C-17208	Parts suitable for use solely or principally with the machinery of headings 84.25 to 84.30 – Parts for boring or sinking machinery	0.828724
870323	SUZUKI ESCUDO 2006	Motor cars and other motor vehicles principally designed for the transport of persons – Of a cylinder capacity exceeding 1 500 cm ³ but not exceeding 3 000 cm ³	0.817948
841899	Spare Parts for 10 TR Air Cooled Water Chiller	Refrigerators, freezers and other refrigerating or freezing equipment – Parts	0.822742
871120	USED CHANGZHOU KWANGYANG MOTORBYKE	Motorcycles (including mopeds) and cycles fitted with an auxiliary motor – With reciprocating internal combustion piston engine of a cylinder capacity exceeding 50 cm ³ but not exceeding 250 cm ³	0.826228
940540	SURFACE MOUNTED LIGHTS	Lamps and lighting fittings including searchlights and spotlights and parts thereof – Other electric lamps and lighting fittings	0.817389
842131	FILTER ASSY, AIR CLEANER	Centrifuges, including centrifugal dryers; filtering or purifying machinery and apparatus for liquids or gases – Intake air filters for internal combustion engines	0.831361

BOTTOM5 casos de similitud mínima:

HS06	Descripción de la mercadería	Descripción completa del HS06	Similitud de coseno
741820	SHOWER HEAD SQUARE BLACK 260X 190mm	Table, kitchen or other household articles and parts thereof, of copper; pot scourers and scouring or polishing pads, gloves and the like, of copper; sanitary ware and parts thereof – Sanitary ware and parts thereof	0.553591
741820	74182000000	Table, kitchen or other household articles and parts thereof, of copper; pot scourers and scouring or polishing	0.552399

		pads, gloves and the like, of copper; sanitary ware and parts thereof – Sanitary ware and parts thereof	
741820	FREESTANDING BATH TOWER	Table, kitchen or other household articles and parts thereof, of copper; pot scourers and scouring or polishing pads, gloves and the like, of copper; sanitary ware and parts thereof – Sanitary ware and parts thereof	0.552331
741820	TRAP ASSEMBLY	Table, kitchen or other household articles and parts thereof, of copper; pot scourers and scouring or polishing pads, gloves and the like, of copper; sanitary ware and parts thereof – Sanitary ware and parts thereof	0.548731
741820	HG BATH MIXER WALL MOUNTED MYSPOORT CHROME	Table, kitchen or other household articles and parts thereof, of copper; pot scourers and scouring or polishing pads, gloves and the like, of copper; sanitary ware and parts thereof – Sanitary ware and parts thereof	0.545765

TOP5 Casos de similitud máxima:

HS06	Descripción de la mercadería	Descripción completa del HS06	Similitud de coseno
640299	Other:Other footwear with outer soles and uppers of rubber or pla:Other footwear	Other footwear with outer soles and uppers of rubber or plastics. – Other footwear – Other	0.98534
520939	Other fabrics:Woven fabrics of cotton, containing 85 % or more by weight of:Dyed	Woven fabrics of cotton, containing 85 % or more by weight of cotton, weighing more than 200 g/m ² – Dyed – Other fabrics	0.984521
700729	Other:Safety glass, consisting of toughened (tempered) or:Laminated safety glass	Safety glass, consisting of toughened (tempered) or laminated glass – Laminated safety glass – Other	0.984296
848220	Tapered roller bearings, including cone and tapered roller assemblies	Ball or roller bearings – Tapered roller bearings, including cone and tapered roller assemblies	0.984269
740729	Other:Copper bars, rods and profiles.:Of copper alloys	Copper bars, rods and profiles – Of copper alloys – Other	0.984245

Los casos de similitud mínima muestran indicios de una mala calidad de la descripción, incluso con un caso donde la descripción es el mismo código. Por otro lado, los casos de

similitud máxima muestran que la descripción comercial, en realidad, consiste en una variante descripción de nomenclatura.

En los casos de similitud dentro del entorno de la media, las descripciones parecen lógicas, relacionadas con la naturaleza de la mercadería, pero un tanto escuetas.

Metodología

En este trabajo, el problema se abordó considerando diversas técnicas de Aprendizaje Automático y Procesamiento de Lenguaje Natural (ML & NLP, por sus siglas en inglés). Se consideró la aplicación de técnicas no-supervisadas, aunque en términos prácticos solo se avanzó con el uso de técnicas supervisadas.

Técnicas no-supervisadas

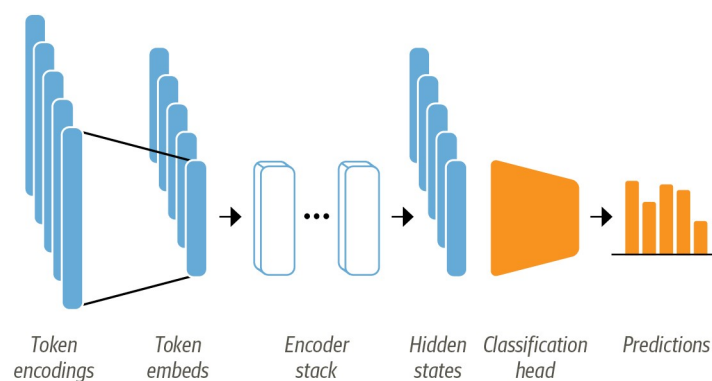
Las técnicas que no requieren el uso del resultado o variable objetivo pueden presentar una solución simplificada a problemáticas de clasificación. Sin embargo, en la literatura, los estudios donde estas técnicas son relativamente eficientes [20] se limitan a un número acotado de mercaderías (10 clases), mientras que en este trabajo se busca una solución holística de todo el universo posible.

Lo evaluado en el Análisis exploratorio motivó la desestimación de técnicas no supervisadas, previendo un desempeño pobre incluso para agrupar descripciones en códigos HS02 (97 clases). En un futuro trabajo, esto debe ser reevaluado para lograr mayor precisión respecto al desempeño alcanzable con estas técnicas.

Técnicas auto supervisadas

Incluso algunas de las técnicas consideradas hoy en día “tradicionales” para la explotación de texto (Doc2Vec o FastText [8] [9]) ya emplean redes neuronales que producen embeddings para luego clasificarlos usando medidas de similitud. Estos modelos deben entrenarse desde cero, con el corpus disponible para resolver la problemática.

Por otro lado, es posible partir de modelos con arquitectura Transformer (BERT, DistilBERT, entre otros [13] [10] [11]), preentrenados con corpus de texto extensos. La forma más sencilla de usar estos modelos es construir una arquitectura Encoder (cuerpo) + Classifier (cabeza), según se observa en la siguiente imagen [21]:



Esto es explotable mediante:

- Transfer learning (transferencia del aprendizaje), entrenando solo el classifier, mientras el encoder permanece fijo (FE – fixed encoder)
- Fine-tuning (ajuste fino) total (FFT – full finetuning) o parcial (PFT – partial finetuning), ajustando el conjunto cabeza y encoder (en todos sus capas -total- o

en algunas de sus capas de salidad -parcial-)

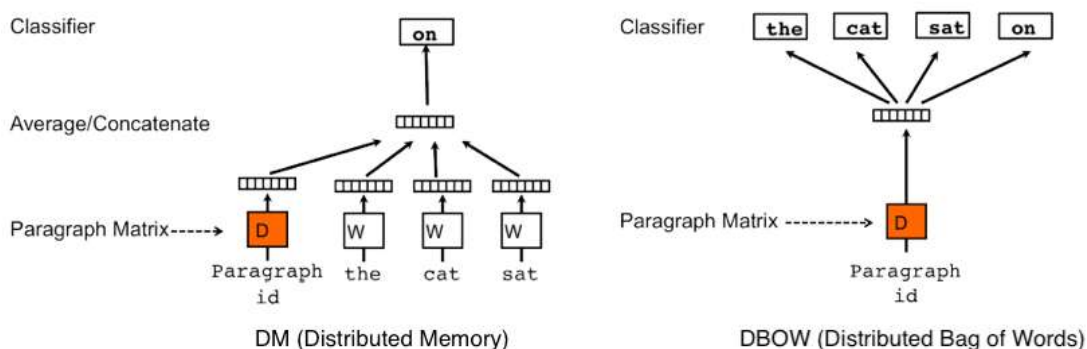
Este trabajo considera a Doc2Vec & FastText como modelos baselines a superar. Y luego, ponemos foco en modelos basados en arquitectura Transformer, particularmente en DistilBERT.

Doc2Vec

Originalmente presentado como Paragraph Vector por Le y Mikolov [8], es una técnica de aprendizaje auto-supervisada diseñada para obtener representaciones vectoriales densas de unidades de texto de longitud variable, tales como oraciones, párrafos o documentos completos. Su desarrollo responde a una limitación importante de los enfoques clásicos de representación textual, especialmente los modelos de **bolsa de palabras** (*bag of words*) y de **n-gramas**, que si bien resultan simples y eficaces en muchos problemas, tienden a perder tanto el orden de las palabras como buena parte de su contenido semántico. Esta limitación ya había sido señalada desde perspectivas distribucionales tempranas del lenguaje [22], y luego en el marco de los modelos distribuidos modernos del lenguaje [23] [24].

La idea central de Doc2Vec consiste en representar cada documento mediante un vector latente/embedding entrenable que contribuya a predecir palabras del propio texto. De esta manera, el modelo aprende una representación compacta y continua que intenta capturar regularidades semánticas y contextuales del documento completo [8]. Este planteo extiende la lógica introducida previamente por los modelos de vectores de palabras, en particular **Word2Vec** [7], donde las palabras se representan en un espacio vectorial continuo tal que palabras semánticamente cercanas tienden a ubicarse próximas entre sí.

En el trabajo original, Le y Mikolov proponen dos variantes principales del método. La primera es PV-DM (Distributed Memory), donde el vector del documento se combina con los vectores de palabras de una ventana de contexto para predecir la siguiente palabra. En este esquema, el vector del documento opera como una memoria global del tema o contenido general del texto. La segunda variante es PV-DBOW (Distributed Bag of Words), que prescinde de las palabras del contexto en la entrada y utiliza únicamente el vector del documento para predecir palabras muestreadas del mismo texto:



Según reportan los autores, PV-DM suele rendir mejor que PV-DBOW por sí solo, aunque la combinación de ambas variantes ofrece resultados más estables y consistentes en distintas tareas [8].

Uno de los aportes más relevantes de Doc2Vec es que permite aprender representaciones de textos **sin requerir etiquetas durante la etapa de aprendizaje de embeddings**. Esto constituye una ventaja importante cuando se dispone de grandes

volúmenes de texto libre pero de un conjunto limitado de ejemplos rotulados, situación frecuente en múltiples aplicaciones reales [8]. Una vez aprendidos los vectores, estos pueden utilizarse como variables de entrada para clasificadores supervisados convencionales, como regresión logística, máquinas de vectores soporte (SVM) o incluso algoritmos de agrupamiento como K-means [8]. En consecuencia, Doc2Vec puede entenderse no solo como un modelo en sí mismo, sino también como una técnica de **representación de características** para transformar texto libre en insumos numéricos aptos para tareas posteriores de clasificación o recuperación de información.

Desde una perspectiva comparativa, la técnica intenta ubicarse entre dos familias metodológicas. Por un lado, supera varias limitaciones de los enfoques puramente dispersos como *bag of words* y *bag of n-grams*, en tanto incorpora información semántica y cierta sensibilidad al orden local de las palabras [8]. Por otro lado, mantiene una estructura bastante más simple que arquitecturas más recientes basadas en parsing o atención profunda.

Para este trabajo, Doc2Vec resulta especialmente pertinente para el problema abordado, representando una alternativa más semántica que los enfoques clásicos de conteo, pero aún más liviana y menos costosa que los modelos Transformer preentrenados. Su utilidad en clasificación arancelaria puede justificarse por varias razones:

Ventajas
Las descripciones comerciales suelen ser breves, heterogéneas, ruidosas y ambiguas, lo cual dificulta el uso de reglas estrictas o coincidencias léxicas exactas. En este tipo de contexto, una representación distribuida de documentos puede ayudar a aproximar descripciones cercanas en significado, aunque no compartan exactamente las mismas palabras, algo central en problemas donde abundan abreviaturas, variantes expresivas o pequeños errores de escritura.
Doc2Vec permite construir una representación densa del texto completo, lo cual puede ser útil cuando la descripción es demasiado escueta como para que un simple conteo de términos resulte suficiente.
Los vectores aprendidos pueden ser reutilizados no solo para clasificación, sino también para análisis de similitud entre productos, búsqueda de vecinos cercanos y exploración de errores.
Doc2Vec ofrece una base comparativa sólida frente a modelos más modernos, al representar un baseline con fundamento teórico y evidencia empírica previa [8]. Su entrenamiento desde cero sobre el corpus disponible lo vuelve adecuado para evaluar cuánto puede aprenderse exclusivamente a partir de las descripciones comerciales del dominio, sin apoyarse en conocimiento lingüístico general previamente incorporado por modelos externos.
Su menor complejidad relativa permite iterar con más rapidez y menor costo computacional que en modelos basados en Transformers, lo que facilita la experimentación inicial.

No obstante, también es importante señalar sus limitaciones:

Limitaciones

Doc2Vec no aprende representaciones profundamente dependientes del contexto completo de cada token, sino una representación fija del documento dentro de un esquema predictivo más simple. Por ello, puede quedar por detrás cuando se requiere captar matices muy finos del lenguaje, relaciones complejas entre términos o patrones de desambiguación más sofisticados.

El modelo construye un vocabulario en base al corpus de entrenamiento, usando cada palabra como tokens completos, lo que lo hace susceptible a errores de tipes, abreviaturas y diferentes formas de escribir una misma palabra.

A diferencia de modelos contextualizados como BERT o DistilBERT, Doc2Vec no modela de forma explícita el contexto completo de cada token. Por ello, en problemas altamente especializados y multiclase, como la recomendación de códigos HS04, es razonable esperar que funcione como una base útil, aunque probablemente inferior a modelos Transformer bien ajustados, especialmente cuando existe suficiente volumen de datos y una estrategia de entrenamiento adecuada. Justamente por esto, su inclusión como baseline tiene sentido práctico y valor explicativo para mostrar el salto metodológico entre embeddings distribuidos “tradicionales” y modelos preentrenados de lenguaje.

FastText

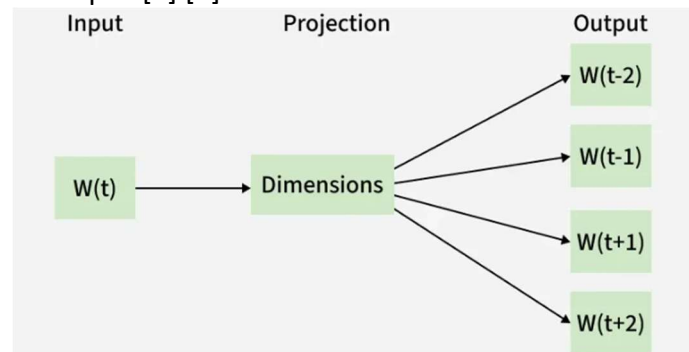
FastText puede entenderse como una evolución directa de Word2Vec orientada a superar una de sus limitaciones más importantes: tratar cada palabra como una unidad atómica e independiente, sin considerar su estructura interna [7] [9]. En los modelos distribucionales clásicos, cada término del vocabulario recibe un vector propio, lo cual funciona razonablemente bien para palabras frecuentes, pero resulta problemático cuando se trabaja con vocabularios grandes, términos raros, formas flexionadas, compuestos, abreviaturas o errores de escritura. Esta limitación fue especialmente señalada en el trabajo de Bojanowski, Grave, Joulin y Mikolov, quienes proponen enriquecer la representación de palabras incorporando información de subpalabras o fragmentos de caracteres, bajo la hipótesis de que buena parte del contenido morfológico y ortográfico de una palabra puede aprovecharse para construir embeddings más robustos [9].

La idea central de FastText consiste en representar cada palabra no solo mediante un identificador único, sino como una bolsa de **n-gramas de caracteres**. En lugar de aprender un único vector por palabra, el modelo aprende vectores para secuencias cortas de caracteres, y la representación final de una palabra se obtiene como la suma de los vectores de esos fragmentos, junto con el vector asociado a la palabra completa [9]. Por ejemplo, una palabra como *where* puede descomponerse en n-gramas tales como <wh, whe, her, ere, re> y <where>, donde los símbolos de borde permiten distinguir prefijos y sufijos de otras secuencias internas [9]. En la formulación presentada por los autores, se emplean usualmente n-gramas de longitud 3 a 6, lo que permite cubrir desde terminaciones cortas hasta raíces o segmentos más largos [9].

Este diseño es conceptualmente muy relevante porque introduce compartición de parámetros entre palabras relacionadas. Dos palabras distintas pueden compartir varios n-gramas, y por lo tanto compartir parcialmente su representación. En consecuencia, FastText logra modelar mejor palabras raras o incluso palabras no vistas durante el entrenamiento, construyendo sus vectores a partir de subcomponentes conocidos [9].

Esta propiedad constituye una ventaja clara frente a Word2Vec tradicional, donde una palabra fuera del vocabulario simplemente no posee embedding útil. En el paper original, los autores muestran precisamente que esta capacidad mejora de forma consistente el tratamiento de palabras infrecuentes y de idiomas con morfología rica, así como el desempeño general en tareas de similitud léxica [9].

Desde el punto de vista metodológico, FastText mantiene la lógica general del modelo skip-gram con negative sampling introducido en Word2Vec [7]. Es decir, el entrenamiento sigue basándose en predecir palabras del contexto a partir de una palabra objetivo, maximizando la capacidad del embedding para capturar regularidades distribucionales del corpus [7] [9].



La diferencia es que, en vez de usar solamente el vector de la palabra objetivo, FastText calcula un score combinando los vectores de todos sus n-gramas de caracteres [9]. Esto permite conservar la eficiencia de entrenamiento de la familia Word2Vec, pero enriqueciendo la representación con una capa de información subléxica. Para controlar el costo en memoria, el modelo utiliza además una función de hashing sobre los n-gramas, específicamente FNV-1a, lo que evita mantener un diccionario explícito gigantesco de fragmentos posibles [9].

Uno de los aspectos más interesantes del trabajo original es que esta modificación, pese a su simplicidad, produce mejoras relevantes en evaluación empírica, los autores reportan resultados sobre nueve idiomas y muestran que el uso de subword information supera a los baselines skip-gram y CBOW en la mayoría de los conjuntos de similitud léxica, especialmente cuando intervienen palabras raras u observaciones fuera de vocabulario [9]. También observan mejoras claras en analogías sintácticas, donde la morfología cumple un papel importante, aunque no necesariamente en todas las analogías semánticas [9]. En otras palabras, FastText parece capturar especialmente bien regularidades relacionadas con formación de palabras, sufijos, prefijos, flexión y composición, más que relaciones semánticas profundas no asociadas a la forma superficial. Esto es coherente con la propia estructura del modelo: al apoyarse en caracteres, su fortaleza principal está en explotar patrones ortográficos y morfológicos. En términos comparativos, FastText ocupa un lugar intermedio muy interesante entre enfoques puramente léxicos y modelos profundamente contextualizados. Por un lado, supera varias limitaciones de Word2Vec y de otros métodos que solo consideran tokens completos, al incorporar una noción explícita de estructura interna de palabra [7] [9]. Por otro lado, sigue siendo mucho más liviano, interpretable y barato de entrenar que modelos basados en Transformers, como BERT o DistilBERT [10] [11]. Esta posición intermedia lo vuelve especialmente útil como baseline fuerte en tareas de clasificación de texto, recuperación semántica o matching aproximado de descripciones.

Para el problema abordado en este trabajo, FastText resulta particularmente pertinente

por las siguientes razones:

Ventajas
El modelo puede beneficiarse de similitudes parciales entre descripciones que no coinciden palabra por palabra. En comercio internacional es frecuente encontrar variantes como abreviaturas, marcas de formato, errores de tipeo, singular/plural, diferencias ortográficas menores o composiciones no estandarizadas. Una descripción como <i>motorbike</i> , <i>motor bike</i> , <i>motor-cycle</i> o incluso una forma mal escrita puede seguir compartiendo subestructuras relevantes. FastText puede explotar esas coincidencias parciales mejor que un enfoque basado únicamente en tokens enteros [9].
FastText puede ofrecer una ventaja operacional en las clases poco frecuentes. Frente al problema planteado y explorado, con alto desbalance, con códigos muy representados y otros con escasas observaciones, incluso casos donde ciertos códigos aparecen una sola vez en niveles más desagregados. En este contexto, la posibilidad de compartir información entre palabras mediante n-gramas puede ayudar a estabilizar la representación de términos infrecuentes y reducir parcialmente el impacto de la escasez de ejemplos. El propio paper muestra que el modelo es robusto en situaciones de menor volumen de entrenamiento y en palabras raras, justamente porque no depende exclusivamente de la frecuencia observada de cada token completo [9].
Es metodológicamente atractivo como baseline porque combina bajo costo computacional con una representación semántico-morfológica más rica que las técnicas clásicas de conteo. Esto permite iterar rápido, probar variantes de preprocesamiento y evaluar el efecto de decisiones como normalización, uso de n-gramas o limpieza de texto. En otras palabras, FastText no solo puede funcionar como clasificador o generador de embeddings, sino también como instrumento experimental para medir cuánto valor agregan las decisiones de preparación textual antes de pasar a modelos más costosos.
Puede reutilizarse más allá de la clasificación directa. Los embeddings resultantes pueden servir para análisis de similitud entre productos, búsqueda de vecinos cercanos, exploración de errores, clustering preliminar o recuperación de descripciones semejantes, usos todos muy pertinentes en un dominio donde interesa no solo predecir una clase, sino también justificar sugerencias y examinar confusiones entre partidas afines.

Sin embargo, también es importante marcar sus limitaciones:

Limitaciones
Aun cuando FastText incorpora información subléxica valiosa, sigue produciendo representaciones esencialmente estáticas. Es decir, una palabra tendrá la misma base representacional independientemente del contexto específico en que aparezca, salvo por el efecto indirecto del entrenamiento sobre corpus [9]. En consecuencia, el modelo puede quedar corto frente a problemas donde la desambiguación depende fuertemente del contexto completo de la frase o de relaciones semánticas complejas entre términos, algo que sí modelan mejor BERT y DistilBERT mediante representaciones contextualizadas [10] [11]. En clasificación arancelaria, esto puede

ser relevante cuando una misma palabra aparece en dominios distintos, o cuando la distinción entre partidas requiere captar relaciones técnicas finas que no se desprenden de la mera forma de los términos.

El sesgo de FastText hacia la forma superficial puede ser una fortaleza y a la vez una debilidad. Ayuda frente a errores ortográficos, flexiones o compuestos, pero también puede inducir similitudes espurias entre palabras formalmente parecidas y semánticamente diferentes. En nomenclaturas y descripciones de productos, donde abundan códigos, siglas, medidas y nombres técnicos, esto exige complementar el modelo con una buena estrategia de limpieza, evaluación y análisis de errores. Del mismo modo, al tratarse de un problema multiclase con miles de etiquetas, es razonable esperar que FastText constituya un baseline competitivo, pero probablemente insuficiente para capturar todo el nivel de especialización semántica requerido por la tarea, especialmente si se busca precisión alta en recomendaciones top-k o robustez frente a clases muy cercanas entre sí.

En síntesis, FastText representa una técnica especialmente valiosa para este trabajo por cuatro motivos principales: extiende de manera elegante la lógica de Word2Vec/Doc2Vec incorporando subpalabras; ofrece mayor robustez frente a ruido, rareza léxica y vocabulario abierto; mantiene un costo computacional moderado que favorece la experimentación; y proporciona un baseline sólido para comparar contra modelos Transformer más sofisticados [7] [9].

En el contexto de clasificación arancelaria HS04 a partir de descripciones comerciales cortas y heterogéneas, su uso está bien justificado tanto por razones teóricas como prácticas. No obstante, también cabe esperar que su desempeño quede por debajo del de modelos contextualizados cuando la tarea demande desambiguación fina, comprensión semántica más profunda o aprovechamiento intensivo del contexto global [10] [11]. Precisamente por eso, su inclusión en la tesis no solo tiene valor predictivo, sino también valor explicativo: permite mostrar cuánto puede lograrse con embeddings distribuidos “tradicionales” enriquecidos con subpalabras, y cuánto adicional aportan luego las arquitecturas basadas en Transformers.

Transformers

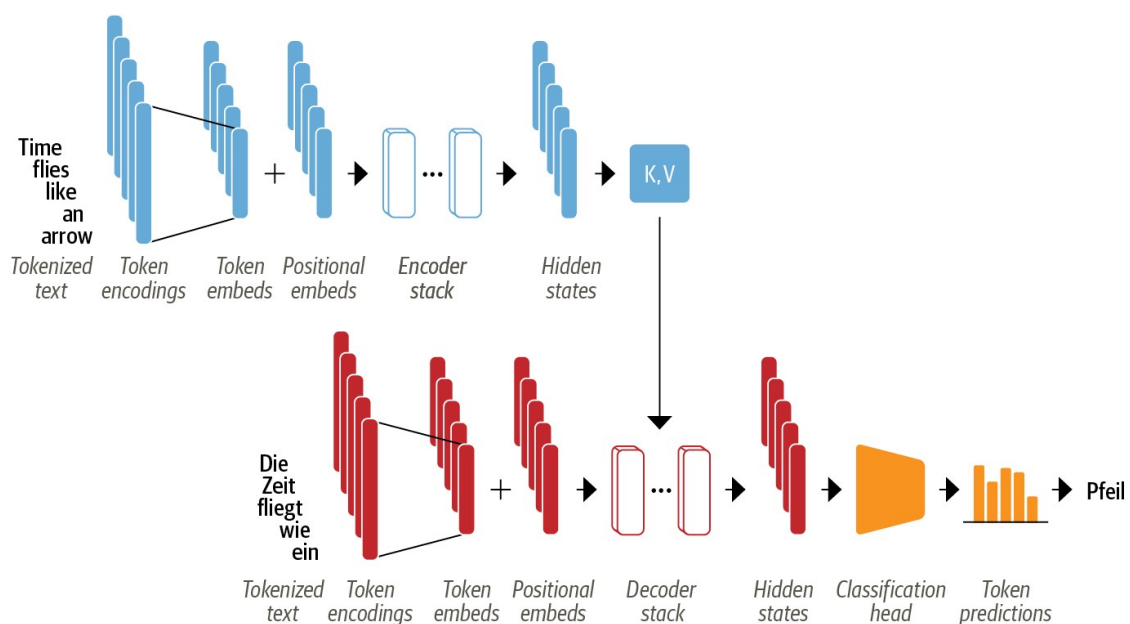
La arquitectura Transformer, introducida por Vaswani et al. en 2017, representa un cambio de paradigma en el procesamiento de secuencias, al proponer un modelo basado exclusivamente en mecanismos de atención, prescindiendo por completo de recurrencias y convoluciones [13]. Frente a las arquitecturas dominantes hasta ese momento —principalmente redes recurrentes como LSTM y GRU, y en menor medida modelos convolucionales para secuencias—, el Transformer plantea que la dependencia entre los elementos de una secuencia puede modelarse de manera más eficiente si cada token “atiende” directamente a los demás mediante operaciones matriciales paralelizables.

La motivación central del trabajo surge de una limitación importante de los modelos recurrentes: su naturaleza secuencial. En una RNN, cada estado oculto depende del estado anterior, lo que dificulta la paralelización del entrenamiento y vuelve más costoso aprender dependencias de largo alcance. Vaswani et al. observan que, aun cuando los mecanismos de atención ya se utilizaban con éxito en traducción automática y otras

tareas de secuencia, todavía se apoyaban sobre una base recurrente [13]. Su propuesta consiste en reemplazar esa base por una arquitectura donde la atención sea el mecanismo principal de cómputo, permitiendo capturar relaciones globales entre posiciones de la secuencia con menos pasos secuenciales y mejor escalabilidad.

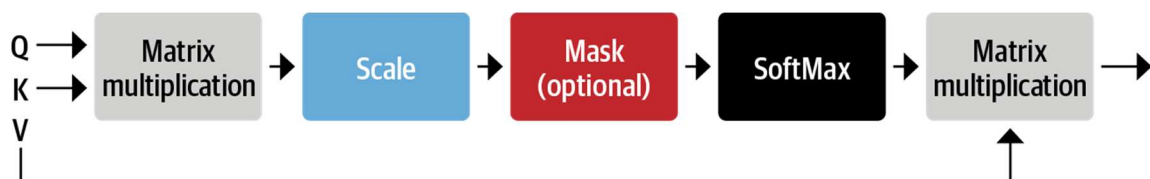
En términos generales, el Transformer mantiene el esquema **encoder-decoder** clásico de los modelos de secuencia a secuencia. El encoder recibe una secuencia de entrada y la transforma en una representación continua contextualizada; luego, el decoder genera la secuencia de salida consumiendo esa representación y los tokens previos generados [13].

Sin embargo, a diferencia de los enfoques anteriores, tanto el encoder como el decoder están contruidos mediante el apilamiento de bloques idénticos compuestos por **self-attention** y redes feed-forward posición a posición, acompañados por conexiones residuales y normalización de capa.

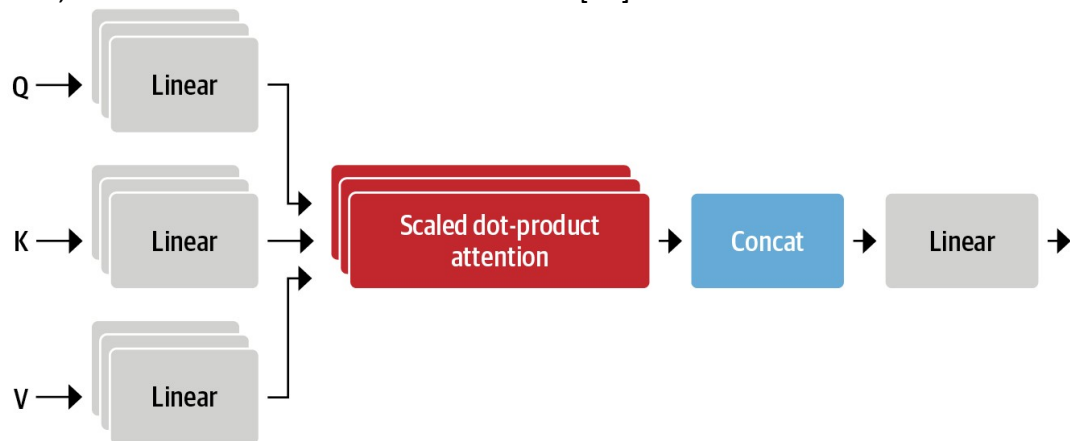


El componente central de la arquitectura es la self-attention. En este mecanismo, cada token de la secuencia se proyecta a tres espacios distintos: query (Q), key (K) y value (V). La atención se calcula midiendo la compatibilidad entre una query y todas las keys, normalizando esos puntajes con softmax, y usando el resultado para combinar los vectores value [13]. En la formulación propuesta por el paper, esta operación se expresa como:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



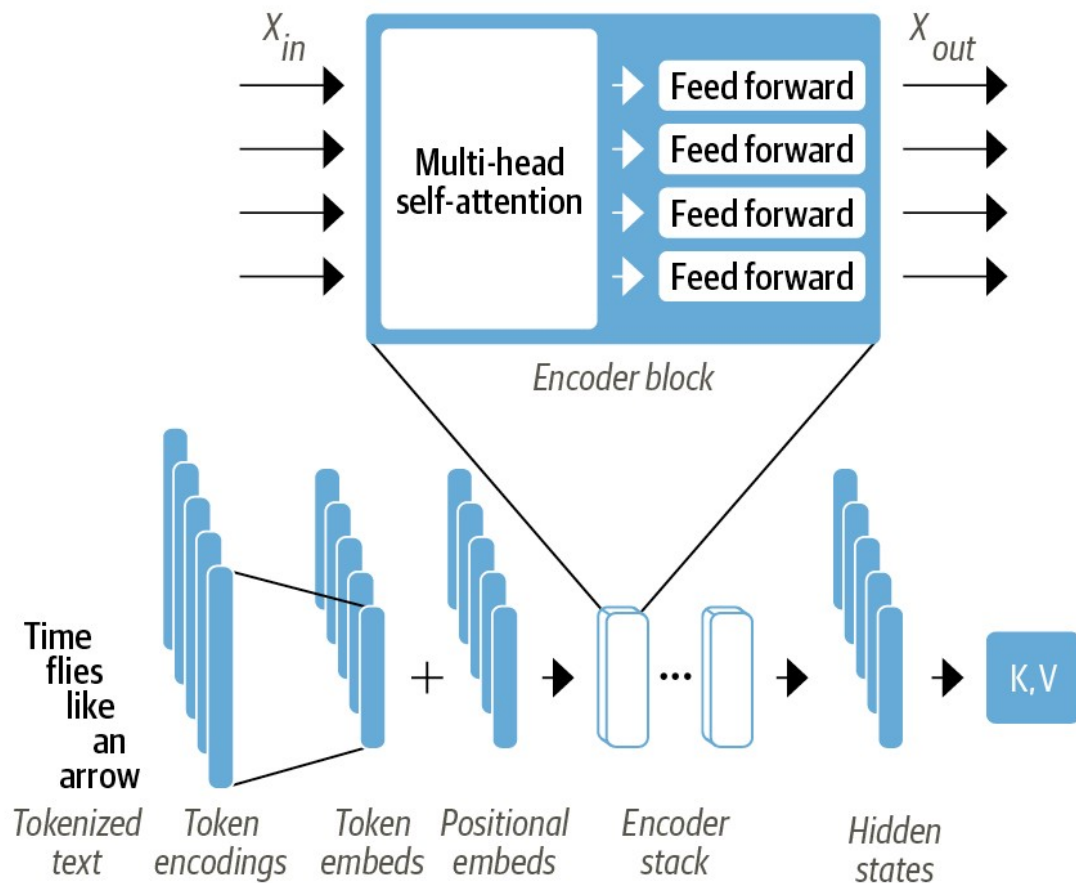
La división por $\sqrt{d_k}$ da lugar a la llamada scaled dot-product attention, que ayuda a estabilizar el entrenamiento cuando la dimensionalidad de las keys es alta [13]. Conceptualmente, este mecanismo permite que cada palabra ajuste dinámicamente cuánto “peso” asigna a otras palabras de la misma secuencia. Esto hace posible capturar dependencias léxicas, sintácticas y semánticas incluso cuando los términos relevantes están muy alejados en el texto, algo especialmente costoso para modelos recurrentes. A su vez, el Transformer no utiliza una sola atención, sino multi-head attention. En lugar de calcular una única matriz de atención, el modelo proyecta Q, K y V múltiples veces y ejecuta varias atenciones en paralelo. Cada “cabeza” puede especializarse en distintos tipos de relación: concordancia gramatical, dependencias a largo plazo, referencia entre entidades, o asociaciones semánticas más locales [13].



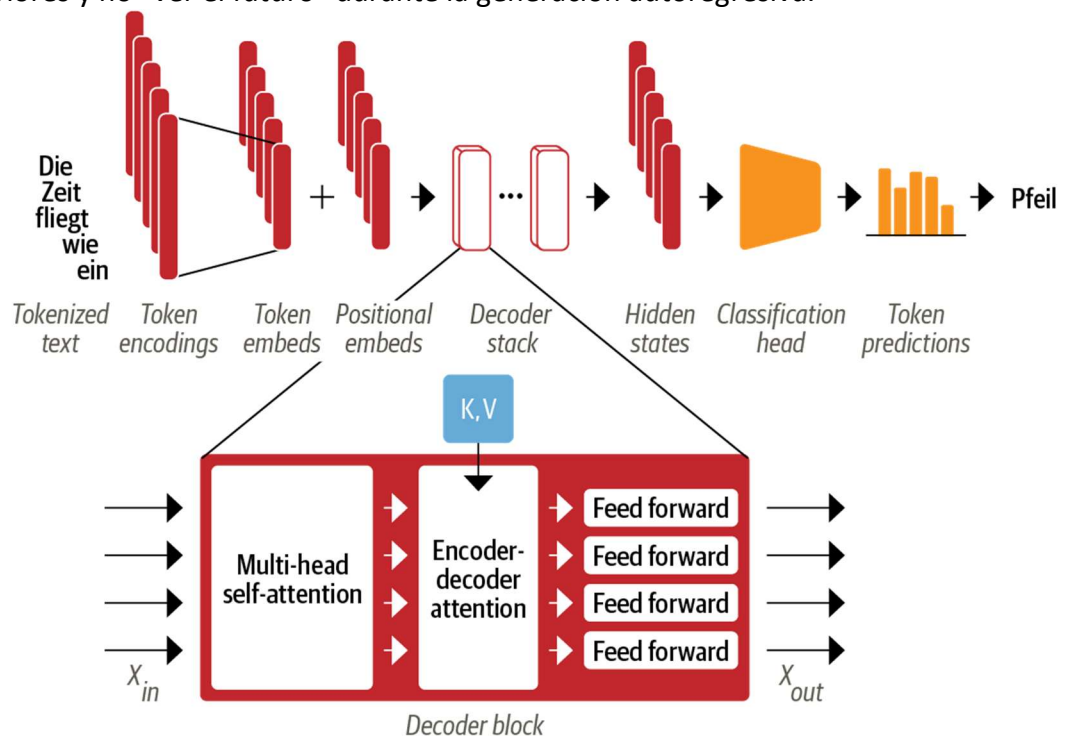
El paper destaca que esta estrategia mejora la capacidad del modelo para atender simultáneamente a información proveniente de distintos subespacios de representación, superando la limitación de una atención única que tendería a promediar demasiadas relaciones.

Otro elemento clave del Transformer es que, al no usar ni recurrencia ni convolución, necesita incorporar explícitamente la noción de orden. Para eso, Vaswani et al. introducen las codificaciones posicionales (*positional encodings*), que se suman a los embeddings de entrada [13]. En el trabajo original se emplean funciones sinusoidales de distintas frecuencias, de modo que cada posición en la secuencia reciba una señal distintiva. Esto permite al modelo inferir tanto posiciones absolutas como relaciones relativas entre tokens, sin depender de una lectura secuencial del texto.

Desde el punto de vista arquitectónico, el encoder del Transformer original está compuesto por una pila de seis capas idénticas. Cada una incluye: primero, una subcapa de multi-head self-attention; segundo, una red feed-forward aplicada de manera independiente a cada posición; y alrededor de ambas, conexiones residuales y layer normalization [13].



El **decoder** también se organiza en seis capas, pero añade una tercera subcapa de atención sobre la salida del encoder. Además, la self-attention del decoder está **enmascarada**, de modo que cada posición solo pueda usar información de tokens anteriores y no “ver el futuro” durante la generación autoregresiva.



Uno de los principales aportes del trabajo original es mostrar que esta arquitectura logra combinar **calidad predictiva, eficiencia computacional y paralelización**. En las tareas de traducción evaluadas por Vaswani et al., el Transformer supera a modelos recurrentes y convolucionales del estado del arte de ese momento, logrando mejores puntajes BLEU con un costo de entrenamiento menor [13]. El paper reporta, por ejemplo, un BLEU de 28,4 en WMT 2014 English-to-German y de 41,8 en English-to-French con el modelo grande, destacando además que pudo entrenarse en tiempos sensiblemente menores gracias a su estructura altamente paralelizable.

Metodológicamente, esto tiene consecuencias muy profundas. Mientras que Word2Vec, Doc2Vec y FastText producen representaciones distribuidas útiles pero relativamente “estáticas”, el Transformer genera **representaciones contextuales dinámicas**: el embedding final de una palabra depende del contexto en que aparece. En otras palabras, el modelo no asigna a cada token una única representación fija, sino una representación dependiente de los demás elementos de la secuencia. Esta propiedad es decisiva en tareas donde la desambiguación depende del contexto y constituye la base conceptual de modelos posteriores como **BERT** [10] y **DistilBERT** [11].

Para el problema abordado en este trabajo, centrado en la clasificación arancelaria HS04 a partir de descripciones comerciales, la arquitectura Transformer resulta especialmente pertinente por varias razones:

Ventajas
Las descripciones de mercaderías suelen ser breves, heterogéneas, ruidosas y muchas veces ambiguas, tal como se describe en el planteo del problema y en el análisis exploratorio del conjunto de datos BACUDA. En ese contexto, la capacidad de self-attention para ponderar dinámicamente qué términos de la descripción son más relevantes puede ayudar a distinguir entre productos cercanos pero no equivalentes, algo crítico cuando existen miles de clases y alto desbalance.
El Transformer ofrece una mejor base para modelar relaciones semánticas finas entre términos técnicos, materiales, usos, partes y atributos funcionales de una mercadería. En clasificación arancelaria, muchas decisiones no dependen solo de palabras aisladas, sino de combinaciones como el tipo de producto, su material, su función principal o el contexto técnico en el que se menciona. Un modelo basado en atención contextualizada puede capturar mejor estas interacciones que un esquema de embeddings estáticos o de coincidencias parciales por subpalabras.
La arquitectura es altamente compatible con estrategias modernas de transfer learning, una de las formas más útiles de explotar Transformers, que consiste en partir de modelos preentrenados sobre grandes corpus y luego adaptarlos a la tarea específica mediante entrenamiento del clasificador final o fine-tuning parcial o total. Esto es particularmente valioso en un dominio como el comercio internacional, donde se dispone de una gran cantidad de descripciones rotuladas, pero no necesariamente de suficiente diversidad lingüística como para entrenar desde cero un modelo profundo de gran escala.
El Transformer no solo sirve para clasificar, sino también para tareas complementarias relevantes para la práctica aduanera: recuperación semántica de casos similares, re-ranking de candidatos, análisis de atención para interpretación parcial del comportamiento del modelo, y construcción de sistemas híbridos con recuperación

documental o RAG. En ese sentido, su utilidad excede la simple predicción de una etiqueta y abre la posibilidad de diseñar asistentes inteligentes que no solo recomienden un código, sino que además recuperen evidencia textual o casos comparables que ayuden al clasificador humano.

De todos modos, también es importante señalar algunas limitaciones:

Limitaciones

La self-attention estándar tiene costo cuadrático en la longitud de la secuencia, lo que puede volverse problemático con textos muy largos [13]. Si bien esto no constituye una gran restricción para descripciones comerciales cortas o medianas, sí marca un límite operativo para otros escenarios documentales más extensos.
--

Los modelos Transformer suelen requerir más recursos de hardware y una estrategia de entrenamiento más cuidadosa que baselines como Doc2Vec o FastText, especialmente cuando se realiza fine-tuning completo.

En perspectiva, la relevancia del Transformer para este trabajo es doble. Por un lado, constituye la base teórica y arquitectónica de los modelos que hoy dominan la clasificación de texto en NLP. Por otro, aporta una mejora conceptual importante respecto de los embeddings distribuidos tradicionales y estáticos: reemplaza la representación fija del texto por una representación contextual y relacional, donde cada token se interpreta en función del resto. Para una tarea tan exigente como la clasificación arancelaria sobre un universo amplio de partidas HS04, esto permite esperar una mejor capacidad de desambiguación, mayor robustez frente a redacciones heterogéneas y una base más sólida para sistemas de asistencia inteligentes aplicados al comercio internacional.

En síntesis, la arquitectura Transformer puede entenderse como el punto de inflexión que reorganiza el campo del NLP moderno. Su propuesta de modelar secuencias únicamente con atención, combinando paralelización, capacidad para capturar dependencias de largo alcance y representaciones contextuales, no solo permitió mejoras sustantivas en traducción automática, sino que sentó las bases de los modelos preentrenados que luego dominarían tareas de clasificación, recuperación y comprensión de texto [13]. En el marco de este trabajo, esto justifica plenamente su inclusión como pieza central del salto metodológico desde enfoques basados en embeddings clásicos hacia modelos de lenguaje profundos adaptados a la clasificación arancelaria.

BERT

BERT (Bidirectional Encoder Representations from Transformers), propuesto por Devlin, Chang, Lee y Toutanova, puede entenderse como uno de los desarrollos más influyentes dentro del NLP moderno, al llevar la arquitectura Transformer al terreno del preentrenamiento general de representaciones lingüísticas profundas y bidireccionales [10] [13]. Mientras que la arquitectura Transformer introducida por Vaswani et al. había mostrado que la atención era suficiente para modelar secuencias con gran capacidad de paralelización y contextualización, BERT dio un paso adicional: demostrar que un modelo preentrenado sobre grandes corpus textuales podía luego adaptarse con pocos cambios a una gran variedad de tareas, logrando mejoras sustantivas en comprensión y

clasificación de texto [10] [13]. Esta transición es especialmente relevante para este trabajo, donde se busca clasificar descripciones comerciales breves y heterogéneas en miles de clases HS04, un escenario donde la sensibilidad al contexto puede marcar una diferencia decisiva.

La innovación central de BERT consiste en aprender representaciones profundamente contextuales y bidireccionales. A diferencia de modelos previos basados en Word2Vec, Doc2Vec o FastText, donde la representación de una palabra o documento es relativamente estática, en BERT el vector asociado a cada token depende de los demás tokens de la secuencia. Esto significa que una palabra no se interpreta de manera aislada, sino en función del contexto completo en que aparece [10] [13]. Desde el punto de vista conceptual, esto constituye una ventaja importante para tareas donde la ambigüedad léxica es alta o donde el significado práctico surge de la interacción entre varios términos, como ocurre frecuentemente en descripciones de mercaderías, donde importan simultáneamente el tipo de bien, el material, la función, la parte o accesorio involucrado, e incluso ciertas especificaciones técnicas.

En términos arquitectónicos, BERT reutiliza el bloque encoder del Transformer, prescindiendo del decoder, ya que su objetivo no es generar secuencias autoregresivas sino codificar texto de entrada con un alto nivel de contextualización [10] [13]. Cada capa del encoder combina mecanismos de multi-head self-attention con redes feed-forward, conexiones residuales y normalización de capa. Gracias a la self-attention, cada token puede ponderar directamente a todos los demás tokens del texto, capturando dependencias semánticas y sintácticas tanto cercanas como lejanas [13]. Para la clasificación de descripciones comerciales, esto resulta especialmente valioso, porque permite que el modelo asigne peso a términos clave aunque estén separados en la frase o aparezcan en combinaciones poco frecuentes. Una descripción como *air cleaner filter assembly for diesel engine* no solo contiene palabras relevantes por separado, sino relaciones funcionales entre ellas que un modelo contextualizado puede explotar mejor que un embedding estático.

El preentrenamiento de BERT se apoya en dos tareas principales:

- La primera es Masked Language Modeling (MLM), donde ciertos tokens de la secuencia se ocultan y el modelo debe predecirlos a partir del contexto restante [10]. Esta estrategia obliga al modelo a desarrollar una comprensión bidireccional del texto, ya que para recuperar una palabra faltante necesita usar tanto el contexto izquierdo como el derecho.
- La segunda tarea es Next Sentence Prediction (NSP), que busca modelar relaciones entre pares de oraciones, entrenando al modelo para decidir si una oración sigue a otra en el corpus [10].

Si bien trabajos posteriores discutieron la importancia relativa de NSP y propusieron variantes más robustas del preentrenamiento, en el marco histórico del paper original estas dos tareas definieron el núcleo del aprendizaje de BERT y explican buena parte de su éxito temprano en benchmarks generales de NLP. El paper de DistilBERT, de hecho, toma a BERT como teacher precisamente porque su esquema de preentrenamiento había demostrado una gran capacidad de generalización en tareas posteriores.

Otro de los aportes decisivos de BERT es metodológico. El modelo consolidó el paradigma de preentrenar y luego adaptar. En lugar de entrenar desde cero una red específica para cada problema, BERT permite partir de un encoder ya entrenado sobre corpus masivos y luego realizar *fine-tuning* para una tarea concreta, añadiendo una

cabeza de clasificación relativamente simple [10] [14]. Esta lógica encaja de forma directa con los objetivos del presente trabajo, donde se propone comparar modelos entrenados desde cero, como Doc2Vec y FastText, frente a modelos preentrenados basados en Transformers, para cuantificar el valor incremental que aporta el conocimiento lingüístico general ya incorporado en estos últimos [12] [14]. En un escenario como BACUDA, con alrededor de 500 mil pares (*descripción, código*), miles de clases y un fuerte desbalance, esta estrategia resulta especialmente atractiva porque permite aprovechar señales lingüísticas generales del inglés aun cuando algunas clases aduaneras tengan relativamente pocos ejemplos.

Para el problema de clasificación arancelaria HS04, BERT presenta varias ventajas teóricas y prácticas, aunque también limitaciones:

Ventajas
Puede capturar mejor la polisemia y la desambiguación contextual. Términos como <i>parts, set, kit, assembly, motor, pump</i> o <i>filter</i> pueden aparecer en múltiples dominios, pero su sentido práctico cambia según las palabras vecinas.
Favorece la representación de relaciones técnicas finas entre atributos del producto, algo crucial cuando la clasificación depende de distinguir, por ejemplo, si un bien corresponde al producto final, a una parte, a un accesorio o a un componente especializado.
BERT ofrece una base muy sólida para tareas complementarias además de la clasificación, como recuperación semántica, re-ranking de candidatos y análisis de confusiones entre partidas cercanas, todos aspectos compatibles con la idea de construir sistemas de asistencia al clasificador humano y no meros clasificadores automáticos cerrados [5] [12].
Limitaciones
Su mayor costo computacional frente a baselines como Doc2Vec o FastText. Los modelos BERT requieren más memoria, más tiempo de entrenamiento y mayor cuidado al ajustar hiperparámetros, especialmente en <i>fine-tuning</i> completo.
Su potencia puede venir acompañada de cierta fragilidad práctica: pequeñas decisiones de truncado, longitud máxima, tasa de aprendizaje o estrategia de balanceo pueden afectar el rendimiento final [14].
Aunque BERT mejora la representación lingüística, no resuelve por sí mismo todos los problemas del dominio aduanero: sigue existiendo desbalance extremo, clases muy cercanas entre sí y necesidad de validación con métricas top-k y análisis de error.

En otras palabras, BERT constituye una base metodológica más fuerte que los embeddings distribuidos tradicionales, pero no reemplaza el trabajo de diseño experimental y evaluación rigurosa que exige un problema como la recomendación de códigos HS04.

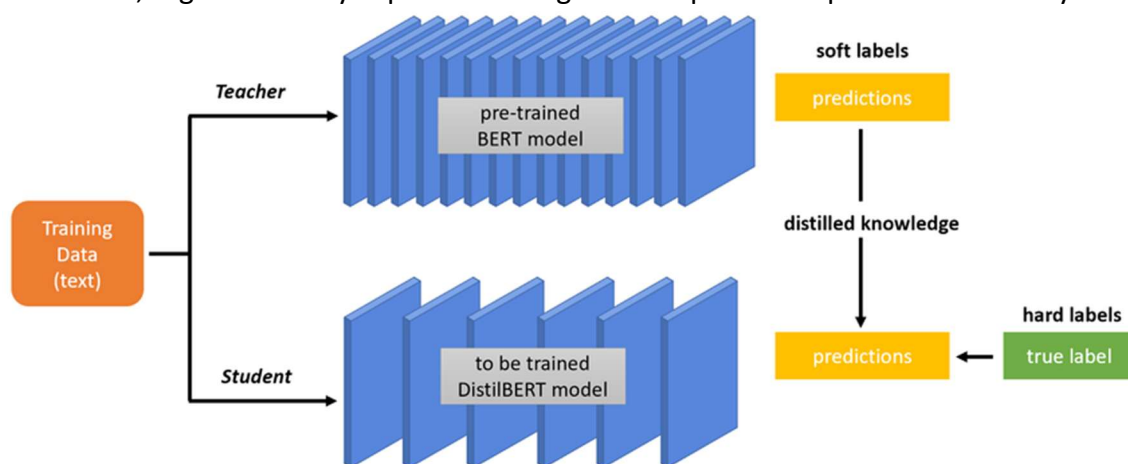
En síntesis, BERT puede entenderse como el modelo que consolidó la utilidad práctica de los Transformers preentrenados para clasificación de texto [10] [13]. Su principal aporte frente a técnicas previas es reemplazar representaciones estáticas por representaciones contextuales bidireccionales, altamente adaptables mediante *transfer learning* [10] [14]. Para esta tesis, esto lo vuelve una referencia ineludible: no solo porque fundamenta el salto metodológico desde Doc2Vec y FastText hacia modelos más

sofisticados, sino también porque ofrece una arquitectura especialmente adecuada para el tipo de descripciones comerciales breves, ambiguas y técnicamente densas que caracterizan la clasificación arancelaria en comercio internacional.

DistilBERT

DistilBERT, propuesto por Sanh, Debut, Chaumond y Wolf, puede interpretarse como una versión comprimida de BERT diseñada para conservar gran parte de su capacidad lingüística, pero con menor costo computacional y mayor velocidad de inferencia [11]. El problema que motiva este desarrollo es claro: a medida que los modelos preentrenados demostraban mejoras consistentes en NLP, también aumentaban su tamaño, sus requerimientos de memoria y su costo energético, dificultando su uso en contextos con restricciones de hardware o presupuestos limitados de entrenamiento e inferencia [11]. En ese marco, DistilBERT se presenta como una respuesta particularmente atractiva para aplicaciones reales, ya que busca retener la fortaleza contextual de BERT sin cargar con todo su peso operativo. Esta motivación dialoga de manera muy directa con una tesis aplicada como la presente, donde la comparación entre rendimiento y costo computacional resulta central para elegir modelos viables en problemas de clasificación a gran escala.

La idea central de DistilBERT es utilizar knowledge distillation durante la fase de preentrenamiento. En este esquema, un modelo grande y potente, llamado teacher — en este caso BERT—, guía el aprendizaje de un modelo más pequeño, llamado student, que intenta reproducir su comportamiento [11]. El valor de esta idea no reside solamente en reducir parámetros, sino en transferir al estudiante parte de los sesgos inductivos, regularidades y capacidades lingüísticas aprendidas por el modelo mayor.



A diferencia de enfoques de compresión puramente posteriores o específicos de tarea, DistilBERT aplica la destilación en el preentrenamiento general, permitiendo luego reutilizar el modelo comprimido en diferentes tareas mediante *fine-tuning*, igual que se hace con BERT [11]. Desde la perspectiva metodológica de este trabajo, esto es especialmente importante, porque convierte a DistilBERT en una alternativa realista a BERT cuando interesa conservar el paradigma de *transfer learning* pero disminuir el costo de experimentación y despliegue.

En términos de arquitectura, DistilBERT mantiene la estructura general encoder-only de

BERT, pero con simplificaciones deliberadas. El modelo elimina las token-type embeddings y el pooler, y reduce a la mitad el número de capas del encoder, conservando la dimensionalidad oculta para facilitar la transferencia de conocimiento desde el teacher [11]. Los autores explican que reducir el número de capas produce un mejor compromiso entre eficiencia y capacidad que reducir otras dimensiones del modelo, dado que muchas operaciones lineales ya están altamente optimizadas en frameworks modernos [11]. Además, el student se inicializa tomando una de cada dos capas del teacher, aprovechando que ambas redes comparten dimensionalidad. Esta decisión no es menor: la inicialización informada desde BERT mejora la convergencia del modelo comprimido y, según el estudio de ablación reportado, constituye un componente importante del desempeño final [11].

Uno de los aspectos más relevantes del paper es su función objetivo de entrenamiento. DistilBERT no se entrena solo con la pérdida clásica de modelado de lenguaje enmascarado, sino con una **triple loss** que combina: una pérdida de distilación sobre las salidas suaves del teacher, una pérdida MLM y una pérdida de similitud coseno entre estados ocultos del teacher y el student [11], según se describe a continuación:

$$\mathcal{L} = \alpha \mathcal{L}_{ce} + \beta \mathcal{L}_{mtm} + \gamma \mathcal{L}_{cos}$$

donde:

- $\mathcal{L}_{ce}$: distillation loss entre la distribución suave del teacher y la del student.
- $\mathcal{L}_{mtm}$: Masked Language Modeling loss.
- $\mathcal{L}_{cos}$: cosine embedding loss para alinear los estados ocultos del student con los del teacher.

La parte de distillation la describen como:

$$\mathcal{L}_{ce} = - \sum_i t_i \log(s_i)$$

Donde t_i es la probabilidad estimada por el teacher y s_i la del student para la clase/token i . Además, usan softmax con temperatura:

$$p_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

Aplicada tanto al teacher como al student durante entrenamiento; en inferencia usan $T = 1$.

La lógica es que el estudiante no solo aprenda a predecir tokens correctos, sino que también reproduzca la distribución de probabilidades del maestro y alinee la geometría interna de sus representaciones. El paper muestra, además, que esta combinación es efectiva: al remover componentes de la triple loss o usar inicialización aleatoria, el desempeño cae de manera apreciable en GLUE [11]. Esto sugiere que DistilBERT no es simplemente “un BERT más chico”, sino el resultado de una estrategia de compresión cuidadosamente diseñada para preservar conocimiento lingüístico útil.

Los resultados reportados por Sanh et al. explican por qué DistilBERT se volvió tan difundido en la práctica. Según el paper, el modelo reduce el tamaño de BERT en un 40 %, retiene aproximadamente el 97 % de sus capacidades de comprensión del lenguaje y es alrededor de un 60 % más rápido en inferencia [11]. En GLUE, DistilBERT obtiene un macro-score de 77,0 frente a 79,5 de BERT-base; en IMDb queda apenas 0,6 puntos

porcentuales por debajo; y en SQuAD se mantiene a una distancia moderada del modelo completo [11]. Además, el trabajo muestra su viabilidad para aplicaciones *on-device*, reportando que en una prueba sobre smartphone DistilBERT fue un 71 % más rápido que BERT-base y podía ejecutarse en un entorno móvil con un tamaño de modelo manejable [11]. Más allá de las cifras puntuales, lo importante para esta tesis es que el paper demuestra empíricamente que existe una alternativa compacta capaz de sostener gran parte del valor predictivo de BERT sin exigir el mismo costo operativo.

Para la clasificación arancelaria HS04, DistilBERT resulta particularmente pertinente por varias razones:

Ventajas
Conserva las ventajas conceptuales de BERT: representaciones contextuales, buena capacidad de desambiguación y compatibilidad con <i>fine-tuning</i> para clasificación de texto [10] [11].
Reduce de forma importante el costo de entrenamiento e inferencia, lo que puede ser decisivo cuando se busca iterar múltiples experimentos sobre un problema multiclase con miles de etiquetas y diferentes estrategias de preprocesamiento, balanceo y evaluación.
Por tratarse de descripciones comerciales generalmente cortas o medianas, el modelo puede explotar bien la self-attention sin enfrentar en exceso la limitación cuadrática de secuencias muy largas.
Su menor peso lo vuelve atractivo no solo para investigación, sino también para escenarios de despliegue institucional, como asistentes de clasificación o prototipos de recomendación integrables a flujos operativos aduaneros.

Desde una perspectiva comparativa, DistilBERT ocupa una posición metodológica muy interesante en este trabajo. Frente a Doc2Vec y FastText, representa un salto claro en calidad de representación lingüística, al introducir contextualización profunda y transferencia desde grandes corpus generales [10] [11]. Frente a BERT, en cambio, representa una decisión de ingeniería: sacrificar una pequeña porción de desempeño potencial a cambio de una ganancia importante en velocidad, tamaño y costo de uso [11]. En un trabajo experimental como este, esa posición intermedia es valiosa, porque permite evaluar no solo qué modelo clasifica mejor, sino también cuál ofrece el mejor equilibrio entre precisión y viabilidad práctica. Este punto es particularmente importante en aplicaciones aduaneras, donde no siempre se dispone de infraestructura abundante ni se busca necesariamente maximizar una métrica aislada al costo que sea.

También conviene señalar algunas limitaciones de DistilBERT:

Limitaciones
Por su propia naturaleza comprimida, puede perder parte de la capacidad expresiva de BERT en tareas especialmente complejas o en dominios donde los matices semánticos son muy finos [11]. Si bien conserva la mayor parte del rendimiento general, no iguala por completo al teacher en todos los benchmarks.
Al haber sido destilado desde un modelo general del inglés, sigue siendo sensible a la calidad del ajuste fino en el dominio específico. En clasificación arancelaria esto implica que cuestiones como desbalance entre códigos, ruido en las descripciones, presencia de abreviaturas, medidas técnicas o nombres comerciales continúan siendo desafíos relevantes.

En otras palabras, DistilBERT reduce el costo del modelo, pero no elimina la necesidad de un buen diseño experimental, curación de datos y análisis de error.

En síntesis, DistilBERT puede entenderse como una versión comprimida y eficiente de BERT que conserva gran parte de su valor metodológico y predictivo, pero con mejores condiciones operativas para entrenamiento y despliegue [11]. Para este trabajo, su inclusión queda plenamente justificada porque combina tres atributos especialmente deseables: representaciones contextuales de alta calidad, compatibilidad con estrategias modernas de *transfer learning* y una relación rendimiento/costo particularmente atractiva para problemas reales de clasificación de texto. En el marco de la clasificación arancelaria HS04 sobre descripciones comerciales, esto lo convierte en un candidato especialmente sólido para superar a los baselines clásicos y, al mismo tiempo, ofrecer una alternativa más ligera que BERT completo.

Diseño experimental

Este trabajo requirió un desarrollo experimental para el entrenamiento y la validación de distintos modelos para la clasificación de texto, en virtud de los objetivos, usando la muestra descrita en el Análisis exploratorio. Esto incluyó el entrenamiento de modelos de referencia (baselines) y de modelos propuestos (DistilBERT en sus variantes de entrenamiento), superadores en la resolución de la problemática. En cada caso, hubo que definir un universo acotado de fuentes de variación, para considerar el impacto en métricas de rendimiento.

Baselines

El diseño experimental de los baselines buscó aislar dos fuentes de variación: por un lado, la familia de embeddings utilizada para representar el texto; por otro, el grado de intervención aplicado sobre la descripción comercial antes del entrenamiento:

Doc2Vec	FastText	Texto de input
D2V_RAW	FT_RAW	`GOODS_DESCRIPTION` en crudo
D2V_PREPRO	FT_PREPRO	`PREPRO_DESCRIPTION` luego de normalización y remoción de ruido
D2V_PREPRO+NGRAM	FT_PREPRO+NGRAM	`PREPRO_DESCRIPTION` + NGRAM_DESCRIPTION` para incorporar combinaciones locales de términos

La evaluación se realizó mediante 10 iteraciones con muestreo Bootstrap, usando para entrenar una **PC laptop con CPU 12th Gen Intel(R) Core(TM) i5-12500H (2.50 GHz) y 16 GB de RAM**. En cada iteración se tomó un 5 % del corpus para validación, equivalente a 13.389 observaciones, mediante muestreo con reemplazo. Luego, el conjunto de entrenamiento se construyó excluyendo del corpus original los índices seleccionados para validación. De este modo, el procedimiento introduce variación entre iteraciones y permite observar no solo la media de desempeño, sino también su estabilidad.

Las métricas reportadas fueron `Top-1` a `Top-5 accuracy`. La elección de métricas

acumuladas de tipo top-k es consistente con el objetivo práctico del problema: asistir la clasificación arancelaria proponiendo un conjunto acotado de códigos candidatos, más que forzar una única predicción exacta. En términos operativos, un sistema de recomendación aduanera es más útil cuando logra ubicar el código correcto dentro de las primeras alternativas sugeridas, aun cuando no siempre ocupe la primera posición.

Doc2Vec

Para `Doc2Vec` se utilizó la implementación de `gensim`, bajo una configuración de hiperparámetros predefinida:

Parametro	Valor	Significado
dm	0	Entrenamiento PV-DBOW
vector_size	254	Dimensiones del embedding
window	5	Tamaño de la ventana de contexto +/-
min_count	1	Ignora palabras únicas en el corpus
sample	1e-4	Umbral de undersampleo de palabras muy frecuentes
alpha	0.025	Learning rate inicial
min_alpha	0.001	Learning rate minimo
epochs	50	Épocas de entrenamiento

.La construcción del vocabulario se realizó en cada iteración únicamente con el conjunto de entrenamiento, y el modelo se entrenó utilizando todos los núcleos disponibles.

Un aspecto importante de esta implementación es que cada texto de entrenamiento se representó como un `TaggedDocument`, pero utilizando el código `HS04` como tag del documento. En otras palabras, el espacio latente no quedó indexado por identificadores únicos de observación, sino directamente por las clases objetivo. Esta decisión convierte al modelo en una variante de recomendación por similitud contra vectores asociados a códigos HS04, más que en un `Doc2Vec` clásico seguido por un clasificador supervisado independiente.

La inferencia se realizó con `infer\_vector` sobre cada descripción del conjunto de validación y, a partir de ese embedding inferido, se recuperaron los cinco vectores más similares dentro de `model.dv`. Como las etiquetas almacenadas en ese espacio son directamente los códigos `HS04`, la salida del modelo ya queda expresada como un ranking de partidas candidatas. Finalmente, la evaluación acumuló aciertos para `Top-1` a `Top-5`.

Desde el punto de vista experimental, esta formulación tiene dos consecuencias. La primera es que `Doc2Vec` actúa como un recomendador de códigos por proximidad semántica en el espacio latente. La segunda es que el preprocesamiento puede tener un efecto dual: si elimina ruido y refuerza regularidades útiles, mejora el embedding; pero si remueve demasiada especificidad léxica, puede deteriorar la discriminación entre códigos cercanos.

FastText

Para `FastText` se implementó un wrapper específico (`FastTextDocVec`) sobre la versión de `gensim`, con el objetivo de aproximar una interfaz comparable a la usada en `Doc2Vec`. La configuración utilizada fue:

Parametro	Valor	Significado
sg	1	Entrenamiento skip-gram
dim	254	Dimensiones del embedding
window	5	Tamaño de la ventana de contexto +/-
min_count	1	Ignora palabras únicas en el corpus
min_n	3	Mínimo largo del subcarácter para usar como token
max_n	6	Máximo largo del subcarácter para usar como token
epochs	50	Épocas de entrenamiento

En consecuencia, el modelo operó bajo una lógica `skip-gram` con información de subpalabras de longitud 3 a 6 caracteres.

La mecánica de representación difiere de `Doc2Vec`. Una vez entrenado el modelo de palabras, el embedding de cada documento se obtuvo como el promedio de los embeddings de sus palabras, seguido de normalización L2. Esos vectores documentales quedaron almacenados para todas las observaciones de entrenamiento. Durante la inferencia, el texto de validación se proyecta al mismo espacio y luego se calcula la similitud coseno respecto de todos los embeddings documentales del entrenamiento.

La recomendación final no surge directamente de los documentos más cercanos, sino de una agregación por etiqueta: el modelo toma los 50 vecinos más próximos, suma sus puntajes de similitud por código `HS04` y devuelve el ranking de clases con mayor score acumulado. Por lo tanto, `FastText` se comporta aquí como un esquema híbrido entre embeddings subléxicos y recuperación por vecinos más cercanos con votación ponderada.

Esta formulación es especialmente pertinente para descripciones comerciales breves y ruidosas. Dado que `FastText` trabaja con n-gramas de caracteres, puede capturar regularidades morfológicas, abreviaturas, variantes ortográficas y fragmentos parcialmente informativos que suelen aparecer en catálogos o descripciones de comercio exterior. Al mismo tiempo, el costo computacional de comparar contra todos los documentos de entrenamiento es mayor que en la versión usada de `Doc2Vec`, lo cual aparece reflejado en los tiempos totales de corrida.

DistilBERT

El bloque experimental con `DistilBERT` tuvo un objetivo más acotado que el de los baselines clásicos. En lugar de comparar familias de embeddings y variantes de texto, aquí se fijó una única representación de entrada, `GOODS\_DESCRIPTION` en crudo, y se estudió el efecto de distintos grados de ajuste fino sobre un encoder preentrenado. La comparación, por lo tanto, no se organiza alrededor del preprocesamiento textual sino del régimen de entrenamiento del modelo: `FE` (`fixed encoder`), `PFT` (`partial fine-tuning`) y `FFT` (`full fine-tuning`), a lo que luego se suma una corrida `FFT11` como variante refinada del esquema de entrenamiento completo.

El uso de `GOODS\_DESCRIPTION` en crudo no implica ausencia total de procesamiento, sino la omisión de etapas manuales de limpieza lingüística agresiva —por ejemplo, eliminación de stopwords, stemming, lematización o construcción manual de n-gramas—, delegando la preparación de la entrada al pipeline nativo del modelo preentrenado y su tokenizador. La justificación de esta decisión se apoya en la propia

lógica de diseño de BERT y sus variantes comprimidas.

En primer lugar, BERT fue concebido como un modelo de representaciones bidireccionales profundas capaz de ser ajustado a múltiples tareas downstream con modificaciones mínimas de arquitectura [10]. Su funcionamiento parte de texto natural tokenizado mediante subunidades, lo que le permite modelar contexto izquierdo y derecho en todos los niveles de representación [10]. En consecuencia, la calidad de sus embeddings no depende de una simplificación previa del texto al estilo de los enfoques clásicos basados en bolsa de palabras o embeddings estáticos, sino de la capacidad del encoder para aprender regularidades contextuales directamente sobre la secuencia de entrada. DistilBERT, por su parte, conserva esta lógica de operación, ya que fue propuesto como una versión destilada de BERT que mantiene gran parte de sus capacidades de comprensión del lenguaje, pero con menor costo computacional [11]. Por lo tanto, desde el punto de vista experimental, resulta coherente preservar el texto en una forma lo más cercana posible a la observada por el modelo durante su preentrenamiento.

En segundo lugar, la tokenización subléxica utilizada por esta familia de modelos reduce la necesidad de aplicar transformaciones manuales orientadas a normalizar variantes superficiales. En lugar de depender exclusivamente de palabras completas, BERT segmenta los textos en subpalabras, lo que le permite manejar términos raros, abreviaturas, sufijos, prefijos, errores menores y variantes morfológicas dentro de un vocabulario controlado [10]. Esta propiedad vuelve menos necesaria la aplicación de técnicas como stemming o lematización, que en modelos tradicionales se empleaban para reducir dispersión léxica. En el contexto de descripciones comerciales, donde abundan nombres compuestos, códigos, abreviaturas técnicas y expresiones no estandarizadas, esta característica es especialmente valiosa, ya que permite conservar mayor fidelidad respecto del texto original sin perder capacidad de generalización semántica.

En tercer lugar, existe una razón vinculada a la preservación de información potencialmente útil. Diversos análisis sobre BERT muestran que sus mecanismos de atención capturan no solo relaciones semánticas generales, sino también patrones sintácticos y estructurales finos, incluyendo dependencias gramaticales, delimitadores y otros elementos superficiales de la secuencia [25]. Esto implica que una etapa de limpieza manual excesiva puede eliminar señales que el modelo es capaz de explotar durante la clasificación. Desde esta perspectiva, remover puntuación, palabras funcionales o ciertas formas léxicas con el objetivo de “reducir ruido” no necesariamente favorece el desempeño, y en algunos casos puede empobrecer la estructura contextual disponible para el encoder.

Todos los entrenamientos con DistilBERT comparten la misma arquitectura base:

Tokenizer	DistilBertTokenizerFast
Encoder	distilbert-base-uncased
max_length	300

Con una cabeza de clasificación compuesta por:

Linear layer	In 768	out 1024
ReLU		

BatchNorm1d		
Dropout 0.3		
Final layer	In 1024	out 1133

Y fueron procesados en Jobs usando **GCP Vertex AI – Customs Training** empleando una máquina virtual g2-standard-4 con CPU de 2 núcleos físicos Intel Xeon, 16 GB de RAM y 10 GB/s de ancho de banda, además de un acelerador GPU NVIDIA L4 con arquitectura Ada Lovelace 24 GB GDDR6, 300 GB/s de ancho de banda y 30.3 TFLOPS en FP32. Otros parámetros relevantes de los entrenamientos:

Función de pérdida	CrossEntropyLoss
Optimizador	Adam
Early stopping	Patience = 3
Warm-up	1 epoch
num_workers	2

Esto vuelve razonable atribuir las diferencias de desempeño principalmente al régimen de ajuste fino y no al hardware. En `FE`, `PFT` y `FFT` la validación usa `test\_fraction=0.05` con `bootstrap=True`, es decir, un 5 % del corpus muestreado con reemplazo. Esto implica una advertencia metodológica importante: la muestra de validación puede repetir observaciones y no coincide con una particion hold-out clásica. En `FFT11`, en cambio, se mantiene el 5 % pero se explicita `--no-bootstrap`, por lo que la validación pasa a ser un muestreo sin reemplazo y resulta más interpretable como estimación de generalización.

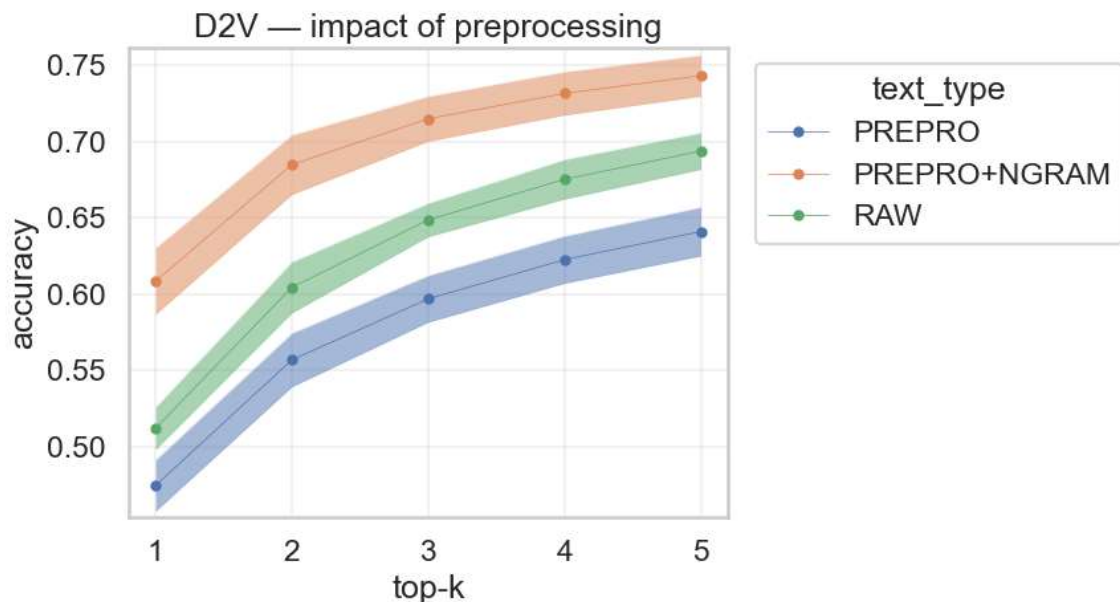
Resultados

La experimentación planteada en la sección Diseño experimental se llevó a cabo recopilando métricas de rendimiento y tiempos de ejecución, además de logs detallados para auditar los entrenamientos y validaciones.

Baselines

Doc2Vec

Los entrenamientos de Doc2Vec (D2V) muestran rendimientos que llegan a un accuracy Top5 de ~74 %, en el mejor caso: PREPRO+NGRAM. Para visualizar esto, se grafica la métrica accuracy con banda de 3 desviaciones estándar, en los primeros códigos recomendados (top-k):



Los modelos con PREPRO pierden rendimiento frente al modelo RAW, indicando pérdida de información en el proceso de limpieza. Luego, el caso PREPRO+NGRAM sí logra superar al RAW notablemente con +9pp en el TOP1 y +5pp en el TOP5.

D2V RAW

Estadísticos de las iteraciones:

	top_1_acc	top_2_acc	top_3_acc	top_4_acc	top_5_acc
count	10	10	10	10	10
mean	0,5120	0,6044	0,6486	0,6752	0,6937
std	0,0046	0,0056	0,0036	0,0043	0,0040
min	0,5015	0,5924	0,6417	0,6670	0,6877
0,2500	0,5104	0,6028	0,6465	0,6729	0,6905
0,5000	0,5125	0,6049	0,6487	0,6756	0,6946
0,7500	0,5143	0,6059	0,6502	0,6781	0,6952
max	0,5191	0,6133	0,6548	0,6821	0,7008

Tiempo por iteración: 4.9 minutos.

D2V PREPRO

Estadísticos de las iteraciones:

	top_1_acc	top_2_acc	top_3_acc	top_4_acc	top_5_acc
count	10	10	10	10	10
mean	0,4745	0,5568	0,5968	0,6225	0,6409
std	0,0056	0,0058	0,0051	0,0051	0,0053
min	0,4637	0,5440	0,5862	0,6126	0,6305
0,2500	0,4714	0,5541	0,5943	0,6193	0,6376
0,5000	0,4745	0,5583	0,5978	0,6244	0,6425
0,7500	0,4775	0,5592	0,5998	0,6255	0,6441
max	0,4836	0,5643	0,6040	0,6287	0,6487

Tiempo por iteración: 7.4 minutos.

D2V PREPRO+NGRAM

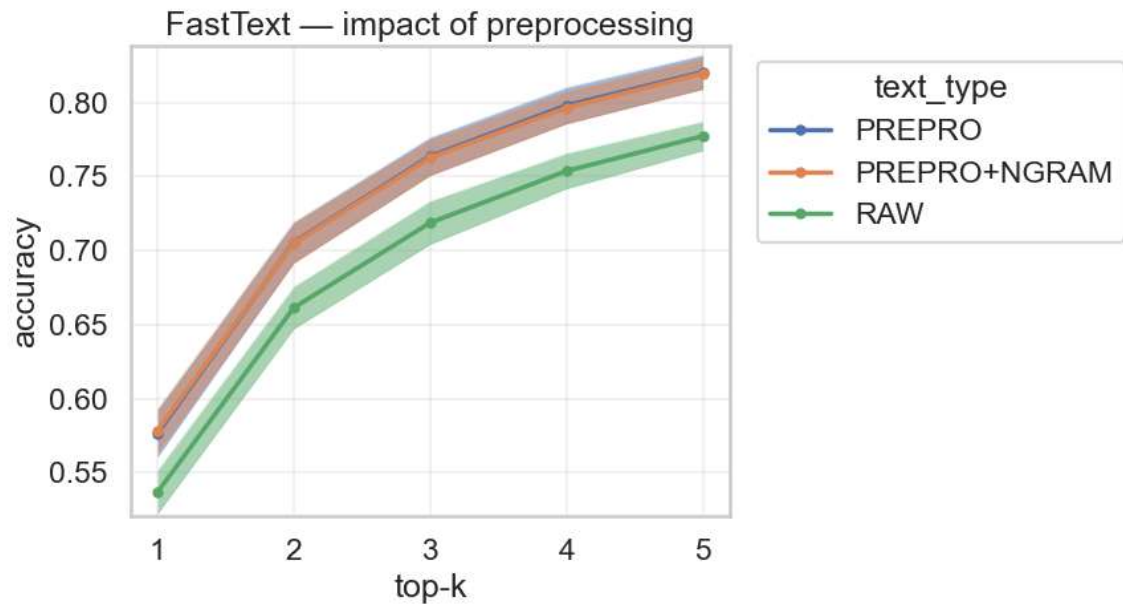
Estadísticos de las iteraciones:

	top_1_acc	top_2_acc	top_3_acc	top_4_acc	top_5_acc
count	10	10	10	10	10
mean	0,6084	0,6848	0,7149	0,7315	0,7431
std	0,0072	0,0065	0,0049	0,0047	0,0045
min	0,5959	0,6739	0,7071	0,7239	0,7352
0,2500	0,6072	0,6816	0,7131	0,7286	0,7404
0,5000	0,6094	0,6857	0,7154	0,7322	0,7431
0,7500	0,6117	0,6886	0,7170	0,7337	0,7453
max	0,6213	0,6955	0,7238	0,7397	0,7511

Tiempo por iteración: 20.4 minutos.

FastText

Los entrenamientos de FastText (FT) muestran rendimientos que llegan a un accuracy Top5 de ~82 %, en el mejor caso: PREPRO. Para visualizar esto, se grafica la métrica accuracy con banda de desviación estándar, en los primeros códigos recomendados (top-k):



En FT, los modelos con PREPRO ganan rendimiento frente al modelo RAW, indicando que este modelo logra aprovechar la limpieza, sin perder información relevante. En este caso, PREPRO incluso supera marginalmente al PREPRO+NGRAM, indicando que los n-gramas no aportan información relevante al método FT.

FT RAW

Estadísticos de las iteraciones:

	top_1_acc	top_2_acc	top_3_acc	top_4_acc	top_5_acc
count	10	10	10	10	10
mean	0,5370	0,6615	0,7188	0,7537	0,7773
std	0,0049	0,0048	0,0049	0,0040	0,0033
min	0,5307	0,6564	0,7118	0,7488	0,7731
0,2500	0,5337	0,6591	0,7173	0,7514	0,7756
0,5000	0,5365	0,6597	0,7175	0,7529	0,7769
0,7500	0,5390	0,6634	0,7189	0,7552	0,7786
max	0,5480	0,6726	0,7305	0,7631	0,7846

Tiempo por iteración: 23.4 minutos.

FT PREPRO

Estadísticos de las iteraciones:

	top_1_acc	top_2_acc	top_3_acc	top_4_acc	top_5_acc
count	10	10	10	10	10
mean	0,5766	0,7056	0,7637	0,7980	0,8205
std	0,0053	0,0045	0,0042	0,0041	0,0038
min	0,5685	0,6998	0,7581	0,7907	0,8157
0,2500	0,5751	0,7029	0,7606	0,7950	0,8173
0,5000	0,5763	0,7057	0,7634	0,7985	0,8203
0,7500	0,5791	0,7071	0,7666	0,8002	0,8227
max	0,5864	0,7151	0,7715	0,8041	0,8279

Tiempo por iteración: 30.2 minutos.

FT PREPRO+NGRAM

Estadísticos de las iteraciones:

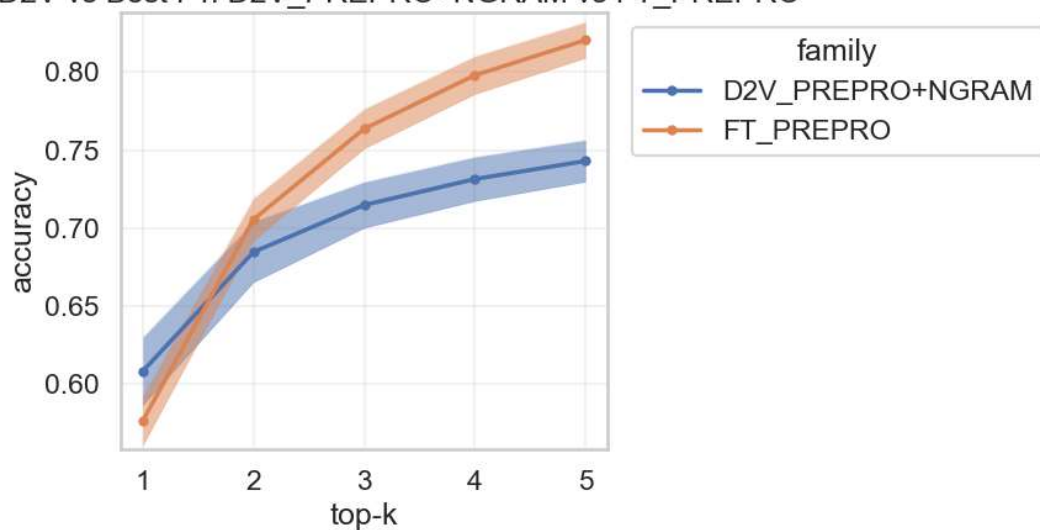
	top_1_acc	top_2_acc	top_3_acc	top_4_acc	top_5_acc
count	10	10	10	10	10
mean	0,5784	0,7052	0,7628	0,7965	0,8197
std	0,0048	0,0046	0,0041	0,0037	0,0036
min	0,5714	0,6989	0,7563	0,7912	0,8146
0,2500	0,5744	0,7013	0,7602	0,7944	0,8176
0,5000	0,5796	0,7055	0,7624	0,7967	0,8189
0,7500	0,5826	0,7068	0,7645	0,7980	0,8210
max	0,5836	0,7135	0,7701	0,8035	0,8262

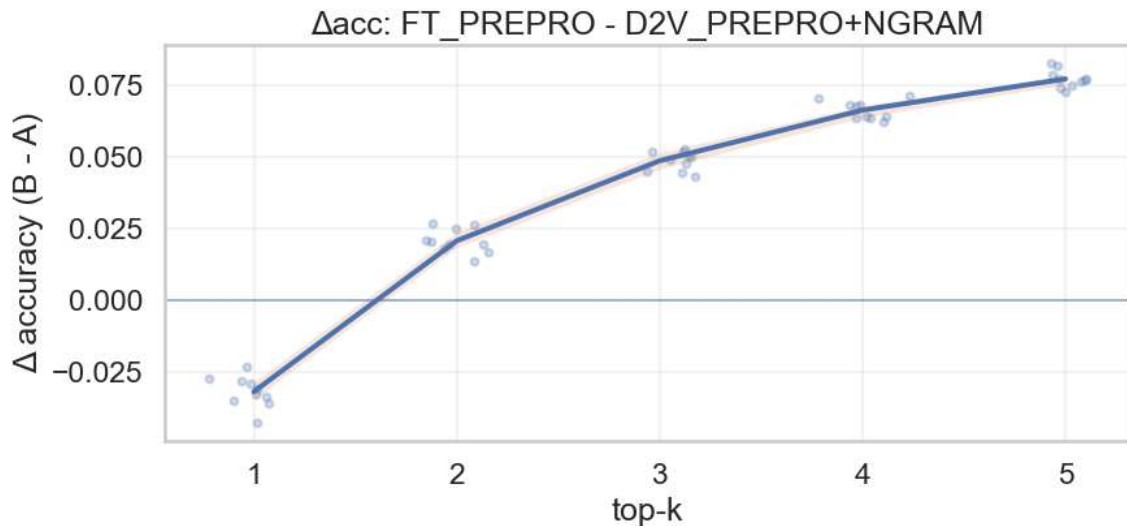
Tiempo por iteración: 53.0 minutos.

Best Baseline

Comparando el mejor D2V vs. el modelo FT, vemos que D2V_PREPRO+NGRAM supera al FT_PREPRO en el Top1 por ~2.5pp. Sin embargo, desde el Top2 en adelante, el FT_PREPRO gana consistentemente, llegando a una brecha máxima en Top5 de ~7.5pp.

Best D2V vs Best FT: D2V_PREPRO+NGRAM vs FT_PREPRO

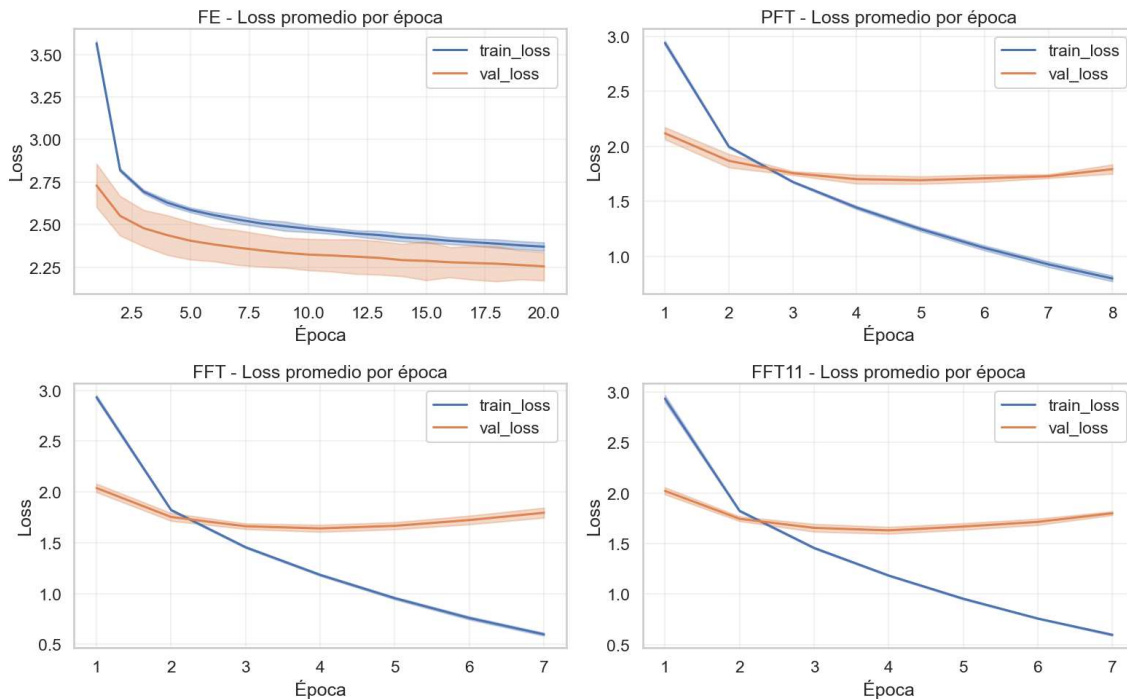




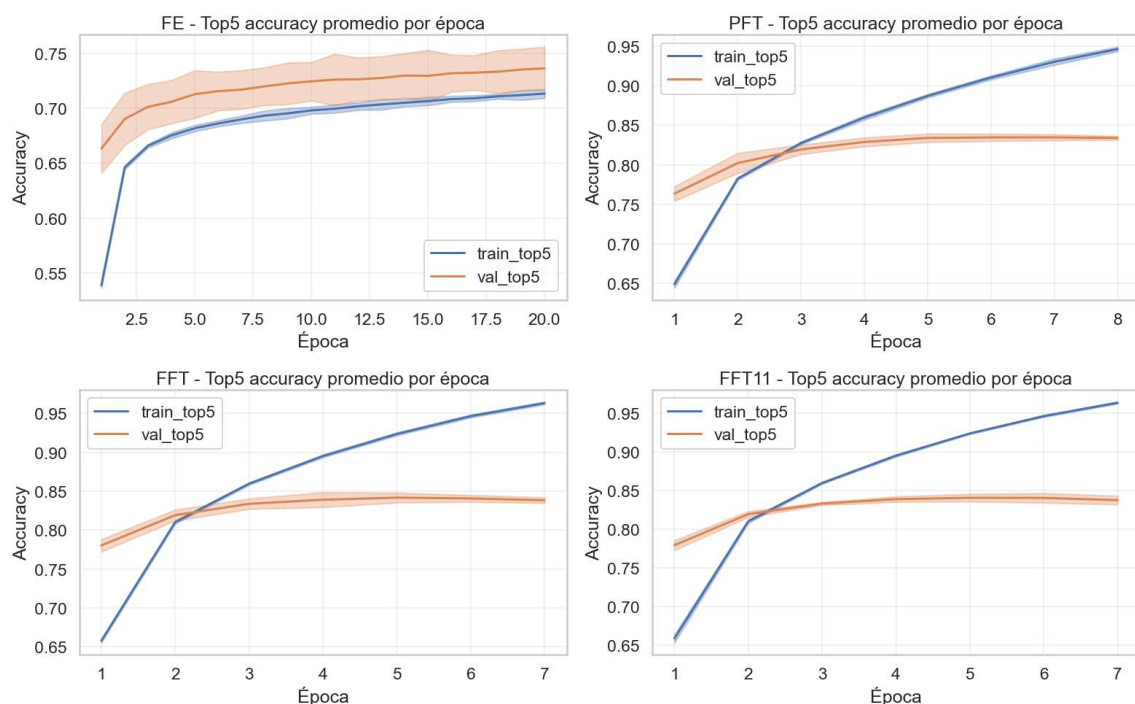
Esto plantea que el espacio de representaciones de D2V_PREPRO+NGRAM puede ser más ruidoso que el generado por FT_PREPRO. A medida que buscamos candidatos cercanos al texto a clasificar, FT_PREPRO encuentra menos ruido (menos clases incorrectas cercanas) que el D2V_PREPRO+NGRAM, aunque los primeros Top1 pueda equivocarse más.

DistilBERT

El proceso de entrenamiento de los distintos experimentos con DistilBERT (DBERT) muestra diferencias marcadas entre FE (fixed-encoder), PFT (partial-finetuning), FFT (full-finetuning) y FFT11 (full-finetuning sin bootstrap). Las pérdidas se observan en la siguiente gráfica, con una banda de 3 desviaciones estándar:



Luego, el Top5 también es graficado para ver la evolución del entrenamiento en esta métricas:



Es notable lo inestable que resulta la validación en el entrenamiento FE, y como mejora en el finetuning del encoder. Es posible que parte de esta variabilidad provenga del bootstrap utilizado en FE, PFT & FFT. En FFT11 (sin bootstrap) se observa una validación más estable que en su par FFT (con bootstrap).

Estos entrenamientos, excepto FE, muestran signos de sobreajuste en las últimas épocas, especialmente al comparar la evolución de la pérdida de entrenamiento con la de validación.

train_type	best_epoch	best_mean_val_top5_acc	n_runs
FE	20	0,7365	5
FFT	5	0,8419	5
FFT11	5	0,8406	5
PFT	7	0,8346	5

Detalles de los jobs para DistilBERT

Repasamos los detalles de la ejecución de cada job y evaluamos los resultados obtenidos.

Fixed encoder job `job\_fe.yml`

Este job implementa el escenario más conservador. En Código equivale a `fine\_tune=False` y `n\_finetune\_layers=0`, por lo que el encoder `DistilBERT` queda completamente congelado y solo se entrena la cabeza de clasificación. Es, en los hechos, una prueba de cuánta información clasificatoria puede extraerse de los embeddings preentrenados sin adaptar el lenguaje del modelo al dominio arancelario.

Configuración

docker image	hs-classifier:v10
regimen	FE
iteraciones	5

max_epochs	30
batch_size	512
lr	5e-4
validación	5 %
bootstrap	True

Métricas

FE	mean	std
top_1_acc	0,5267	0,0043
top_2_acc	0,6286	0,0046
top_3_acc	0,6793	0,0058
top_4_acc	0,7124	0,0061
top_5_acc	0,7371	0,0060

El comportamiento por época muestra una convergencia lenta y relativamente estable. Los mejores puntos de validación aparecen recién al final del horizonte de entrenamiento, entre las épocas 19 y 20, lo que es consistente con un esquema donde solo aprende la capa superior y el encoder no puede reorganizar sus representaciones internas. Las validaciones de este régimen resultan altamente inestables entre iteraciones, en comparación con los siguientes Jobs, posiblemente explicado parcialmente por el bootstrap True.

Partial finetuning job `job\_pft.yml`

Este job implementa el caso intermedio. En Código corresponde a `fine\_tune=True` y `n\_finetune\_layers=2`, es decir, se descongelan únicamente los dos últimos bloques transformer de `DistilBERT`. La idea experimental es permitir cierta adaptación al dominio sin asumir el costo completo, ni el riesgo de sobreajuste, de un fine-tuning total.

Configuración

docker image	hs-classifier:v10
regimen	PFT
iteraciones	5
max_epochs	12
batch_size	128
lr	1e-4
validación	5 %
bootstrap	True

Métricas

PFT	mean	std
top_1_acc	0,6507	0,0028
top_2_acc	0,7463	0,0027
top_3_acc	0,7901	0,0029
top_4_acc	0,8148	0,0023
top_5_acc	0,8330	0,0020

En `PFT` la mejora respecto de `FE` es inmediata. Los mejores valores de validación aparecen temprano, entre las épocas 4 y 5, aunque el `early stopping` deja corridas efectivas de 7 u 8 épocas. Esto sugiere que una porción limitada del encoder alcanza para capturar regularidades del dominio que la cabeza sola no logra absorber.

Full finetuning job `job\_fft.yml`

Este job implementa el ajuste fino completo de `DistilBERT`. En código se expresa como `fine\_tune=True` y `n\_finetune\_layers=0`, donde el cero no significa congelar sino exactamente lo contrario: liberar todos los parámetros del encoder. Conceptualmente, este es el escenario donde el modelo tiene mayor capacidad para reacomodar sus representaciones semánticas al problema HS04.

Configuración

docker image	hs-classifier:v10
regimen	FFT
iteraciones	5
max_epochs	7
batch_size	128
lr	5e-5
validación	5 %
bootstrap	True

Métricas

FFT	mean	std
top_1_acc	0,6627	0,0048
top_2_acc	0,7569	0,0029
top_3_acc	0,7977	0,0034
top_4_acc	0,8228	0,0036
top_5_acc	0,8392	0,0032

El resultado más interesante de este job no es solo el mejor nivel de accuracy, sino también la forma de la curva. En las cinco iteraciones el mejor `val\_loss` se alcanza sistemáticamente en la época 4, y luego aparecen pequeñas oscilaciones o deterioros. Esto sugiere que, una vez habilitado el ajuste fino total, el modelo absorbe muy rápido la señal útil del dominio.

Full finetuning job sin bootstrap `job\_fft11.yml`

Este job conserva la logica de `FFT` pero introduce dos cambios operativos relevantes. Primero, usa la imagen `v11`, que refleja una versión posterior del pipeline. Segundo, desactiva el bootstrap mediante `--no-bootstrap`. Experimentalmente, `FFT11` funciona como una repetición controlada del mejor régimen anterior bajo una validación más limpia.

Configuración

docker image	hs-classifier:v11
regimen	FFT

iteraciones	5
max_epochs	7
batch_size	128
lr	5e-5
validación	5 %
bootstrap	False

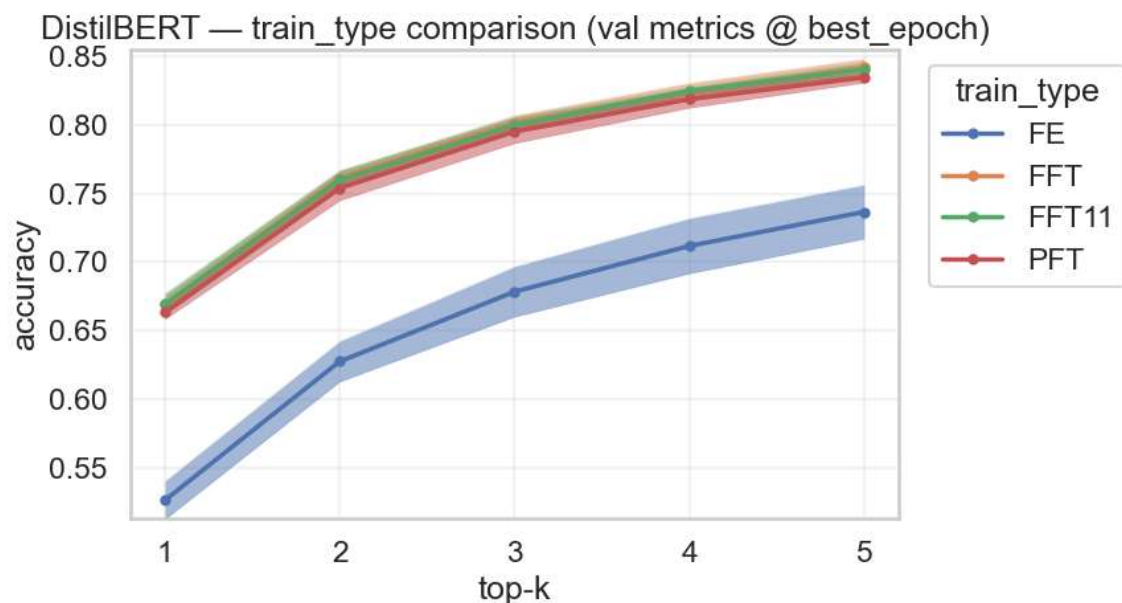
Métricas

FFT11	mean	std
top_1_acc	0,6637	0,0015
top_2_acc	0,7553	0,0020
top_3_acc	0,7976	0,0019
top_4_acc	0,8222	0,0017
top_5_acc	0,8388	0,0012

Igual que en `FFT`, el mejor `val\_loss` aparece en la época 4 en las cinco corridas. La principal diferencia es la reducción marginal de la varianza entre iteraciones, pero sin ganar rendimiento en métricas. Los resultados de FFT11 son comparables con los de FFT, pero en estas iteraciones el bootstrap False demuestra métricas más estables.

Resultados de DistilBERT

Basados en la mejor época de cada tipo de entrenamiento, se evaluó el rendimiento de los modelos respecto a los top-k 1 a 5, incluyendo la variabilidad encontrada en las 5 iteraciones que se corrieron de forma independiente. Los resultados muestran aun FE de bajo rendimiento, contra modelos superadores: FFT11, FFT y un poco por debajo el PFT:

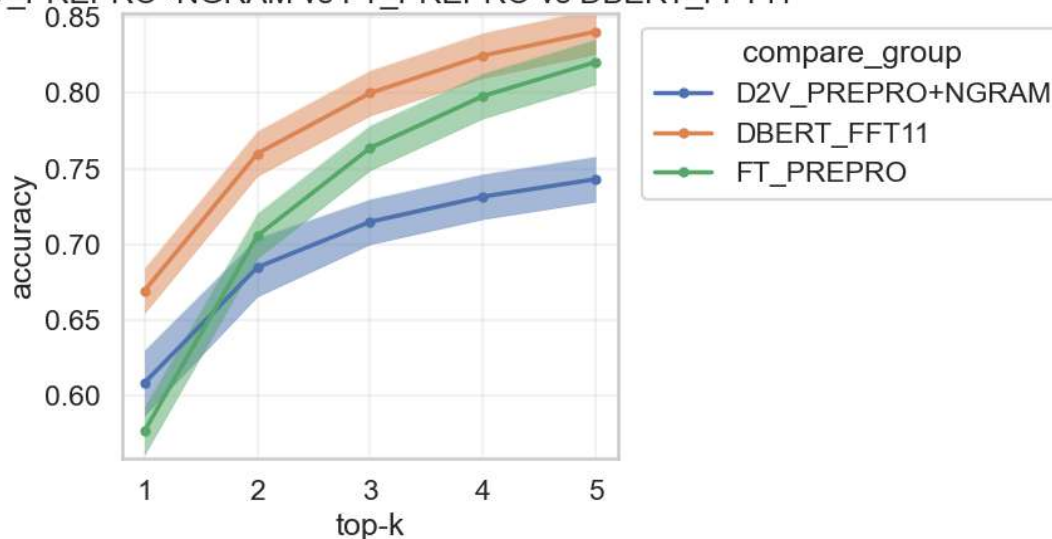


Luego, corresponde comparar al mejor de estos, el DBERT_FFT11, contra los baselines que destacaron en los experimentos antes descriptos.

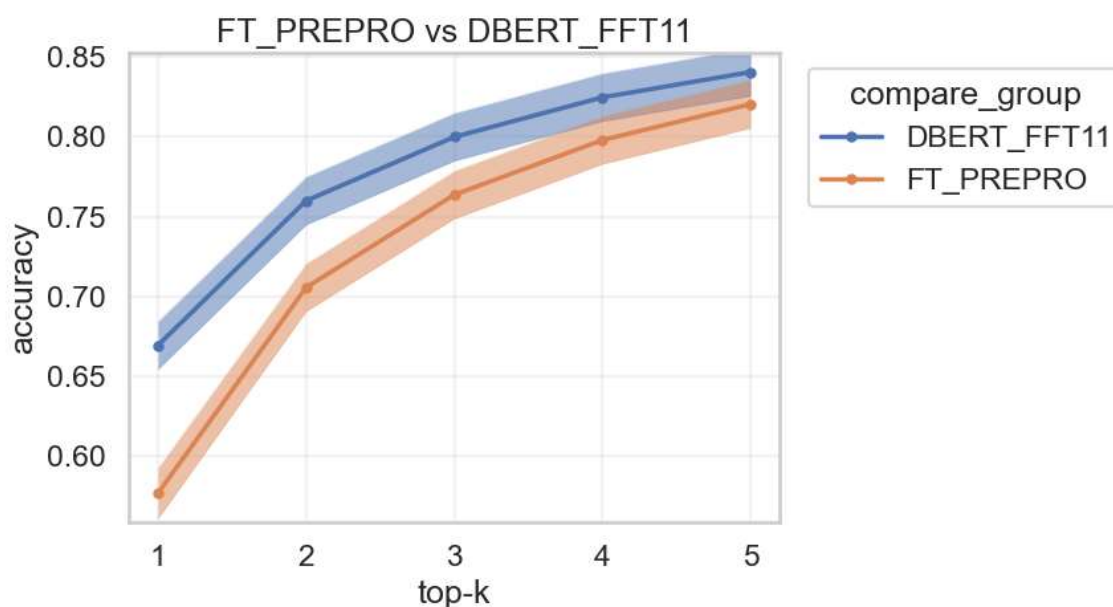
Comparación

Para extraer conclusiones sobre la bondad del DistilBERT en contraste con Doc2Vec & FastText, superponemos las curvas de accuracy con bandas de 3 desviaciones estándar, seleccionando a D2V_PREPRO+NGRAM & FT_PREPRO como baselines y DBERT_FFT11 como el mejor modelo DBERT:

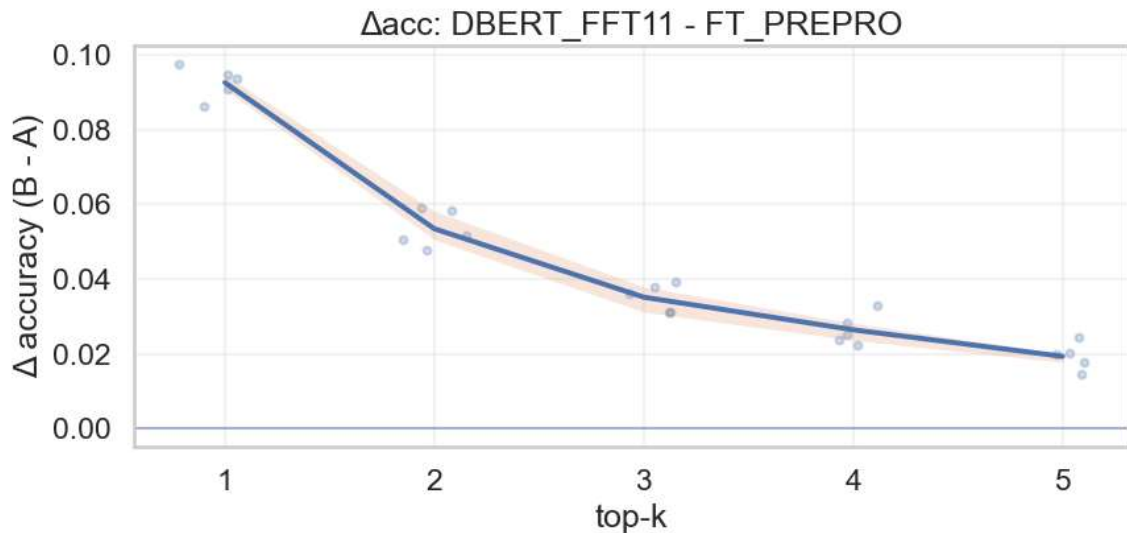
D2V_PREPRO+NGRAM vs FT_PREPRO vs DBERT_FFT11



Otra vez, resulta razonable descartar a D2V_PREPRO+NGRAM, a favor de los mejores FT & DBERT, para compararlos 1 vs. 1:



El BERT_FFT11 es claro ganador frente a los modelos baselines, ganando en promedio +9pp en Top1, lo que se extiende en Top5 a +2pp.



Análisis somero del error

Con el fin de complementar la evaluación del modelo DistilBERT para clasificación arancelaria en HS04, se abordó un análisis del error evaluado agregado por código. Mientras que las métricas generales de desempeño permiten conocer el rendimiento promedio del clasificador, no muestran con suficiente detalle en qué tipos de partidas el modelo funciona mejor o peor, ni qué características de los datos podrían estar asociadas a esas diferencias.

Cabe destacar que, este análisis podría extenderse y profundizar en la explicabilidad del modelo, tratando de identificar y comparar variables encontradas como relevantes, incluso sería propicio investigar activación y comportamiento dentro de las capas de la arquitectura Transformers. Sin embargo, esto lo dejamos para futuros trabajos y extensiones de esta tesis.

Entonces, para llevar a cabo este análisis simplificado, partimos de observar casos donde el modelo recomienda correctamente, siendo certero el Top1, vemos que la “Proba Top1” tiene un valor consistentemente alto:

Description	True Label	Top1	Proba Top1	Top2	Proba Top2	Top3	Proba Top3
PERFUMES-LOVE INTETION	3303	3303	0,94202	3307	0,03605	3302	0,01518
BEARING 6805 ZZ BRAND KG	8482	8482	0,99548	8483	0,00029	7319	0,00004
USED SOFA 3SEATER	9403	9403	0,62180	9401	0,36676	9404	0,00130
A9060530008 VALVE CAP (MERCEDES 3029 TRUCK SPARE PARTS)	8481	8481	0,47839	8409	0,38959	8431	0,01750
LS MAXI DRESS-	6204	6204	0,85633	6104	0,11877	6209	0,00677

CREAM							
HP LJ pro M501dn printer:EUR - J8H61A	8443	8443	0,97974	8471	0,00179	3707	0,00155
CABLE 12G2.5MMQ FG16OR16 0.6-1KV	8544	8544	0,97656	7413	0,01184	9001	0,00226
PULLEY 2W8494	8483	8483	0,95631	8448	0,00810	8431	0,00674
WALL LAMP 12W 3000K IP65 OUTDOOR	9405	9405	0,99566	8539	0,00110	9403	0,00049
SUPER STAR BRAID LONG LABELS	4821	4821	0,78264	3920	0,08534	3921	0,02357

Al mismo tiempo, si observamos casos donde el modelo no acierta en el Top1, encontramos Probas más bajas, pero también Probas más generosas en 2 y 3.

Description	True Label	Top1	Proba Top1	Top2	Proba Top2	Top3	Proba Top3
sterilizing cabinet	8419	9402	0,26592	9403	0,24737	9018	0,22774
High voltage insulating tape	3920	8546	0,44508	3919	0,23464	5906	0,08970
FLAME TREATMNET MACHINE	8419	8479	0,28622	8438	0,20511	8419	0,08504
1538921 PIPE	7306	8409	0,30679	7304	0,17232	8708	0,14397
250UL FOR 1 LIQUID IN-LINE MIXER	8421	8479	0,92469	8474	0,02225	8438	0,00438
AV13 Air Vent, BSA1T Bellows Sealed	8481	8414	0,17451	8481	0,15273	8415	0,13645
BELOW 16KW COPPER BRANCH PIPE ARBLN01621	7412	7411	0,13595	7307	0,08071	8607	0,08048
USED IMPACT DRILL MACHINES; BRAND; WURTH	8459	8467	0,70405	8459	0,10653	8205	0,06345
SPACER-BEARI	8431	8483	0,61041	8708	0,20186	8431	0,10243
LAMP FRAME	8708	9405	0,87065	8512	0,03999	8539	0,03502

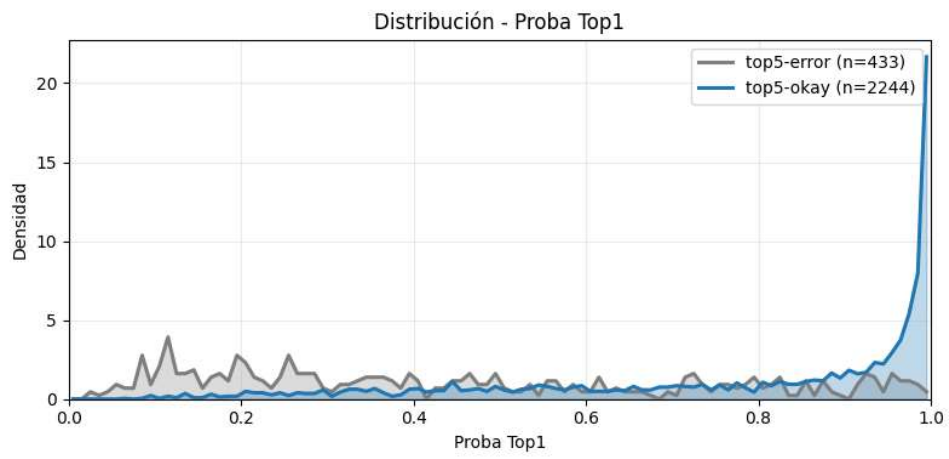
Luego, ordenamos los casos problemáticos, en donde el “True Label” no está en los Top5 recomendados, según la “Proba Top1” ascendente. Encontramos, por un lado, casos donde todas las Probas son bajas y que corresponden a descripciones escuetas, con poca información:

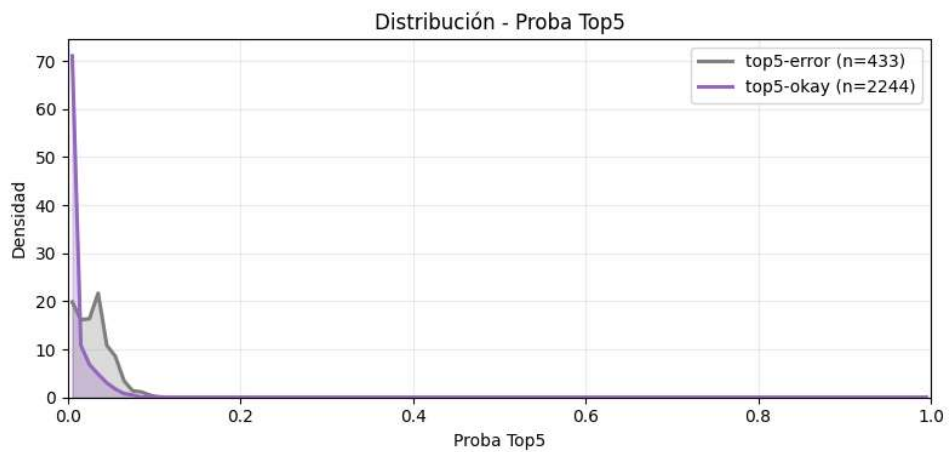
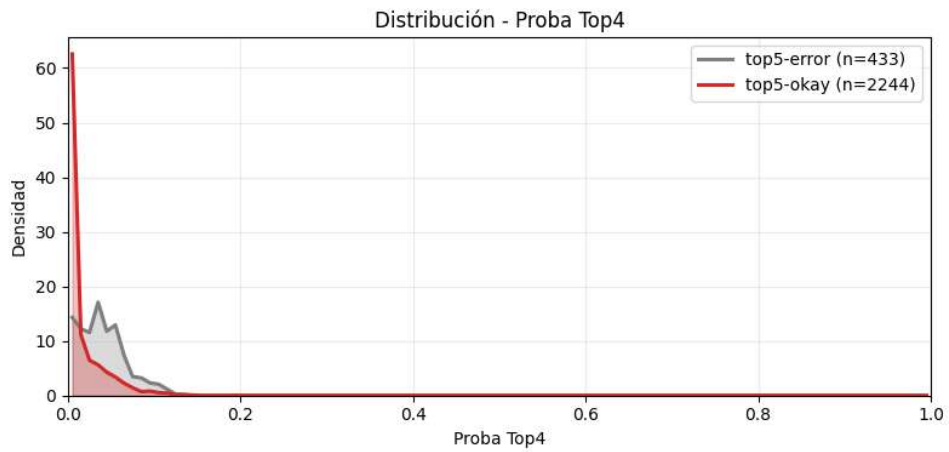
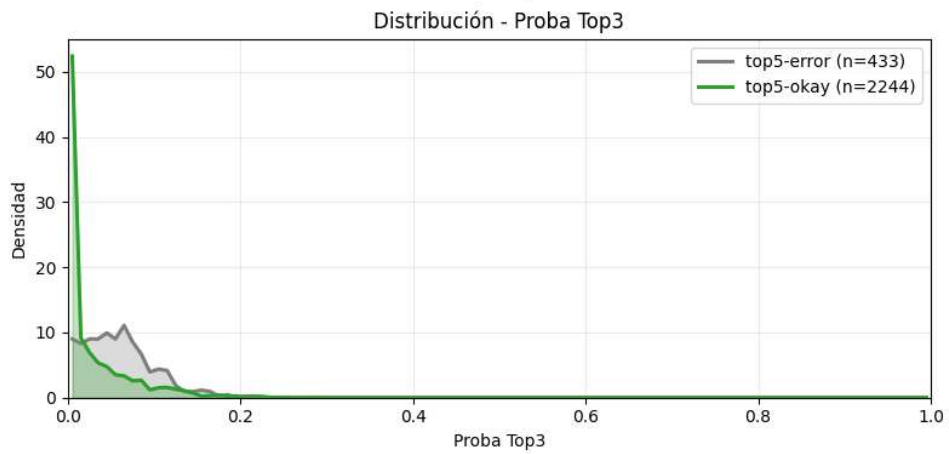
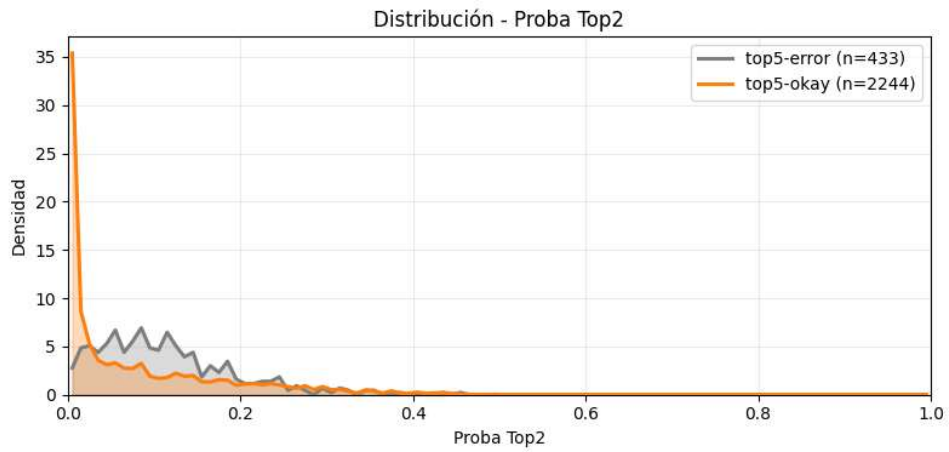
Description	True Label	Top1	Proba Top1	Top2	Proba Top2	Top3	Proba Top3
FGP - 48	3707	3811	0,02403	3824	0,02005	3907	0,01976
COLGANTE DE CUERO	7117	6204	0,02813	2103	0,02512	6907	0,02395
CABAS	4202	6203	0,03256	2710	0,02806	8544	0,02615
BUCKETS	6911	9506	0,04303	7318	0,03315	9403	0,03276
SOULDER	6217	8421	0,04365	8714	0,03247	3808	0,03146

Y, por otro, también hay casos donde el modelo se equivoca con altas “Probabs Top1”:

Description	True Label	Top1	Proba Top1	Top2	Proba Top2	Top3	Proba Top3
PISTON (SPARE PARTS FOR DIESEL ALTERNATOR; 12454441)	8413	8409	0,99516	8431	0,00127	8414	0,00058
HANDKERCHIEF, BRAND: ROSE,MODEL: 36749	5802	6213	0,99503	6310	0,00076	6117	0,00056
PRESIDENT CAMEMBERT 12X145G	0403	0406	0,99156	2101	0,00131	1904	0,00128
USED SCANIA TIPPERS	8701	8704	0,99129	8701	0,00254	8705	0,00091
THERMAL PRINTER,SEIKO-PART NO.LTP01-245-11	8473	8443	0,98961	8422	0,00367	8442	0,00064

Esto deja ver que el error puede tener diversas causas, partiendo de la base que “malas descripciones” probablemente deriven en una incorrecta clasificación. A su vez, el output del modelo tiene información para aportar en cuanto a la certeza en las recomendaciones, como se observa en la distribución de probabilidades en el test:





El objetivo de este análisis del error, aunque simple y somero, fue estudiar si el desempeño del modelo sobre cada código arancelario HS04 se relaciona con factores tales como:

- Cantidad de ejemplos disponibles en entrenamiento,
- Longitud de las descripciones,
- Complejidad de tokenización del texto,
- Confianza de las predicciones,
- Distribución de aciertos y errores por código.

Este análisis buscó entonces identificar patrones sistemáticos de dificultad, distinguir clases más o menos favorables para el modelo, y generar evidencia para comprender mejor las causas del error, más allá de las métricas globales tradicionales.

Enfoque metodológico

El análisis se estructuró a partir de una unidad de observación agregada: los códigos HS04. En lugar de limitarse a revisar ejemplos individuales clasificados correcta o incorrectamente, se resumió la información del conjunto de entrenamiento y del conjunto de prueba por grupos arancelarios, con el fin de caracterizar el comportamiento del modelo a nivel de clase/código.

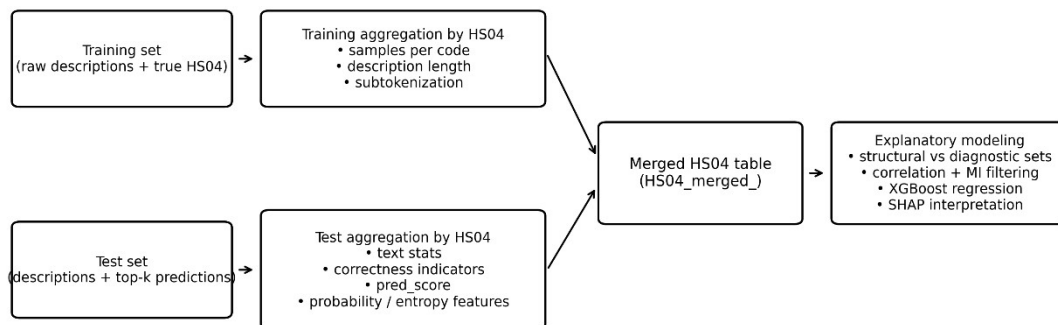
La lógica del procedimiento fue la siguiente:

1. Caracterizar cada código HS en el conjunto de entrenamiento, para medir cuánta información tuvo el modelo disponible durante el aprendizaje y qué rasgos textuales presentaban esas observaciones.
2. Caracterizar cada código HS en el conjunto de prueba, incorporando no solo variables descriptivas del texto, sino también indicadores derivados de las predicciones del modelo.
3. Integrar ambas fuentes de información en una tabla consolidada, que permitiera analizar conjuntamente estructura del entrenamiento, comportamiento en prueba y su desempeño.
4. Explorar relaciones entre variables explicativas y el desempeño, tanto de manera descriptiva como mediante modelos explicativos auxiliares.

De esta forma, el análisis permitió pasar de una evaluación puramente global del clasificador a una evaluación estructural del error, orientada a detectar qué atributos de los datos y de las predicciones se asocian con una mayor o menor dificultad de clasificación.

Análisis agregado por HS04

Como se planteó, el proceso de análisis incluye la construcción de estadísticos del conjunto de entrenamiento, test y resultados de las predicciones. En la siguiente figura, se observa el pipeline del análisis:



Caracterización del set de entrenamiento

En una primera etapa, se generaron tablas agregadas por código HS04 a partir del conjunto de entrenamiento. El objetivo fue describir la estructura de datos disponible para cada clase antes de evaluar el comportamiento del modelo.

Para cada código, se calcularon estadísticas tales como:

- cantidad de muestras de entrenamiento,
- longitud de las descripciones en cantidad de palabras,
- longitud de las descripciones en caracteres,
- indicador de subtokenización, entendido como una aproximación a la complejidad de segmentación del texto por parte del tokenizador del modelo.

Estas variables fueron resumidas mediante estadísticas descriptivas como cantidad, suma, media, mediana, mínimo, máximo y desvío estándar.

El resultado de esta etapa fue una caracterización estructural de cada código HS en función del material disponible para entrenamiento y de ciertas propiedades textuales de sus descripciones.

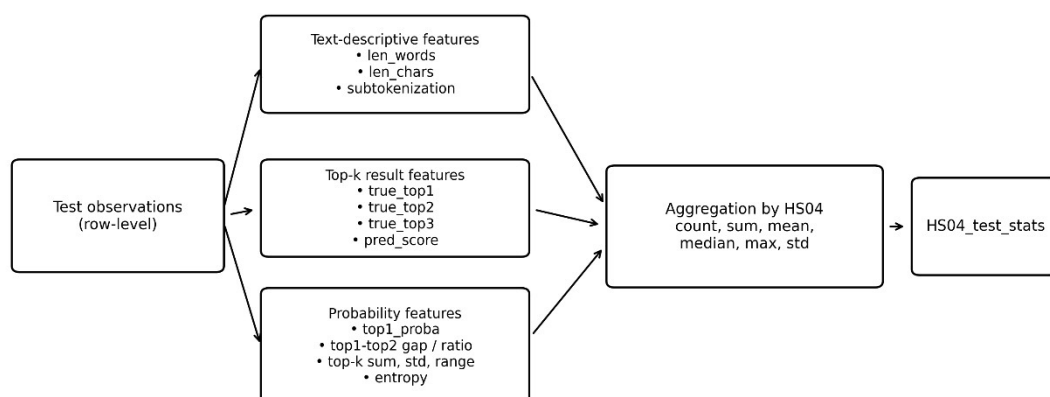
Caracterización del set de prueba/validación

En una segunda etapa, se construyeron tablas agregadas por código HS a partir del conjunto de prueba. Además de las variables descriptivas análogas a las utilizadas en entrenamiento, en este caso se incorporaron variables derivadas de las predicciones generadas por el clasificador.

Entre ellas se incluyeron:

- indicadores de acierto en top-1 a top-3,
- puntajes agregados de desempeño de la predicción,
- probabilidades asociadas a las clases más probables,
- medidas de concentración, dispersión y separación entre probabilidades,
- e indicadores de incertidumbre de la salida del modelo.

Y el proceso de feature engineering se esquematiza en el siguiente diagrama:



Esto permitió caracterizar no solo cómo eran las descripciones de cada código en prueba, sino también cómo respondía el modelo frente a ellas en términos de acierto y confianza. Cabe aclarar que, el puntaje usado (pred_score) asigna 5 puntos al caso cuando el modelo acierta en el Top1, 4 si acierta en el Top2, y así sucesivamente, hasta puntuar en 0 si no se acierta en la sucesión top-1 a top-5.

Integración de la información por código HS04

Una vez obtenidas las estadísticas agregadas del entrenamiento y de la prueba, ambas tablas fueron integradas por código HS. Esta consolidación permitió disponer, para cada clase/código arancelario, de una vista conjunta de:

- su representación en entrenamiento,
- sus características textuales,
- su comportamiento en el conjunto de prueba,
- y su desempeño predictivo agregado.

A partir de esta extensa tabla consolidada de 220 features resulta desafiante analizar una a una las variables que pueden explicar el performance del modelo. Ordenando las agregaciones según la media del score, filtrando los casos donde la cantidad de muestras en el conjunto de prueba es mayor a 5, tenemos un top 10 de códigos:

HS04	True Label count trainset	GOODS DESCRIPTION len chars mean	Subtokenization indicator mean trainset	Subtokenization indicator mean testset	top1 proba mean	Pred score mean
1511	247	25,57	15,75	15,48	0,99	5,00
3303	464	27,50	17,39	19,28	0,91	5,00
3004	1425	36,11	25,37	27,65	0,87	5,00
3901	564	32,86	24,59	23,71	0,93	5,00
8429	582	38,07	22,54	22,68	0,97	5,00
9609	265	28,22	17,84	20,69	0,73	5,00
8711	2916	58,43	31,94	33,81	0,99	4,95
1704	1561	30,53	20,99	32,76	0,91	4,94
4011	1847	34,59	24,37	26,83	0,97	4,92
8701	1386	43,71	19,74	19,75	1,00	4,91

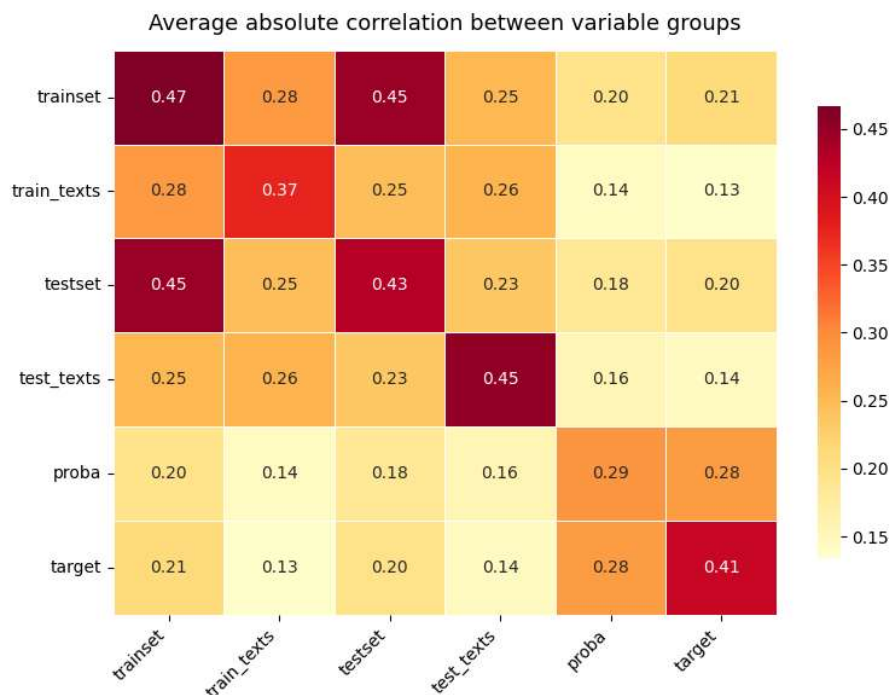
Y los peores 10 HS04:

HS04	True Label count trainset	GOODS DESCRIPTION len chars mean	Subtokenization indicator mean trainset	Subtokenization indicator mean testset	top1 proba mean	Pred score mean
3926	1288	25,61	17,64	21,48	0,60	2,57
7306	297	31,92	20,14	15,07	0,47	2,57
3920	684	29,68	18,06	15,31	0,52	2,56
8415	450	30,30	19,79	19,26	0,58	2,33
8419	701	31,18	17,51	21,52	0,58	1,71
8803	255	25,44	19,25	21,80	0,60	1,43
8473	676	29,68	20,31	24,53	0,59	1,38
8543	619	31,00	19,30	17,00	0,49	1,08
8479	1119	33,42	18,23	26,80	0,50	1,00
7616	612	31,94	17,14	12,33	0,52	0,83

En este punto, resultó interesante evaluar correlaciones entre las features creadas. Sin embargo, una matriz de correlación 220x220 resulta difícil tanto de visualizar, como de interpretar. Por esto, se agruparon las features según los siguientes tipos:

- trainset (conteos y stats del train),
- train_texts (relativas a los textos del train),
- testset (conteos y stats del test),
- test_texts (relativas a los textos del test),
- proba (stats del outputs del modelo) y
- target (stats del performance a evaluar),

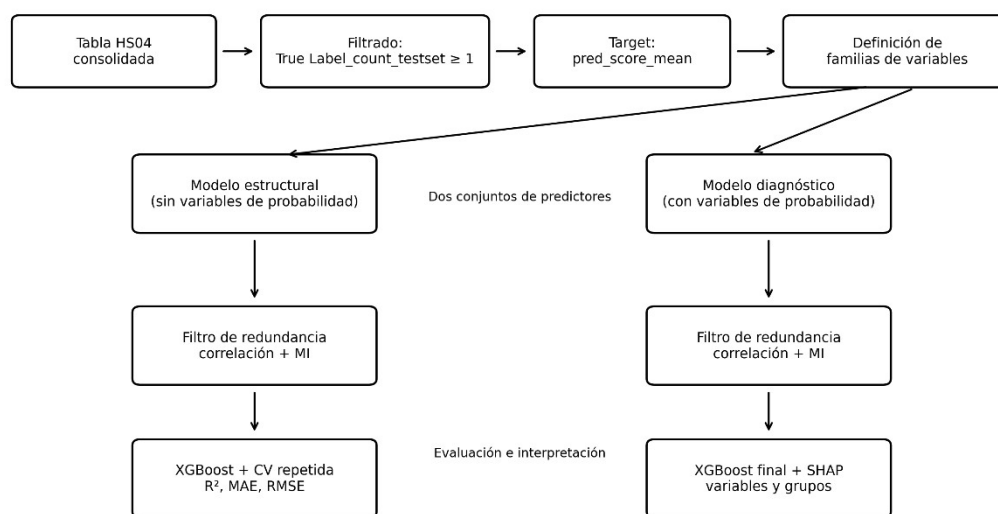
Dando con una matriz que nos anticipa que las features de proba correlacionan en promedio mejor con el performance que el resto. Sin embargo, en esta visualización no se observan diferencias entre el conjunto de features de entrenamiento vs. el de prueba:



Por estas limitaciones, se procedió al uso de técnicas de modelado para explicar el fenómeno con un abordaje holístico y conforme a las buenas prácticas del Machine Learning.

Modelado explicativo

En el siguiente diagrama se explica el proceso utilizado para explicar el desempeño desde variables estructurales (información del conjunto de entrenamiento y de evaluación, sin variables relacionadas con el output/recomendaciones) y diagnósticas (que también incorpora las variables basadas en probabilidades y en incertidumbre del modelo):



El propósito de esta etapa no fue sustituir al clasificador original, sino analizar en qué medida el performance o rendimiento promedio observado para cada código HS04 podía ser explicado a partir de atributos estructurales y diagnósticos de dicha clase.

Filtrado

Como anticipamos, cada fila de la tabla de modelado representó un código HS04, mientras que cada columna resumió una propiedad agregada derivada del conjunto de entrenamiento, del conjunto de prueba y de las salidas del modelo. Con el fin de restringir el análisis a clases efectivamente observadas en la evaluación, se filtró la tabla consolidada conservando únicamente aquellas etiquetas con al menos una observación en el conjunto de prueba (True Label_count_testset >= 1).

Variable objetivo/Target

La variable objetivo seleccionada fue **pred_score_mean**. Esta variable resume, para cada código HS04, la calidad media del posicionamiento de la clase verdadera dentro del ranking top-k generado por el clasificador. A nivel fila, pred_score asigna un puntaje ordinal según la posición en la que aparece la clase correcta entre las predicciones: el valor más alto corresponde a los casos en que la clase verdadera ocupa el primer lugar Top1, valores menores se asignan cuando aparece en posiciones subsiguientes del top-k, y se asigna cero cuando la clase no figura entre las predicciones consideradas. Luego de la agregación por HS04, pred_score_mean opera como una aproximación continua

de la calidad predictiva por clase.

Familias de variables/predictores

A partir de esta tabla consolidada se definieron dos conjuntos alternativos de predictores:

- El primero correspondió a un conjunto de variables de tipo estructural. En términos operativos, este conjunto incluyó todas las variables numéricas o booleanas disponibles, excluyendo: (i) las columnas identificadoras, (ii) la variable objetivo, (iii) las variables de performance directa —por ejemplo, aquellas derivadas de `pred_score_*` o `true_top*`— y (iv) la familia de variables de probabilidad y confianza derivadas de las predicciones. Por lo tanto, el modelo estructural no quedó limitado exclusivamente a atributos del entrenamiento, sino que también pudo incorporar variables descriptivas no probabilísticas agregadas sobre el conjunto de prueba, tales como medidas de longitud textual o indicadores de tokenización.
- El segundo conjunto correspondió a un conjunto de variables diagnósticas. Este conjunto también excluyó identificadores y variables de performance directa, pero sí incorporó las variables basadas en probabilidades y en incertidumbre del modelo. En consecuencia, además de atributos estructurales, este modelo incluyó medidas derivadas de las probabilidades top-k, márgenes, razones, concentración y entropía, permitiendo capturar no solo características de la clase, sino también señales internas del comportamiento del clasificador al emitir sus predicciones.

Esta distinción resulta metodológicamente útil porque separa dos preguntas analíticas complementarias: por un lado, si el desempeño promedio por clase puede explicarse a partir de propiedades estructurales de los datos; por otro, si dicha explicación mejora al incorporar además señales de confianza e incertidumbre provenientes de la salida del modelo.

Reducción de dimensiones

Antes de ajustar los modelos de regresión, se aplicó una etapa de reducción de redundancia entre predictores. Dado que el análisis descriptivo previo generó un conjunto amplio de variables agregadas, muchas de ellas presentaban una elevada correlación y aportaban información parcialmente solapada. Para mitigar este problema, se implementó un filtro basado en la correlación absoluta entre variables. Cuando un par de predictores superaba un umbral prefijado de correlación, la decisión sobre cuál conservar no se realizaba de manera arbitraria, sino utilizando la información mutua respecto de la variable objetivo: se retenía la variable con mayor asociación con `pred_score_mean`, removiéndose la otra. Este procedimiento siguió el enfoque de estimación de información mutua implementado en scikit-learn, basado en desarrollos previos sobre estimadores no paramétricos de entropía e información mutua [26] [27]. Este paso se ejecutó por separado para ambos conjuntos de variables y con umbrales diferentes ($|\text{corr}| > 0,90$ para el modelo estructural & $|\text{corr}| > 0,95$ para el modelo diagnóstico). La razón de esta diferencia es que el conjunto diagnóstico contiene una cantidad mayor de variables derivadas de probabilidades, las cuales tienden a exhibir alta redundancia.

Modelo de regresión

Una vez finalizado el filtrado por correlación, y para ambos tipos de modelos (estructural y diagnóstico), se ajustaron modelos de regresión basados en XGBoost para estimar la relación entre los predictores retenidos y la variable objetivo `pred_score_mean`. XGBoost fue elegido por su capacidad para modelar relaciones no lineales e interacciones entre variables sin requerir supuestos paramétricos fuertes, además de su buen desempeño en problemas tabulares heterogéneos [28]. En este contexto, el modelo cumple el rol de meta-modelo explicativo sobre resúmenes a nivel HS04, y no el de reemplazo del clasificador DistilBERT original.

La evaluación del poder explicativo de estos modelos se realizó mediante validación cruzada Repeated K-Fold, con 5 particiones, 10 repeticiones y semilla fija. Como métricas de desempeño se reportaron R^2 , para cuantificar la proporción de varianza explicada; MAE, para medir el error absoluto medio; y RMSE, para penalizar con mayor peso los desvíos más grandes.

Para esta etapa de evaluación, se utilizó un `XGBRegressor` con `objective = "reg:squarederror"`. Esta parametrización privilegia una evaluación relativamente regularizada de la señal predictiva a nivel de clase:

Parámetro	Valor	Significado
<code>n_estimators</code>	400	Cantidad de árboles en el bosque
<code>max_depth</code>	3	Niveles de profundidad máximo de los árboles
<code>learning_rate</code>	0.03	Tasa de aprendizaje entre árbol y árbol
<code>subsample</code>	0.8	Tasa de filas que recibe cada árbol
<code>colsample_bytree</code>	0.8	Tasa de columnas que recibe cada árbol
<code>reg_alpha</code>	1.0	Regularización L1/Lasso
<code>reg_lambda</code>	1.0	Regularización L2/Ridge
<code>min_child_weight</code>	5	mínima cantidad de casos para formar hojas

Una vez estimado el desempeño por validación cruzada, se ajustó un segundo modelo XGBoost sobre el conjunto completo ya filtrado con el propósito de realizar el análisis de interpretabilidad mediante SHAP.

Interpretación de modelos

Para interpretar los modelos ajustados se utilizó SHAP (SHapley Additive exPlanations), mediante `shap.TreeExplainer`, una herramienta especialmente adecuada para modelos basados en árboles [29]. SHAP asigna a cada predictor una contribución al valor predicho, permitiendo tanto interpretaciones locales como globales. En este trabajo, la importancia global de cada variable se resumió mediante el valor medio absoluto de SHAP, lo que permitió ordenar los predictores según su contribución promedio a la explicación de la calidad o score de la clasificación por cada código HS04.

Adicionalmente, el análisis no se limitó al nivel de variable individual. Se implementó una segunda agregación en la cual las variables fueron agrupadas en familias conceptuales: `train_related`, `test_related`, `proba_related`, `performance_related` y `other`. La importancia de cada grupo se calculó sumando los valores absolutos medios de SHAP de las variables pertenecientes a esa familia. De este modo, fue posible interpretar si la dificultad de clasificación observada para cada código HS04 se asociaba más fuertemente con la

cantidad y estructura de datos de entrenamiento, con características descriptivas del conjunto de prueba o con la estructura de confianza e incertidumbre de las predicciones emitidas por el modelo.

Observaciones sobre el error

Explicación estructural

El modelo explicativo basado en variables estructurales mostró una limitada capacidad predictiva de la performance del modelo DistilBERT. En el esquema de validación cruzada Repeated K-Fold, el desempeño promedio fue de $R^2 = 0,043$ (DE = 0,114), MAE = 1,299 (DE = 0,076) y RMSE = 1,638 (DE = 0,091). Dado que la variable objetivo `pred_score_mean` toma valores en el intervalo [0, 5], estos resultados indican que las variables estructurales, consideradas de manera aislada, capturan solo una fracción reducida de la variabilidad del desempeño medio por código HS04.

En este sentido, la señal explicativa aportada por este conjunto de predictores es débil y presenta además cierta inestabilidad entre particiones (la explicación difiere considerablemente según el grupo de códigos evaluados), como sugiere la dispersión observada en R^2 . Asimismo, los valores de MAE y RMSE muestran que el modelo no resulta suficientemente preciso para estimar el valor puntual de `pred_score_mean` a nivel de clase. En consecuencia, los resultados deben interpretarse en clave exploratoria, como una aproximación a la relación entre atributos estructurales de los datos y la calidad de la clasificación, más que como un modelo predictivo en sí mismo.

Aun con estas limitaciones, el modelo permite examinar qué variables estructurales concentran la señal explicativa disponible dentro del conjunto analizado. Sobre esta base, se utilizó SHAP para descomponer la contribución relativa de los predictores retenidos y analizar qué dimensiones estructurales aparecen más asociadas al desempeño agregado por código HS04.

El análisis de importancia global mediante SHAP mostró que, dentro del modelo estructural, la señal explicativa se concentró principalmente en variables asociadas a la longitud de las descripciones y a la subtokenización, junto con algunas medidas de disponibilidad de muestras por capítulo o clase.

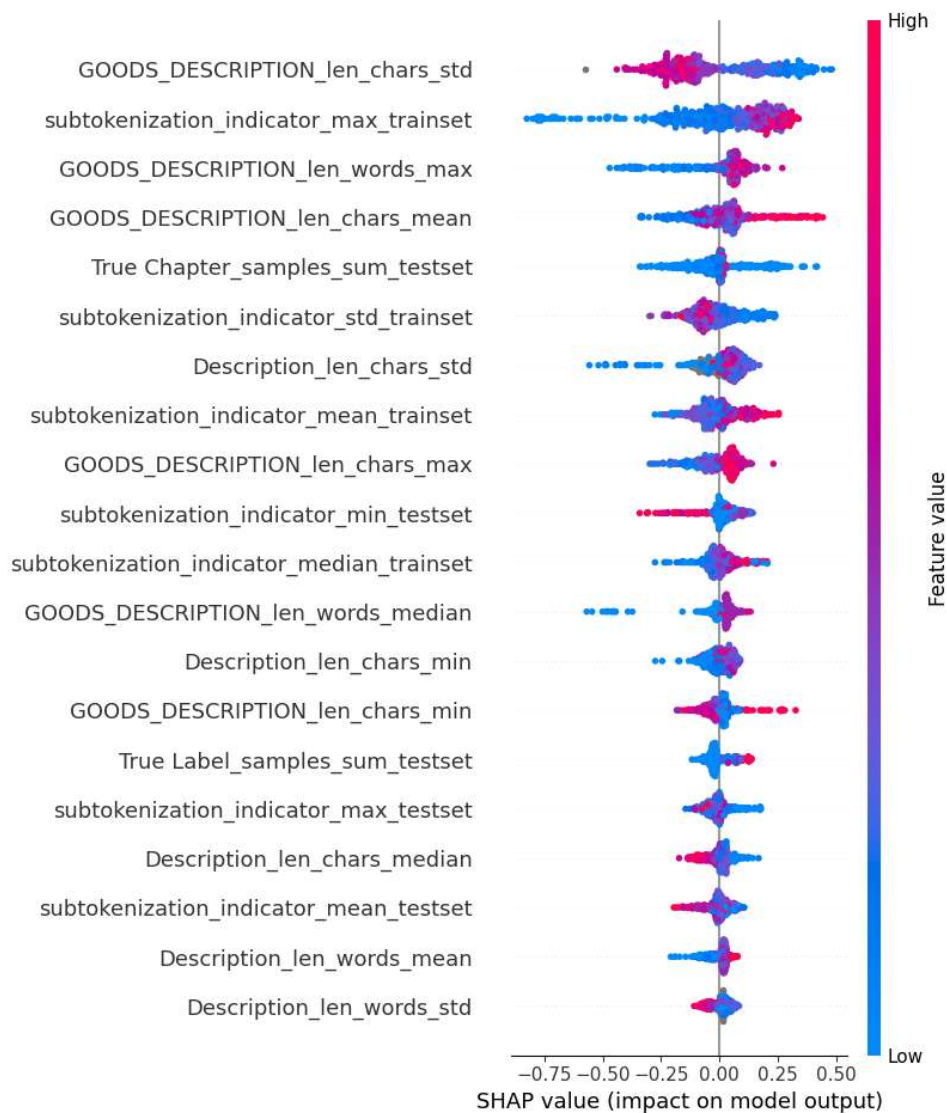
En términos sustantivos, estos resultados sugieren que la heterogeneidad del desempeño entre códigos HS04 se relaciona, dentro del conjunto estructural analizado, con dos dimensiones principales. Por un lado, la **complejidad y variabilidad textual** de las descripciones asociadas a cada partida; por otro, la **cantidad de ejemplos disponibles** o el volumen agregado de observaciones representadas en el corpus, es decir en los conjuntos de entrenamiento y de evaluación (que consiste en una muestra aleatoria del propio entrenamiento). En particular, la presencia reiterada de variables de longitud y subtokenización entre las más influyentes indica que la estructura formal del texto constituye una fuente relevante de señal para explicar diferencias de dificultad de clasificación entre clases.

El análisis por dirección del efecto mostró, además, que algunos predictores estructurales se asocian con un menor desempeño medio cuando toman valores altos. En particular, valores elevados de:

- `GOODS_DESCRIPTION_len_chars_std`,
- `subtokenization_indicator_std_trainset` y
- `subtokenization_indicator_min_testset`

empujan las predicciones del meta-modelo hacia menores valores de `pred_score_mean`. Esta evidencia sugiere que una mayor dispersión en la longitud de las descripciones y una mayor heterogeneidad en la fragmentación del texto en subtokens tienden a asociarse con clases/códigos más difíciles de clasificar. En otras palabras, no solo importa cuánto texto hay en promedio, sino también cuán homogéneo o heterogéneo es ese patrón dentro de cada código HS04.

A nivel agregado por familias conceptuales, la importancia SHAP mostró una clara predominancia de las variables `train_related`, que concentraron el 65,5 % de la señal explicativa total, frente al 34,5 % de las variables `test_related`. Este resultado sugiere que, dentro del modelo estructural completo, la explicación del desempeño por código HS04 depende en mayor medida de atributos vinculados con la información disponible durante el entrenamiento que de las características descriptivas agregadas del conjunto de prueba. En términos metodológicos, ello refuerza la hipótesis de que la cantidad, composición y complejidad del material con el que el clasificador aprendió cada partida constituyen una dimensión relevante para comprender su rendimiento posterior.



Con el fin de desagregar esta señal, se ajustaron además dos modelos estructurales

parciales: uno basado exclusivamente en variables del conjunto de entrenamiento y otro construido únicamente con variables agregadas del conjunto de prueba.

El modelo train only presentó un desempeño levemente superior al del modelo estructural completo en términos de R^2 , con un valor medio de 0,068 (DE = 0,078), mientras que los errores permanecieron en niveles similares (MAE = 1,296; RMSE = 1,619). Dentro de este modelo, las variables más relevantes según SHAP fueron:

- True Label_samples_sum_trainset,
- subtokenization_indicator_median_trainset,
- subtokenization_indicator_std_trainset,
- True Chapter_samples_min_trainset y
- subtokenization_indicator_max_trainset.

Esta configuración refuerza la idea de que la disponibilidad de muestras por clase y la complejidad de tokenización observada en entrenamiento son las principales fuentes de señal estructural cuando se considera exclusivamente la información previa a la evaluación. En particular, la aparición de True Label_samples_sum_trainset como variable dominante sugiere que el volumen de ejemplos observados por clase constituye un determinante relevante, aunque no suficiente por sí mismo, del desempeño promedio posterior.

En contraste, el modelo test only mostró un desempeño claramente más débil, con un R^2 medio de -0,135 (DE = 0,103), acompañado de errores más altos (MAE = 1,413; RMSE = 1,786). Este resultado indica que las variables estructurales derivadas únicamente del conjunto de prueba no contienen señal suficiente para explicar el desempeño medio por HS04 y, de hecho, resultan inferiores a una referencia ingenua basada en la media. Las variables más importantes en este caso fueron:

- True Chapter_count_testset,
- True Chapter_samples_sum_testset,
- subtokenization_indicator_std_testset,
- True Chapter_samples_min_testset y
- subtokenization_indicator_min_testset.

Aunque estas variables conservan cierta coherencia temática —especialmente en torno a volumen y subtokenización—, el mal desempeño global del modelo sugiere que, aisladas del contexto de entrenamiento, no permiten construir una explicación robusta del comportamiento del clasificador.

En conjunto, la comparación entre ambos modelos parciales resulta consistente con la importancia por grupos observada en el modelo estructural general: la información de entrenamiento aporta una señal explicativa más útil que la información estructural proveniente solo del conjunto de prueba. Esto no implica que las variables test-related sean irrelevantes, sino que su capacidad explicativa parece depender en buena medida de ser interpretadas en relación con la estructura previa del entrenamiento.

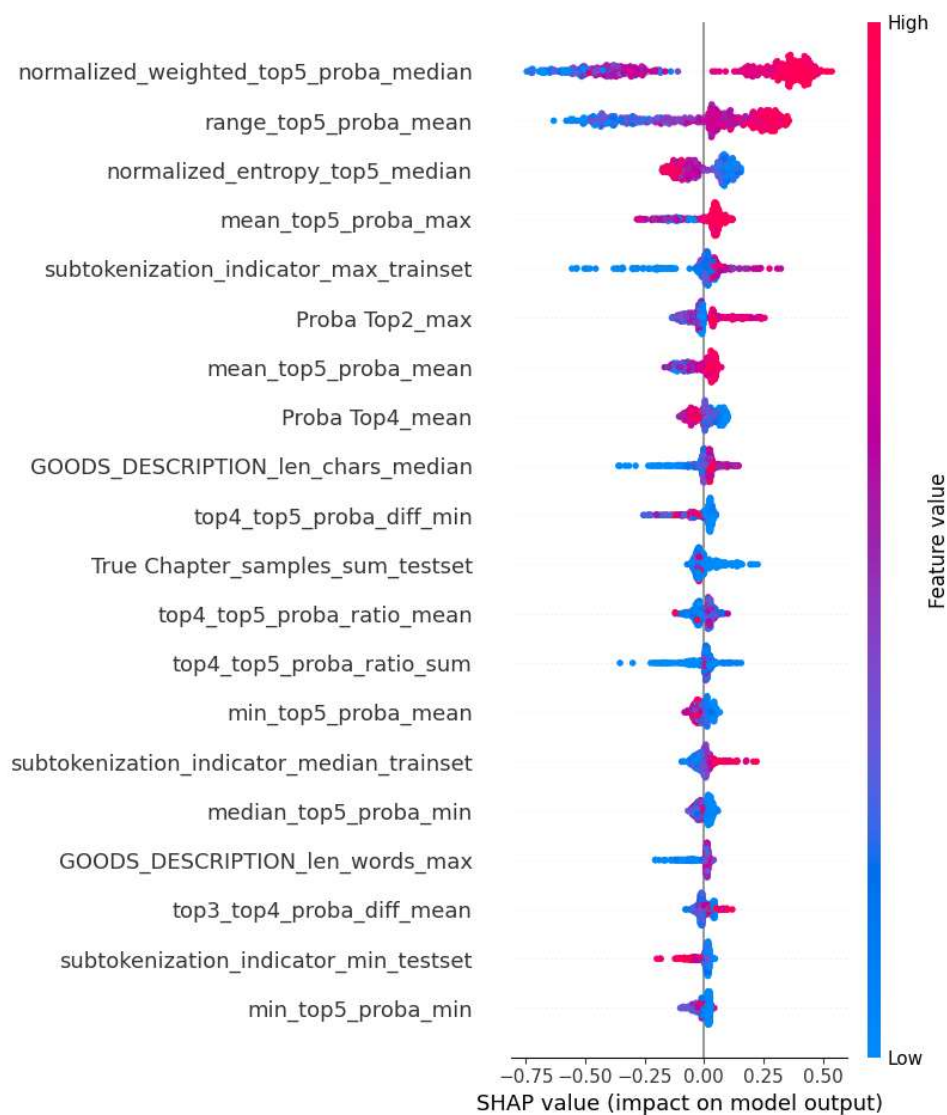
Explicación diagnóstica

El modelo explicativo basado en variables diagnósticas (train_related, test_related y proba_related) presentó un desempeño claramente superior al observado en el modelo estructural. En validación cruzada Repeated K-Fold, los resultados promedio fueron R^2 = 0,326 (DE = 0,099), MAE = 1,004 (DE = 0,082) y RMSE = 1,374 (DE = 0,107). En comparación con el modelo estructural, estos valores indican una mejora apreciable

tanto en la proporción de varianza explicada como en la magnitud del error de predicción.

Este resultado sugiere que las variables diagnósticas —es decir, aquellas derivadas de las probabilidades, márgenes, concentración e incertidumbre de la salida del clasificador— contienen una señal explicativa más estrechamente vinculada con el desempeño medio por código HS04. En otras palabras, la dificultad de clasificación entre partidas parece reflejarse con mayor claridad en el patrón de confianza e incertidumbre que el propio modelo exhibe al emitir sus predicciones que en los atributos puramente estructurales de los datos.

Si bien el poder predictivo alcanzado sigue siendo moderado en términos absolutos, el incremento del R^2 y la reducción de MAE y RMSE muestran que el conjunto diagnóstico permite describir de manera más adecuada la heterogeneidad del error entre clases. Por lo tanto, este modelo ofrece una base más sólida para el análisis interpretativo posterior, orientado a identificar qué señales internas del clasificador resultan más asociadas al desempeño agregado por código HS04.



Los resultados de SHAP confirman esta lectura. Las variables más importantes del

modelo fueron:

- normalized_weighted_top5_proba_median,
- range_top5_proba_mean,
- normalized_entropy_top5_median,
- mean_top5_proba_max,
- subtokenization_indicator_max_trainset,
- Proba Top2_max,
- mean_top5_proba_mean,
- Proba Top4_mean,
- GOODS_DESCRIPTION_len_chars_median y
- top4_top5_proba_diff_min.

La mayoría de estas variables pertenece a la familia de indicadores probabilísticos, lo que muestra que la explicación del error agregado mejora notablemente cuando se incorporan señales internas del comportamiento del clasificador, en particular aquellas relacionadas con la concentración, la dispersión y la incertidumbre del ranking predictivo.

A nivel agregado por grupos, esta dominancia es aún más clara: las variables proba_related concentraron el 79,8 % de la importancia total, mientras que las familias train_related y test_related representaron solo el 13,5 % y el 6,7 %, respectivamente. En términos sustantivos, este resultado indica que las diferencias de desempeño entre partidas se encuentran mucho más estrechamente asociadas a la forma en que el modelo distribuye sus probabilidades entre clases competidoras que a las propiedades estructurales de los datos consideradas de manera aislada.

El análisis de dirección del efecto también aporta evidencia relevante. Se observó que valores altos de:

- normalized_entropy_top5_median,
- Proba Top4_mean y
- top4_top5_proba_diff_min

Empujan el desempeño hacia valores más bajos de pred_score_mean. Esto resulta conceptualmente consistente. En primer lugar, una mayor entropía normalizada en el top-5 indica distribuciones predictivas más dispersas y, por lo tanto, mayor incertidumbre. En segundo lugar, una mayor probabilidad promedio asignada a posiciones inferiores del ranking, como Top4, sugiere menor concentración del modelo en las primeras alternativas. Finalmente, un mayor valor de top4_top5_proba_diff_min parece reflejar situaciones en las que incluso los puestos más bajos del ranking presentan estructuras de separación que no favorecen una identificación clara de la clase correcta. En conjunto, estas variables describen escenarios donde la salida del clasificador es más ambigua o menos decisiva, y por ello se asocian con un peor desempeño agregado por clase.

De manera complementaria, variables como:

- normalized_weighted_top5_proba_median,
- range_top5_proba_mean,
- mean_top5_proba_max o
- Proba Top2_max

Aparecen asociadas a mejoras en el desempeño cuando toman valores altos. En términos generales, esto sugiere que las clases mejor resueltas por el modelo tienden a exhibir

distribuciones predictivas más informativas: mayor concentración en las primeras posiciones, mayores diferencias entre probabilidades relevantes y menor ambigüedad general en la salida.

En síntesis, la comparación entre ambos modelos explicativos muestra que las variables estructurales aportan una señal limitada, mientras que las variables diagnósticas — especialmente aquellas derivadas de probabilidades e incertidumbre— resultan considerablemente más informativas para explicar la variación del desempeño entre códigos HS04. Esto sugiere que una parte sustantiva de la dificultad de clasificación se manifiesta en la propia estructura de la salida del clasificador.

Este resultado abre naturalmente el paso hacia un análisis a nivel de observación individual. Si la distribución de probabilidades contiene información relevante sobre la calidad del desempeño por clase, entonces también es razonable esperar que esa señal permita valorar cuán confiable es una recomendación concreta. En función de ello, la sección siguiente aborda la valoración de la recomendación, con el objetivo de estudiar si las señales internas del modelo pueden transformarse en un scoring de confianza útil para decidir cuándo aceptar automáticamente una predicción y cuándo derivarla a revisión.

Valoración de la recomendación

La sección anterior mostró que una parte relevante del desempeño del clasificador entre códigos HS04 se asocia no solo con atributos estructurales de los datos, sino también con señales derivadas de su propia salida, especialmente probabilidades, márgenes e indicadores de incertidumbre. Esto sugiere que la recomendación emitida por el modelo no debe interpretarse únicamente como una etiqueta propuesta, sino también como una salida cuya estructura interna puede aportar información útil sobre su confiabilidad. Sobre esta base, en esta sección se aborda un problema complementario al de la clasificación: además de analizar qué código HS04 recomienda el modelo, se busca estimar cuán conveniente resulta aceptar esa recomendación en cada observación individual.

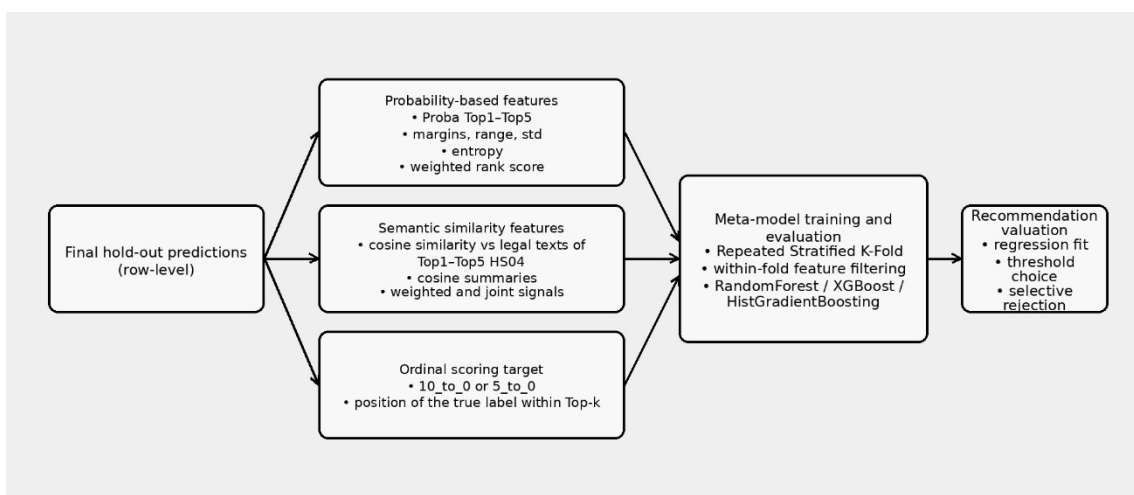
Cabe aclarar que, debido a la acotada muestra de evaluación (con 2677 casos), esto se considera solo un ejercicio conceptual o prueba de concepto, proponiendo los lineamientos para desarrollar un meta-modelo de scoring más robusto y performante cuando se disponga de un conjunto de datos productivo mayor. En términos operativos, ello supondría pasar de una lógica de clasificación obligatoria a una lógica de clasificación asistida, en la que el sistema puede distinguir entre casos suficientemente confiables y casos ambiguos que conviene derivar a revisión humana.

Con este fin, se construye una capa adicional (meta-modelo) de scoring sobre las predicciones del modelo final DistilBERT. Esta capa utiliza variables derivadas de la salida top-k del clasificador y señales de similitud semántica con los textos legales asociados a los códigos recomendados, con el objetivo de estimar la calidad esperada de cada recomendación y evaluar la factibilidad de una política de aceptación automática o rechazo selectivo.

Objetivo y enfoque metodológico

El objetivo de esta etapa fue complementar la evaluación del clasificador final DistilBERT mediante una capa post hoc orientada a valorar la confiabilidad de cada recomendación individual. A diferencia de las secciones previas, centradas en el análisis agregado del error por código HS04, aquí el interés se desplaza al nivel observacional: no solo importa qué código propone el modelo, sino también con qué grado de confianza conviene utilizar esa recomendación.

Metodológicamente, el enfoque no implica reentrenar el clasificador base, sino construir un meta-modelo de regresión sobre sus salidas. Para ello, se parte de las predicciones del modelo final sobre el hold-out y se generan dos familias principales de variables: por un lado, features probabilísticas derivadas de la salida top-k; por otro, features de similitud semántica entre la descripción comercial y los textos legales de los códigos HS04 recomendados. A partir de estas variables se define un score ordinal basado en la posición de la clase verdadera dentro del ranking top-5, considerando dos escalas alternativas. Finalmente, se entrenan y evalúan meta-modelos tabulares bajo validación cruzada repetida, incorporando selección de variables dentro de cada fold para reducir el riesgo de overfitting. En la siguiente figura se observan los pasos para este entrenamiento:



Cabe aclarar que este meta-modelo puede mejorar en futuros desarrollos, siempre que su construcción sea prevista durante el propio entrenamiento de modelo clasificador. En este trabajo se partió de un set de datos acotado, lo que deriva en un set de evaluación también limitado. Por ejemplo, el modelo de scoring podría entrenarse con los outputs/salidas de cada fold/iteración de validación. Luego, la definición metodológica para entrenar este meta-modelo deberá integrarse desde el diseño experimental del clasificador base en futuros desarrollos.

Construcción de variables de confianza

La capa de scoring se construyó a partir de variables derivadas exclusivamente de información disponible al momento de inferencia. El objetivo de estas variables fue capturar, para cada observación individual, distintas señales de confianza asociadas a la recomendación emitida por el clasificador final. En términos generales, el conjunto de

features quedó organizado en dos familias principales: variables basadas en la estructura probabilística de la salida top-k y variables basadas en la similitud semántica entre la descripción comercial y los textos legales de los códigos sugeridos por el modelo.

La primera familia estuvo compuesta por variables de probabilidad. Además de las probabilidades directas asociadas a Top1 a Top5, se construyeron resúmenes orientados a representar distintos aspectos de la distribución predictiva, tales como dispersión, amplitud, separación entre alternativas y grado de concentración. Entre ellas se incluyeron medidas como prob_std, prob_range, prob_margin_1_2, prob_margin_1_3, prob_entropy_norm y prob_weighted_rank_score. En conjunto, estas variables permiten caracterizar si la recomendación del modelo aparece fuertemente concentrada en las primeras posiciones del ranking o si, por el contrario, presenta una distribución más repartida y ambigua entre varias clases competidoras.

La segunda familia estuvo compuesta por variables de similitud semántica con la nomenclatura legal. Para construirlas, se reutilizó el encoder del modelo final con el fin de obtener embeddings tanto de las descripciones comerciales del hold-out como de los textos legales resumidos asociados a los códigos HS04 recomendados en Top1 a Top5. A partir de estos vectores se calcularon similitudes coseno entre la descripción observada y cada una de las alternativas sugeridas, obteniéndose variables como cosine_sim_desc_vs_top1_hs_text a cosine_sim_desc_vs_top5_hs_text. A ello se añadieron resúmenes adicionales, tales como cosine_pred_mean, cosine_pred_std, cosine_pred_range, cosine_margin_1_2, cosine_weighted_by_proba y joint_top1_signal, orientados a sintetizar la consistencia semántica global del conjunto de recomendaciones.

Un aspecto metodológicamente importante es que el conjunto final de variables se restringió a señales **production-safe**, es decir, disponibles al momento de utilizar el sistema. Además, estas variables buscan representar dos dimensiones complementarias de la confianza. Por un lado, la forma en que el clasificador distribuye su probabilidad entre las clases candidatas; por otro, el grado de alineación semántica entre la descripción del producto y los códigos sugeridos. Bajo esta formulación, una recomendación debería considerarse más confiable cuando combina una salida probabilística más concentrada con una mayor coherencia semántica respecto de la nomenclatura legal asociada a las alternativas propuestas.

Definición del score objetivo

La calidad esperada de cada recomendación individual fue definida en función del conjunto de prueba, construyendo una variable ordinal de scoring basada en la posición que ocupa la clase verdadera dentro del ranking top-5 generado por el modelo. Esta formulación permite resumir en una única escala la utilidad operativa de la recomendación emitida, diferenciando entre aciertos en las primeras posiciones del ranking, aciertos en posiciones posteriores y casos en los que la clase correcta no aparece entre las alternativas sugeridas.

Con el fin de evaluar distintas formas de representar esta información, se compararon dos escalas ordinales alternativas. La primera, denominada 10_to_0, asigna valor 10 cuando la clase verdadera coincide con Top1, 9 cuando coincide con Top2, 8 en Top3, 7 en Top4, 6 en Top5 y 0 cuando la clase correcta no figura dentro de las cinco recomendaciones. La segunda, denominada 5_to_0, conserva la misma lógica ordinal,

pero en una escala más compacta: 5 para Top1, 4 para Top2, 3 para Top3, 2 para Top4, 1 para Top5 y 0 fuera del top-5. En ambos casos, el valor del score aumenta a medida que la clase verdadera aparece mejor posicionada en el ranking predictivo.

La diferencia entre ambas definiciones no es meramente formal. La escala 10_to_0 introduce una mayor separación entre los casos en que la clase correcta aparece dentro del top-k y aquellos en que queda fuera de las cinco recomendaciones, enfatizando así la distinción entre recomendaciones utilizables y recomendaciones fallidas. En cambio, la escala 5_to_0 resume la misma estructura ordinal en un rango más compacto, lo que puede resultar favorable desde el punto de vista del ajuste como problema de regresión. Por este motivo, la comparación entre ambas escalas no se planteó únicamente en términos de error predictivo, sino también en función de su utilidad para separar observaciones aceptables de observaciones potencialmente rechazables.

En términos metodológicos, el score definido no debe interpretarse como una probabilidad calibrada de acierto, sino como una medida ordinal de calidad de la recomendación. Su función es proporcionar un target sintético que permita entrenar un meta-modelo capaz de anticipar, a partir de señales de confianza, qué tan valiosa o confiable será la salida del clasificador en cada caso. Bajo esta formulación, el problema deja de ser estrictamente clasificatorio y pasa a entenderse como una estimación de utilidad esperada de la recomendación emitida.

Entrenamiento y validación del meta-modelo

Sobre la base de las variables de confianza definidas previamente, se formularon meta-modelos tabulares de regresión orientados a predecir el score ordinal de cada observación. El objetivo de estos modelos no fue reemplazar al clasificador DistilBERT ni recalibrar directamente sus probabilidades, sino construir una capa auxiliar capaz de anticipar la calidad esperada de la recomendación emitida a partir de señales disponibles al momento de inferencia.

Para cada una de las dos escalas objetivo (10_to_0 y 5_to_0) se evaluaron tres algoritmos de regresión: HistGradientBoostingRegressor, RandomForestRegressor y XGBoostRegressor. Esta comparación permitió analizar no solo si existía señal útil en las variables de confianza, sino también qué tipo de meta-modelo resultaba más adecuado para capturarla en un problema tabular de baja dimensión relativa y con interacciones potencialmente no lineales entre predictores.

Dado que la muestra de evaluación es acotada, la validación no se apoyó en una única partición interna del hold-out final, sino en un esquema de **Repeated Stratified K-Fold** con 5 folds y 10 repeticiones, totalizando 50 particiones por combinación de modelo y escala. Este diseño permitió obtener estimaciones más robustas y menos sensibles a una partición particular de los datos. La estratificación se aplicó sobre la variable ordinal objetivo, con el fin de preservar razonablemente su distribución en los distintos folds de entrenamiento y validación.

Adicionalmente, dentro de cada fold de entrenamiento se incorporó una etapa de filtrado de variables destinada a reducir el riesgo de sobreajuste. En particular, se eliminaron columnas constantes, se imputaron los valores faltantes mediante la mediana calculada sobre el fold de entrenamiento, se removieron predictores altamente correlacionados y se conservaron las diez variables más informativas según un criterio de información mutua. De este modo, la selección de features quedó integrada al propio

proceso de validación y no fue resuelta de manera externa sobre la totalidad de los datos, lo que mejora la consistencia metodológica del experimento.

La evaluación del meta-modelo se realizó mediante las métricas **MAE**, **RMSE** y **R²**. El MAE se utilizó para cuantificar el error absoluto medio de predicción sobre la escala ordinal definida, mientras que el RMSE permitió penalizar con mayor peso los desvíos más grandes. Por su parte, el R² ofreció una medida complementaria de la proporción de variabilidad del score que lograba ser explicada por cada meta-modelo. En conjunto, estas métricas permiten juzgar el problema tanto desde una perspectiva de ajuste continuo como desde la estabilidad general de la señal capturada.

En suma, el entrenamiento y la validación del meta-modelo se diseñaron como una prueba controlada de si las variables probabilísticas y semánticas derivadas del output del clasificador contienen información suficiente para anticipar la calidad de una recomendación individual. Sobre esta base, la sección siguiente presenta los resultados comparativos obtenidos para cada combinación de escala y algoritmo.

Resultados

Los resultados obtenidos confirman que las variables derivadas de la salida del clasificador final DistilBERT contienen señal útil para anticipar la calidad esperada de cada recomendación individual. Esta conclusión se mantiene de manera consistente bajo ambas definiciones del score objetivo (10 - 0 y 5 - 0), así como también a través de los distintos algoritmos de regresión evaluados.

En primer lugar, el análisis de frecuencia de selección de variables dentro de la validación cruzada mostró una alta estabilidad en un subconjunto reducido de predictores. En ambas escalas, las variables más relevantes fueron principalmente probabilísticas (Proba Top2, Proba Top3, Proba Top5, Proba Top4, prob_range) y, en segundo término, variables de similitud semántica con los textos legales (cosine_sim_desc_vs_top1_hs_text, cosine_weighted_by_proba, joint_top1_signal, cosine_pred_mean). Esto sugiere que la confiabilidad de la recomendación depende tanto de la forma en que el clasificador distribuye su masa de probabilidad entre las alternativas como de la coherencia semántica entre la descripción comercial y la nomenclatura legal asociada a los códigos sugeridos.

feature	10 - 5	5 - 0
	selection_pct	selection_pct
Proba Top2	100	100
Proba Top3	100	100
Proba Top5	100	100
cosine_sim_desc_vs_top1_hs_text	100	100
cosine_weighted_by_proba	100	100
joint_top1_signal	100	100
Proba Top4	98	98
cosine_pred_mean	84	82
prob_range	80	80
cosine_pred_std	-	58
cosine_margin_1_2	54	-

En segundo lugar, la comparación de meta-modelos bajo validación cruzada mostró que RandomForest fue el mejor algoritmo en ambas definiciones del target ordinal. Para la escala 10 - 0, alcanzó un MAE medio de 1.954, un RMSE medio de 3.034 y un R^2 medio de 0.302. Para la escala 5 - 0, obtuvo un MAE medio de 1.035, un RMSE medio de 1.527 y un R^2 medio de 0.340. Aunque la escala 5 - 0 exhibe menores errores absolutos, ello se explica en parte por su rango más compacto; por lo tanto, la comparación entre escalas debe complementarse con métricas de utilidad operativa.

Metric	RandomForest		XGBoost		HistGradientBoosting	
	10 - 0	5 - 0	10 - 0	5 - 0	10 - 0	5 - 0
mae_mean	1,954	1,035	1,966	1,039	2,012	1,058
mae_std	0,064	0,038	0,073	0,040	0,079	0,040
rmse_mean	3,034	1,527	3,087	1,553	3,209	1,614
rmse_std	0,095	0,051	0,108	0,055	0,118	0,059
r2_mean	0,302	0,340	0,277	0,317	0,219	0,263
r2_std	0,044	0,044	0,051	0,048	0,058	0,053

Cabe aclarar que, estos modelos se usan en su configuración por defecto, lo que deja lugar a mejoras mediante la optimización de hiperparámetros donde modelos como XGBoost y HistGradientBoosting podrían sacar ventaja sobre el RandomForest.

Luego, al introducir umbrales sobre el score predicho, ambos esquemas mostraron un comportamiento muy similar y favorable. Para esto, se definieron métricas para buscar el mejor ajuste, y poder comparar modelos. Sumando el pred_threshold como umbral de score predicho a partir del cual se acepta automáticamente la recomendación, le siguen:

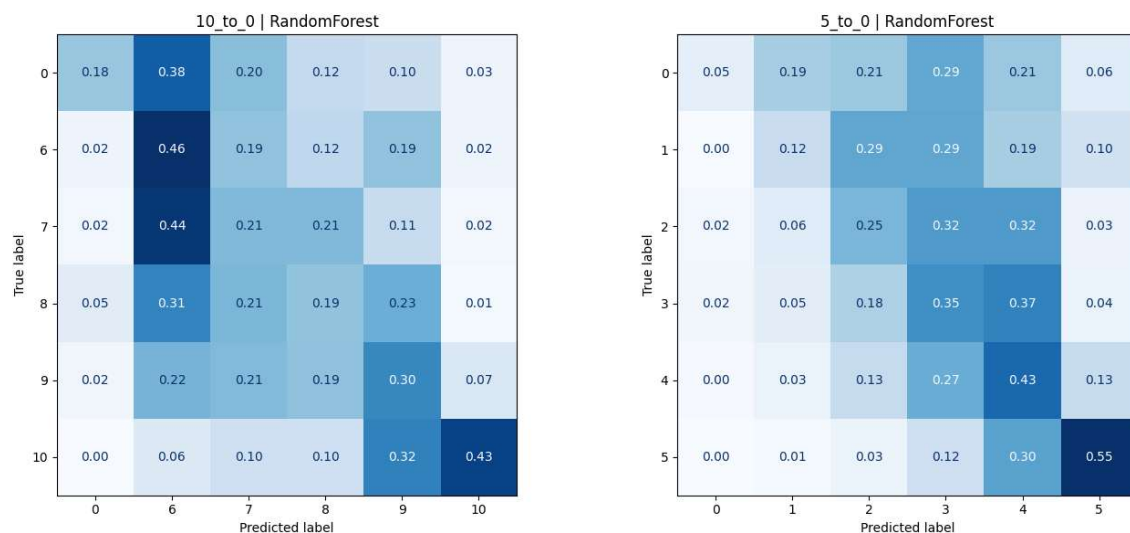
- coverage_pct_mean: porcentaje promedio de observaciones que quedarían aceptadas automáticamente bajo ese umbral.
- precision_good_pct_mean: entre los casos aceptados, porcentaje promedio de casos “buenos”, es decir, con score real > 0.
- precision_lift_vs_baseline_pp_mean: mejora en puntos porcentuales de esa precisión respecto del baseline sin filtrar.
- good_case_recall_pct_mean: porcentaje de los casos realmente buenos que el umbral logra retener.
- bad_case_filter_pct_mean: porcentaje de los casos malos que el umbral logra excluir.
- threshold_utility_score: score pesado de utilidad operativa que combina:
 - precisión $w = 0.35$
 - lift $w = 0.2$
 - recall de buenos casos $w = 0.2$
 - filtrado de malos casos $w = 0.15$
 - cobertura $w = 0.1$

Scoring operative metric	RandomForest		XGBoost		HistGradientBoosting	
	10 - 0	5 - 0	10 - 0	5 - 0	10 - 0	5 - 0

pred threshold	7.5	3.5	7.5	3.5	7.5	3.5
coverage pct mean [%]	68,6	69,1	70,7	70,3	70,7	70,4
precision good pct mean [%]	93,8	93,5	92,9	93,0	92,4	92,6
precision lift vs baseline pp mean	9,99	9,72	9,03	9,20	8,57	8,81
good case recall pct mean [%]	76,7	77,1	78,3	78,0	77,9	77,7
bad case filter pct mean [%]	73,7	72,3	68,7	69,6	66,7	67,9
threshold utility score [%]	68,1	67,9	67,4	67,5	66,7	67,0

En el caso de 10 - 0, la combinación RandomForest + umbral 7.5 logró un threshold_utility_score de 68.097, con una cobertura media de 68.6 %, una precisión sobre casos aceptados de 93.8 %, un lift de 9.99 puntos porcentuales respecto del baseline, una retención de 76.7 % de los casos buenos y un filtrado de 73.7 % de los casos malos. Para 5 - 0, la mejor combinación fue RandomForest + umbral 3.5, con un threshold_utility_score de 67.852, cobertura de 69.1 %, precisión de 93.5 %, lift de 9.7 puntos y filtrado de 72.3 % de los casos malos.

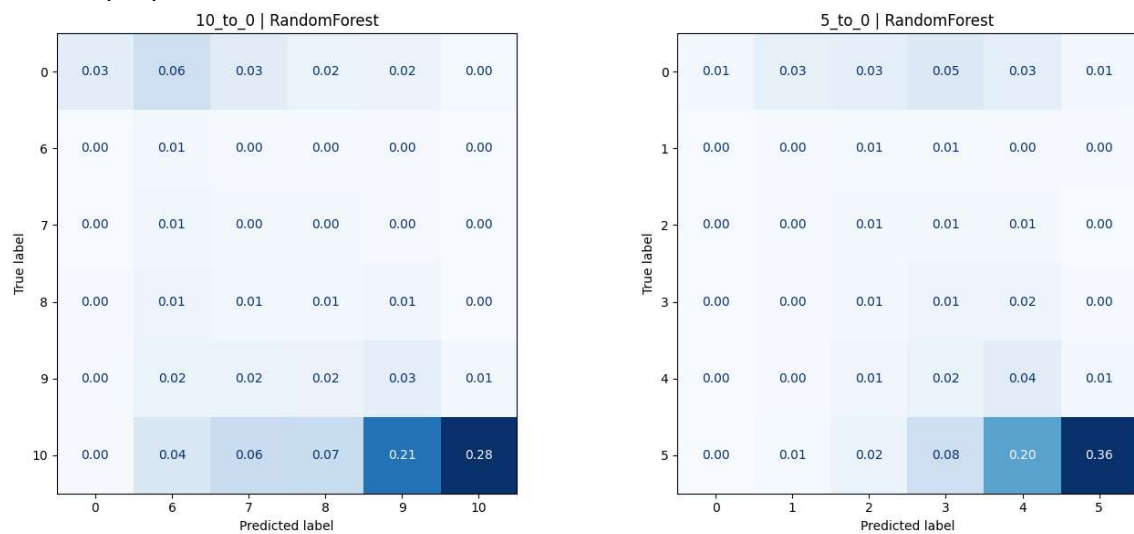
Para comparar los modelos RandomForest en ambas versiones, armamos una matriz de confusión normalizada por True label, que permite ver como se predicen los score de la clasificación según su distribución por etiqueta:



Podemos observar que el caso 10 – 0, si bien logra un mayor lift de la métrica principal del modelo, confunde una porción relevante de los casos con True label < 10, sobre todo para el label 7, que queda 44 % en el rango de scores predichos cercanos a 6.

Para una apreciación más completa de la matriz de confusión, también considerando las distribuciones de los Predicted label, observamos la matriz normalizada en ambos ejes, lo que muestra la bondad de esta técnica, reflejada en las precisiones alcanzadas con el

umbral propuesto:



En conjunto, estos resultados indican que la capa de scoring permite transformar la salida del clasificador en una señal operativamente utilizable para distinguir entre recomendaciones suficientemente confiables y recomendaciones ambiguas. Si bien ambas escalas muestran desempeño muy cercano, la configuración 10 - 0 + RandomForest resulta levemente superior al integrar ajuste predictivo, lift y utilidad bajo umbral. Por este motivo, se la considera la alternativa recomendada dentro de esta prueba de concepto, sin perjuicio de reportar también 5 - 0 por su comportamiento robusto y su mayor simplicidad interpretativa.

Partiendo del modelo DistilBERT final/elegido, según la sección de Comparación, cuyo uso se limita a considerar los códigos/clases recomendadas, con un porcentaje de precisión en Top5 del 83,8 %, este ejercicio conceptual demuestra que hay oportunidades de mejoras en el performance de la recomendación de códigos HS04. Particularmente, mediante meta-modelos que ayuden a definir la confiabilidad de una clasificación, observamos mejoras del orden de 8 a 10 puntos porcentuales, superando los 93 % de accuracy promedio en Top 5. Operativamente, esto podría usarse para descartar clasificaciones automáticas y/o sumar el score a modelos predictivos (en procesos que requieren inferencia rápida u online) o bien proponer interacción con el usuario solicitando una mejor descripción de la mercadería o producto a clasificar.

Conclusiones

La presente tesis abordó el problema de la clasificación arancelaria automática de mercaderías a nivel HS04 a partir de descripciones comerciales en texto libre, utilizando un enfoque basado en técnicas de aprendizaje automático y procesamiento de lenguaje natural. El punto de partida fue un problema de alta relevancia práctica para el comercio internacional y para las administraciones aduaneras: la necesidad de asistir decisiones de clasificación en un contexto caracterizado por grandes volúmenes operativos, descripciones breves y heterogéneas, y una nomenclatura extensa y compleja, cuya correcta aplicación resulta crítica desde el punto de vista normativo y operativo [2] [5] [6]. En este marco, el trabajo se propuso evaluar si un conjunto limitado de datos rotulados, proveniente del proyecto BACUDA de la OMA [5] [12], permitía entrenar modelos con un nivel de desempeño útil para recomendar códigos HS04 en escenarios realistas.

Los resultados obtenidos permiten concluir, en primer lugar, que el problema planteado es abordable mediante enfoques supervisados de NLP, incluso en un escenario de alta complejidad multiclase y fuerte desbalance. La evidencia empírica desarrollada a lo largo del trabajo mostró que tanto los modelos basados en embeddings distribuidos tradicionales como los modelos basados en Transformers son capaces de capturar parte de la señal presente en las descripciones comerciales. Sin embargo, también mostró con claridad que no todos los enfoques ofrecen el mismo nivel de adecuación frente a las exigencias semánticas y operativas del problema.

En ese sentido, una primera conclusión relevante es que los modelos Doc2Vec y FastText, utilizados como baselines, constituyen referencias metodológicamente valiosas y con capacidad predictiva no despreciable, aunque con comportamientos diferenciados frente al tipo de preprocesamiento aplicado. En el caso de Doc2Vec, originalmente propuesto por Le y Mikolov como una extensión de Word2Vec para representaciones densas de textos de longitud variable [8], el trabajo mostró que una limpieza estándar del texto puede deteriorar el rendimiento si elimina información específica relevante para la discriminación entre clases próximas, mientras que la incorporación de n-gramas permite recuperar parte de esa estructura local y mejorar de manera apreciable las métricas top-k. En FastText, por el contrario, el mejor rendimiento se obtuvo con el texto preprocesado, mientras que el agregado explícito de n-gramas no produjo mejoras relevantes. Este comportamiento resulta coherente con la formulación original del método, que ya incorpora información subléxica mediante fragmentos de caracteres [9], y por lo tanto tiende a capturar regularidades morfológicas y ortográficas sin requerir una intervención manual tan intensa como en otros enfoques.

Aun así, el mejor baseline global —FastText con texto preprocesado— fue superado de manera consistente por los modelos basados en DistilBERT, lo cual constituye una segunda conclusión central del trabajo. Este resultado es consistente con la literatura que destaca la superioridad de las representaciones contextuales profundas frente a embeddings estáticos o semiestáticos en tareas de clasificación de texto [10] [11] [13] [14]. Mientras que Word2Vec, Doc2Vec y FastText aprenden representaciones útiles pero relativamente fijas [7] [8] [9], los modelos de la familia BERT, construidos sobre la arquitectura Transformer [13], generan representaciones contextualizadas capaces de modelar relaciones más finas entre términos, materiales, funciones, partes y atributos técnicos del producto [10]. En el dominio de la clasificación arancelaria, donde la

desambiguación suele depender de matices semánticos sutiles y de combinaciones específicas de términos, esta capacidad resulta especialmente relevante.

Dentro de los experimentos con **DistilBERT**, la evidencia mostró que el **grado de ajuste fino del encoder** constituye una dimensión decisiva del desempeño. El esquema más conservador, con encoder congelado (**fixed encoder**), obtuvo resultados claramente inferiores a los regímenes con ajuste parcial o total. A su vez, el fine-tuning completo fue el que ofreció el mejor desempeño global, con una mejora clara respecto de los baselines y también respecto del ajuste parcial. Este hallazgo resulta consistente tanto con el paradigma de adaptación de modelos preentrenados desarrollado en BERT [10] como con la literatura sobre fine-tuning para clasificación de texto [14], y sugiere que, en este problema, no alcanza con utilizar embeddings preentrenados como insumo fijo: es necesario permitir que el encoder reorganice sus representaciones internas para capturar regularidades propias del dominio arancelario.

En particular, la variante FFT11, que combina fine-tuning completo con validación sin bootstrap, emergió como la alternativa metodológicamente más sólida dentro de los ensayos realizados. Si bien sus métricas son muy próximas a las obtenidas por el esquema FFT con bootstrap, mostró una mayor estabilidad entre corridas, lo que vuelve más interpretable la estimación del desempeño. Sobre esta base, puede concluirse que el mejor modelo del trabajo logra un nivel de accuracy top-k compatible con un uso como herramienta de recomendación, confirmando que los modelos compactos derivados de BERT, como DistilBERT, pueden constituir una alternativa de alto valor predictivo y costo computacional razonable [11]. Este punto no es menor en un trabajo aplicado como el presente, donde la discusión no se limita a qué modelo rinde más, sino también a cuál ofrece una relación razonable entre desempeño, costo y viabilidad de despliegue.

Una tercera conclusión importante se desprende del análisis del error. Las métricas globales mostraron que el clasificador final presenta un desempeño elevado en términos promedio; sin embargo, el análisis agregado por código HS04 permitió observar que el rendimiento no es homogéneo entre clases. Existen partidas/clases con una performance prácticamente perfecta y otras donde el modelo exhibe dificultades persistentes, incluso cuando no se trata de clases extremadamente infrecuentes. Esta observación sugiere que la dificultad de clasificación no puede reducirse únicamente a la disponibilidad de muestras, sino que también depende de la heterogeneidad interna de las descripciones, de la proximidad semántica entre clases competidoras y de la calidad informativa del texto disponible. En otras palabras, el problema no es solamente cuantitativo, sino también estructural y semántico.

Los ejemplos revisados en el análisis cualitativo refuerzan esta interpretación. Por un lado, se identificaron errores asociados a descripciones escuetas, ambiguas o directamente defectuosas, lo cual era esperable en un dominio donde abundan abreviaturas, nombres incompletos, siglas y expresiones no estandarizadas. Por otro, también aparecieron errores de alta confianza en casos semánticamente fronterizos, donde la descripción contenía información razonable, pero la distinción entre partidas exigía un nivel de desambiguación especialmente fino. Este hallazgo resulta consistente con antecedentes que destacan la persistente dificultad de la clasificación arancelaria incluso cuando se emplean herramientas inteligentes de apoyo [5] [6] [15] [16] [17]. Por lo tanto, los resultados del trabajo no solo muestran que el problema puede modelarse con buen desempeño promedio, sino también que subsiste un núcleo de dificultad estructural que debe ser comprendido y gestionado, más que simplemente eliminado.

En esta línea, el modelado explicativo desarrollado sobre la tabla agregada por código HS04 aporta una cuarta conclusión relevante. Cuando se intentó explicar el desempeño medio por clase a partir de variables puramente estructurales —vinculadas a cantidad de muestras, longitud de descripciones o complejidad de tokenización—, el poder explicativo obtenido fue limitado. Esto indica que dichos atributos, aunque informativos, capturan solo una fracción modesta de la heterogeneidad observada. En cambio, al incorporar variables diagnósticas derivadas de la propia salida del modelo —particularmente probabilidades, márgenes, medidas de concentración y entropía—, la explicación mejoró de manera apreciable. Este hallazgo sugiere que una parte importante de la dificultad de clasificación no reside únicamente en las propiedades estructurales de los datos, sino que se manifiesta directamente en la forma en que el propio clasificador distribuye su incertidumbre.

Desde el punto de vista metodológico, este resultado es especialmente interesante, porque desplaza parcialmente la interpretación del error desde una lógica centrada en los datos hacia una lógica centrada también en la estructura de la predicción. Dicho de otro modo, la salida probabilística del clasificador no solo sirve para ordenar clases candidatas, sino que también contiene una señal útil para explicar y anticipar la calidad de su desempeño. La predominancia de variables proba-related en el análisis de SHAP refuerza esta lectura. En términos conceptuales, esto es consistente con la idea de que el output de modelos modernos de clasificación puede ser explotado no solo para decidir “qué clase predecir”, sino también para analizar “qué tan confiable es esa decisión”.

Precisamente sobre esta base se construyó la capa de valoración de la recomendación, que constituye una quinta conclusión sustantiva del trabajo. Los resultados de esta prueba de concepto mostraron que, a partir de variables derivadas de la distribución top-k y de medidas de similitud semántica entre la descripción comercial y los textos legales asociados a los códigos recomendados, es posible entrenar un meta-modelo capaz de anticipar la calidad esperada de cada recomendación individual. En particular, el mejor esquema evaluado permitió elevar la precisión de los casos aceptados automáticamente a niveles superiores al 93 %, con coberturas cercanas al 69 %. Si bien esta etapa debe interpretarse con cautela por el tamaño acotado del conjunto de evaluación, la evidencia obtenida es suficientemente clara como para sostener que el valor práctico de estos sistemas no reside solamente en recomendar códigos, sino también en distinguir entre recomendaciones confiables y recomendaciones ambiguas.

Este resultado tiene implicancias importantes para la discusión aplicada de la tesis. En lugar de concebir la clasificación arancelaria asistida como un problema de sustitución completa del juicio humano, los hallazgos invitan a pensarla como un problema de automatización selectiva o clasificación asistida. Bajo este enfoque, el sistema puede automatizar los casos donde la recomendación aparece como suficientemente robusta, y derivar a revisión humana aquellos donde la salida presenta señales de incertidumbre, dispersión o baja coherencia semántica. Esta formulación resulta coherente con la visión de la OMA sobre el uso de inteligencia artificial como tecnología de apoyo a la labor aduanera [5], y también con desarrollos recientes en comercio y compliance que destacan la utilidad de arquitecturas híbridas y de asistencia, antes que de reemplazo total [16] [17].

A nivel más general, la tesis también permite extraer una conclusión de orden metodológico sobre el uso de texto crudo en modelos de la familia BERT. Mientras que en los modelos basados en embeddings tradicionales el preprocesamiento resulta una

fuerza de variación experimental crucial, en DistilBERT la decisión de trabajar con GOODS_DESCRIPTION en crudo se mostró razonable y consistente con la literatura. La tokenización subléxica, la contextualización bidireccional y la capacidad del encoder para explotar información superficial y contextual del texto [10] [11] [25] justifican prescindir de etapas manuales agresivas de limpieza, las cuales podrían incluso remover señales útiles. Este resultado refuerza la idea de que el diseño experimental en NLP no debe trasladar mecánicamente estrategias válidas para modelos clásicos a arquitecturas basadas en Transformers, sino considerar la lógica particular de representación y aprendizaje de cada familia de modelos.

Limitaciones

Los resultados también deben leerse a la luz de las limitaciones del trabajo. En primer lugar, el estudio se desarrolló sobre un conjunto de datos limitado, en idioma inglés y proveniente de una fuente particular (probablemente específica de un país no declarado). Luego, la generalización a otros contextos regulatorios, a otros idiomas y (en especial) a otro país o región, donde el balance de códigos/clases relacionado al volumen y tipo de productos importados y exportados, requiere una rigurosa validación adicional. En segundo lugar, la variable objetivo se limitó a HS04, lo que reduce la complejidad respecto de la clasificación a HS06 o a nomenclaturas regionales o nacionales más desagregadas. En tercer lugar, la prueba de concepto del meta-modelo de scoring se apoyó en un conjunto de evaluación relativamente pequeño, lo que limita su madurez como solución operativa cerrada. Finalmente, si bien el trabajo incorpora análisis del error y modelado explicativo, no agota las posibilidades de explicabilidad del clasificador, especialmente en lo referente al comportamiento interno de las capas Transformer, a la atención y a la dinámica de activación del modelo.

Estas limitaciones, sin embargo, no reducen el valor del aporte realizado, sino que delinean con claridad un conjunto de líneas de trabajo futuro. Entre ellas, se destacan la extensión del enfoque a HS06 o a nomenclaturas regionales/nacionales completas; la incorporación de textos normativos, notas explicativas o catálogos técnicos como fuentes adicionales de señal; el desarrollo de arquitecturas híbridas con recuperación semántica, re-ranking experto o esquemas RAG; la ampliación del meta-modelo de scoring a contextos productivos con mayores volúmenes de validación; y la profundización del análisis de explicabilidad sobre el comportamiento interno de los modelos basados en Transformers. Asimismo, resultaría especialmente valioso estudiar el desempeño del enfoque en escenarios multilingües o directamente sobre flujos de comercio electrónico transfronterizo, donde la calidad de las descripciones suele ser aún más variable y ruidosa.

Cierre

En síntesis, la evidencia desarrollada en esta tesis permite sostener que la clasificación arancelaria asistida por modelos de lenguaje constituye una línea de trabajo técnicamente viable, metodológicamente fértil y operativamente prometedora. Entre los modelos evaluados, DistilBERT con fine-tuning completo se mostró como la alternativa más efectiva, superando con claridad a los baselines construidos con embeddings tradicionales y ofreciendo una relación favorable entre desempeño y costo

computacional [11]. A su vez, el análisis del error y la valoración de la recomendación mostraron que el problema no debe entenderse únicamente como una tarea de clasificación, sino también como una tarea de gestión de incertidumbre y de apoyo inteligente a la decisión.

Por ello, la principal contribución de esta tesis no radica solamente en haber identificado un modelo con buen rendimiento predictivo, sino en haber propuesto una mirada integral sobre el problema: desde la comparación entre familias de modelos, pasando por el análisis estructural y diagnóstico del error, hasta la construcción de una capa adicional de scoring orientada a la utilidad operativa. En conjunto, estos resultados aportan evidencia a favor de que las técnicas contemporáneas de NLP pueden contribuir de manera concreta al desarrollo de asistentes inteligentes para clasificación arancelaria, capaces de mejorar la productividad, priorizar la revisión humana y fortalecer la calidad de las decisiones en contextos reales de comercio internacional.

Bibliografía

- [1] UNCTAD, "Global Trade Update," 2024. [Online]. Available: <https://unctad.org/publication/global-trade-update-december-2024>. [Accessed 10 11 2025].
- [2] WCO - World Customs Organization, "What is the Harmonized System (HS)?," 2025. [Online]. Available: <https://www.wcoomd.org/en/topics/nomenclature/overview/what-is-the-harmonized-system.aspx>. [Accessed 10 11 2025].
- [3] Ministerio de Economía - República Argentina, «Ventanilla Única del Comercio Exterior,» 29 06 2025. [En línea]. Available: <https://www.vuce.gob.ar/>. [Último acceso: 11 7 2025].
- [4] UNCTAD, "Business e-commerce sales and the role of online platforms," 2024. [Online]. Available: <https://unctad.org/publication/business-e-commerce-sales-and-role-online-platforms>. [Accessed 10 11 2025].
- [5] WCO - World Customs Organization, "How Artificial Intelligence (AI) can help Customs in automating HS Classification," 2022. [Online]. Available: <https://www.wcoomd.org/en/media/newsroom/2022/april/how-ai-can-help-customs-in-automating-hs-classification.aspx>. [Accessed 10 11 2025].
- [6] A. Grainger, "Customs Tariff Classification and the Use of Assistive Technologies," *World Customs Journal*, vol. 18, no. 2, 2024.
- [7] T. Mikolov, K. Chen, G. Corrado and J. Dean, "Efficient Estimation of Word Representations in Vector Space," 2013. [Online]. Available: <https://arxiv.org/abs/1301.3781>. [Accessed 10 11 2025].
- [8] Q. V. Le and T. Mikolov, "Distributed Representations of Sentences and Documents," in *International Conference on Machine Learning (ICML)*, 2014.
- [9] P. Bojanowski, E. Grave, A. Joulin and T. Mikolov, "Enriching Word Vectors with Subword Information," *Transactions of the Association for Computational Linguistics*, vol. 5, 2017.
- [10] J. Devlin, M. W. Chang, K. Lee and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *NAACL-HLT*, 2019.
- [11] V. Sanh, L. Debut, J. Chaumond and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," 2020. [Online]. Available: <https://arxiv.org/abs/1910.01108>. [Accessed 10 11 2025].
- [12] WCO - World Customs Organization, "WCO BACUDA Project," 26 10 2025. [Online]. Available: <https://bacuda.wcoomd.org/>. [Accessed 10 11 2025].
- [13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser and I. Polosukhin, "Attention Is All You Need," *Neural Information Processing Systems*, 2017.
- [14] C. Sun, X. Qiu, Y. Xu and X. Huang, "How to Fine-Tune BERT for Text Classification?," in *Chinese Computational Linguistics (CCL 2019), Lecture Notes in Computer Science*, vol. 11856, 2019.
- [15] E. Lee, S. Kim, S. Kim, S. Jung, H. Kim and M. Cha, "Explainable Product Classification

for Customs,” 2023.

- [16] I. Marra de Artiñano, F. Riottini Depetris and C. Volpe Martincus, “Automatic Product Classification in International Trade: Machine Learning and Large Language Models,” 2023. [Online]. Available: <https://publications.iadb.org/en/automatic-product-classification-international-trade-machine-learning-and-large-language-models>. [Accessed 11 10 2025].
- [17] S. Gholamian, G. Romani, B. Rudnikowicz and S. Skylaki, “LLM-Based Robust Product Classification in Commerce and Compliance,” in *CustomNLP4U*, Miami, Florida, USA, 2024.
- [18] K. Liu, T. Liu, F. Wang and R. Pan, “A BERT-based Hierarchical Classification Model with Applications in Chinese Commodity Classification,” 2025. [Online]. Available: <https://arxiv.org/abs/2508.15800>. [Accessed 10 11 2025].
- [19] B. Judy, “Benchmarking Harmonized Tariff Schedule Classification Models,” 2024. [Online]. Available: <https://arxiv.org/abs/2412.14179>. [Accessed 10 11 2025].
- [20] Y. S. D. Xie, «Text classification in shipping industry using unsupervised models and Transformer based supervised models,» 2022.
- [21] L. V. W. L. W. T. Tunstall, «Text Classification,» de *Natural Language Processing with Transformers*, O'Reilly, 2022, pp. 21-54.
- [22] Z. Harris, *Distributional structure*, Word, 1954.
- [23] Y. S. H. S. J.-S. M. F. & G. J.-L. Bengio, «Neural probabilistic language models,» *Innovations in Machine Learning*, p. 137–186, 2006.
- [24] T. C. K. C. G. & D. J. Mikolov, «Efficient estimation of word representations in vector space,» 2013. [En línea]. Available: <https://arxiv.org/pdf/1301.3781>. [Último acceso: 15 Marzo 2026].
- [25] K. K. U. L. O. M. C. D. Clark, «What Does BERT Look At? An Analysis of BERT's Attention,» 2019. [En línea]. Available: <https://arxiv.org/abs/1906.04341>. [Último acceso: 2026 marzo 16].
- [26] H. S. y. P. G. A. Kraskov, «Estimating Mutual Information,» *Physical Review E*, vol. 69, nº 6, 2004.
- [27] B. C. Ross, «Mutual Information between Discrete and Continuous Data Sets,» *PLOS ONE*, vol. 9, nº 2, 2014.
- [28] T. C. y. C. Guestrin, «XGBoost: A Scalable Tree Boosting System,» *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785-794, 2016.
- [29] S. M. L. y. S.-I. Lee, «A Unified Approach to Interpreting Model Predictions,» *Advances in Neural Information Processing Systems 30*, pp. 4765-4774, 2017.